

Constitution Assistant

Understanding the Codebase

Documentation for the Constitution of Nepal RAG System

Contents

- 01 Constitution Assistant — System Overview
- 02 Constitution Assistant — Repository Tour
- 03 Architecture & Module Dependencies
- 04 Application Startup
- 05 Request Lifecycle — `POST /api/v1/ask` (with LLM)
- 06 Domain Model
- 07 Important Classes & Modules
- 08 Call Graph — Most Important Execution Paths
- 09 Data Flow — The User Query
- 10 State Management
- 11 Design Patterns
- 12 Code Reading Guide
- 13 Deep Dive — File-by-File Analysis
- 14 Function-by-Function: `backend/src/core/search\_engine.py`
- 15 Code Review — Staff Engineer Review
- 16 Architecture Critique — Pre-Release Review
- 17 Teach Me Like the Author — Design Rationale

Constitution Assistant — System Overview

1. What problem does this project solve?

Citizens, lawyers, and students need to ask natural-language questions about the Constitution of Nepal (2072 / 2015) and get grounded, citable answers — not LLM hallucinations. The project replaces manual legal research with a specialized search + RAG system that returns both the relevant constitutional provisions and an AI-generated answer strictly limited to those provisions.

2. Who are the users?

- General public — Nepali citizens asking about their constitutional rights
- Legal professionals — lawyers and judges researching provisions
- Students & academics — studying constitutional law
- Administrators — managing the system

3. What are the major capabilities?

Capability	Description
Hybrid Legal Search	Custom BM25 + term proximity + title boost retrieval engine tuned for legal text
RAG (Retrieval-Augmented Generation)	Local LLM (Ollama) answers grounded strictly in retrieved articles
Synonym Expansion	44 legal synonym groups (e.g., arrest/detention/custody) to improve recall
Multi-stage Reranking	RRF fusion + MMR diversity + rule-based boost for precision and diversity
Article Promotion	Clause/sub-clause results merged into full articles with matched-clause tracking
Streaming Responses	SSE-based real-time token delivery for LLM answers
User Authentication	JWT-based registration, login, logout with token version invalidation
Chat History	Full CRUD with pagination, ownership enforcement
Graceful Degradation	Falls back to retrieval-only if Ollama is unavailable
Offline Ingestion Pipeline	Preprocesses nested JSON constitution into search indexes

4. What are the major components/modules?

```

Constitution Assistant
├─ Backend (Flask / Python 3.13)
│   ├── Routes (Blueprints)      — HTTP endpoints
│   ├── Controllers              — Input validation, orchestration
│   ├── Services                 — QA orchestration, user/message/article CRUD
│   ├── IR Engine (src/core/)    — BM25, proximity, search, reranker
│   ├── RAG Layer (src/llm/)     — Retrieval, prompt formatting, Ollama client
│   ├── Models (ODM)             — User, Message, ReferencedArticle
│   ├── Preprocessing Scripts    — Flatten JSON → build indexes
│   └─ Config                   — MongoDB connection singleton
├─ Frontend (React 19 / Vite 8)
│   ├── Pages                   — Home, Login, Register, History, etc.
│   ├── Components              — SearchBar, ResultDisplay, ArticleCard, etc.
│   ├── UI Primitives           — Button, Input, Card, Spinner, etc.
│   ├── Auth Context            — JWT management, login/logout
│   └─ Streaming Hook           — SSE reader for real-time answers
└─ Infrastructure
    ├── MongoDB 8               — Users, messages, referenced articles
    └─ Ollama (optional)        — Local LLM inference (default: qwen3:8b)

```

5. How does data flow through the system?

Offline path:

```

nepal_constitution.json (nested)
  → flatten_constitution.py → ~700 flat documents
  → IndexBuilder → tf_index.json + pos_index.json + doc_stats.json

```

Online Q&A path:

```

User query (plain English)
  → SearchEngine: tokenize → synonym expansion → BM25 + proximity + title boost
  → Reranker: RRF fusion → MMR diversity → rule boost
  → Article Promotion: clauses → merged articles (top 8)
  → [Optional] Ollama LLM: truncated context → grounded answer
  → MongoDB persistence: upsert articles, save message
  → Response: provisions + citations + [optional] LLM answer

```

6. Which technologies and frameworks are used?

Layer	Technology
Backend	Python 3.13, Flask 3.x, mongoengine ODM, spaCy, PyJWT, bcrypt
Frontend	React 19, Vite 8, Tailwind CSS v4, react-router-dom, react-markdown
Database	MongoDB 8
IR Engine	**Custom Python** — no Elasticsearch/Solr
LLM	Ollama (local), default model `qwen3:8b`
Auth	JWT (HS256, 12h expiry) + token versioning

7. What architectural style is used?

Layered monolith with a service-oriented internal design. The backend is a single Flask process with clear separation:

- Routes layer (thin blueprints) → Controllers (validation) → Services (orchestration) → Core engine (pure logic, no Flask deps)

- The IR engine (src/core/) is framework-agnostic and independently testable
- Frontend is a separate SPA communicating via REST + SSE

8. Which parts seem most important?

1. src/core/search_engine.py — the custom hybrid search engine (BM25 + proximity + title boost); it's the heart of retrieval quality
2. src/core/reranker.py — RRF + MMR + rule boost determines final ranking
3. src/llm/rag_repository.py — bridges retrieval and LLM; handles article promotion, context truncation, and Ollama integration
4. controllers/api_controller.py — the main ask() and ask_stream() endpoints plus message persistence
5. The three index files (tf_index.json, pos_index.json, doc_stats.json) — generated offline, loaded at startup, essential for all retrieval

9. Which parts are supporting infrastructure?

- User authentication (routes/controllers/models) — required but not core to the constitutional Q&A mission
- Chat history CRUD — convenience feature
- Frontend UI components — replaceable; the API is the real product
- Preprocessing scripts — run once, produce artifacts; not in the critical runtime path
- json_builder_tools/ — browser JSON editing helpers, mostly tooling
- Tests (backend/temp/tests/) — verification, not production

10. Architecture diagram

FRONTEND (React 19 + Vite 8)

AuthProvider (JWT → Bearer header)
useAskStream (SSE reader for streaming tokens)

Pages: Home|Login|Register|History|MessageDetail|About|HowItWorks|
Components: NavBar|MainSearchBar|ResultDisplay|ArticleCard
UI: Button|Input|Toggle|Card|Alert|Badge|Spinner|Dialog

HTTP/SSE (JWT auth)

BACKEND (Flask / Python 3.13)

ROUTES (api_routes.py, auth_routes.py)
/ask | /ask-stream | /messages | /auth/*

CONTROLLERS (api_controller.py, auth_controller.py)
Validation ← JWT decorator → _persist_message()

SERVICES (qa_service.py, user_service.py, etc.)
QAService.answer_query() → RAGWorkflow.ask()

RAG LAYER (src/llm/)

RAGRepository (retrieve + promote + call_llm)	RAGFormatter (prompts)	ollama_llm.py (Ollama client factory)
--	---------------------------	---

IR ENGINE (src/core/) – PURE PYTHON, NO FLASK DEPS

BM25Scorer (k1=1.5 b=1.0)	Proximity Scorer (pairs)	SearchEng ine (hybrid)	Reranker (RRF+MMR +boost)
---------------------------------	--------------------------------	------------------------------	---------------------------------

QueryExpander

MISC: Models (User/Message/Article) | Config (MongoDB)

MongoDB 8	Ollama (opt)	Data/Indexes
ECIRAS DB	qwen3:8b	tf_index.json
users msgs articles		pos_index.json
		doc_stats.json

11. 5-minute pitch to a senior engineer

Constitution Assistant is a RAG system for the Constitution of Nepal, built as a layered Flask monolith serving a React SPA.

The interesting part is the custom IR engine in `src/core/` — it doesn't use Elasticsearch or any off-the-shelf search library. Instead, it implements BM25 ($k_1=1.5$, $b=1.0$) with a proximity scoring heuristic (ordered term pairs, capped at 30-token windows) and title boosting (5x per title match). The top 30 candidates go through a 3-stage reranker: RRF fusion ($k=60$) of the BM25/proximity/title signals, then MMR diversity ($\lambda=0.5$) via BM25 cosine similarity, then a rule-based boost factoring in document-level, part-level, and hierarchical-level multipliers. Clause/sub-clause results are promoted to article level post-reranking with matched-clause tracking.

Key design choices: $b=1.0$ (full length normalization) for varied-length legal provisions; two separate text processors (lemmatized+stopwords-removed for BM25, raw+stopwords-preserved for proximity); synonym expansion from 44 legal synonym groups applied only to BM25 tokens; and Ollama is optional — the system degrades gracefully to retrieval-only.

The backend delivers via two endpoints: `/ask` (JSON) and `/ask-stream` (SSE), both JWT-protected, both persisting results to MongoDB through a fire-and-forget `_persist_message()` hook that never breaks the response.

The frontend is standard React 19 with Tailwind v4 — auth context for JWT, SSE hook for streaming, reusable UI primitives. Nothing exotic there.

Biggest piece of technical debt: the `async def` methods in `message_service.py` and `article_service.py` look like they should be awaitable but aren't — they use synchronous mongoengine calls, so they're misleading but functionally correct. The service layer has more development potential, e.g., admin APIs exist in `UserService` but no admin blueprint wires them up.

Constitution Assistant — Repository Tour

Root Directory

Purpose: Project container — orchestration, documentation, and metadata lives here.

File	Purpose
`Makefile`	Quick-start commands (`make run`, `make backend`, `make frontend`)
`AGENTS.md`	Instructions for AI coding assistants navigating the repo
`.prettierrc`	Code formatting config (frontend concern, placed at root)

The root is infrastructure — no business logic. It ties the two apps together.

backend/ — Flask API + IR Engine

Purpose: Everything server-side. The core of the system.

Subdirectories (in order of a request's journey):

routes/ — API surface

- Files: `api_routes.py` (Q&A endpoints), `auth_routes.py` (auth endpoints)
- Purpose: Flask Blueprints — thin URL-to-function mapping, no logic
- Interactions: Calls controllers
- Nature: Infrastructure

controllers/ — Request validation and orchestration

- Files: `api_controller.py` (`ask/ask-stream/message` CRUD), `auth_controller.py` (`register/login/logout`), `decorators.py` (JWT `@token_required`)
- Purpose: Validate input → delegate to services → format responses. `_persist_message()` lives here.
- Interactions: Calls services
- Nature: Thin orchestration layer (some business logic in validation rules)

services/ — Business logic facades

- Files: `qa_service.py` (lazy singleton `RAGWorkflow`, `answer_query()`), `user_service.py`, `message_service.py`, `article_service.py`
- Purpose: Higher-level orchestration that doesn't belong in pure engine code but isn't HTTP-aware either
- Interactions: `qa_service.py` calls into `src/llm/` and `src/core/`; the others talk to models (MongoDB)
- Nature: Business logic

src/core/ — The IR Engine (no Flask dependency)

- Files: `search_engine.py`, `bm25_scorer.py`, `proximity.py`, `reranker.py`, `text_processor.py`, `engine_factory.py`, `query_expander.py`, `index_builder.py`, `document.py`
- Purpose: Pure Python BM25 + proximity + reranking. This is the project's crown jewel.
- Interactions: Called by `src/workflows/` and `src/llm/`. Zero Flask/HTTP coupling.
- Nature: Core business logic

src/llm/ — RAG Layer

- Files: `rag_repository.py` (`retrieval` + `Ollama client` + `article promotion`), `rag_formatter.py` (`prompt construction`), `rag_workflow.py` (`orchestrator`)
- Purpose: Bridges retrieval engine and LLM; handles context truncation, prompt building, retry logic
- Interactions: Uses `src/workflows/` for retrieval, calls Ollama externally
- Nature: Business logic

src/workflows/ — Orchestration Workflows

- Files: retrieval_workflow.py (search → rerank pipeline), ingestion_workflow.py (offline index building)
- Purpose: Higher-level compositions of core engine components
- Interactions: Calls src/core/ components
- Nature: Business logic

src/constants/ — Static data

- Files: contraction_map.py (57 contractions), stopwords.py (~120 stopwords)
- Nature: Infrastructure/data

models/ — MongoDB ODM (mongoengine)

- Files: user_model.py, message_model.py, referenced_article_model.py
- Interactions: Used by services for persistence
- Nature: Infrastructure (data mapping)

config/ — Connection management

- Files: db_connect.py (MongoDB singleton), log_config.py
- Nature: Infrastructure

preprocessing_scripts/ — One-time offline pipeline

- Files: run_ingestion.py (full pipeline), flatten_constitution.py (nested JSON → flat docs), build_index.py (flat docs → indexes)
- Interactions: Writes to data/output/. Called manually, not at runtime.
- Nature: Infrastructure/tooling

data/ — Input data and generated artifacts

- Files: nepal_constitution_new.json (nested input), synonyms.json (44 groups), output/tf_index.json, output/pos_index.json, output/doc_stats.json (generated)
- Nature: Data

temp/tests/ — Test suite

- Nature: Verification

frontend/ — React 19 SPA

Purpose: Browser UI.

Subdirectories:

Folder	Purpose
`src/pages/`	Page-level components (Home, Login, Register, History, MessageDetail, About, HowItWorks, NotFound)
`src/components/`	Reusable UI (Navbar, MainSearchBar, ResultDisplay, ArticleCard, Suggestion, ConfidenceBadge, ProtectedRoute) + `ui/` primitives (Button, Input, Toggle, Card, Alert, Badge, Spinner, Pagination, Dialog)
`src/context/`	Auth context (`AuthProvider.jsx`, `useAuth.js`) — JWT stored in localStorage
`src/hooks/`	`useAskStream.js` — SSE streaming hook
`src/api/`	`client.js` — fetch wrapper with Bearer token and timeout
`src/utils/`	`highlight.jsx` — term highlighting in article cards

`main.jsx` `	Mount point
`App.jsx`	Router + AuthProvider wrapper

Interactions with backend: REST calls to Flask API with JWT Bearer auth. SSE for streaming. Nature: UI with no business logic — all intelligence is server-side.

docs/ — Documentation

Files: PROJECT_DOCUMENTATION.md (the source of truth), api_docs.md, algorithm_details.md, design.md, SYSTEM_OVERVIEW.md, mermaid/ (diagrams)

Other Top-Level Directories

Directory	Purpose
`postman/`	API testing collection
`references/`	Academic PDF papers
`temp/`	Scratch notes, temporary files

Where everything lives

Question	Answer
Where does the application start?	`backend/app.py` — Flask factory, `app.run()`.
Where do requests begin?	`backend/routes/api_routes.py` — Blueprint maps URLs to controllers.
Where does business logic live?	`backend/src/core/` (IR engine), `backend/src/llm/` (RAG), `backend/src/workflows/` (orchestration), `backend/services/qa_service.py`.
Where does persistence happen?	`backend/services/message_service.py` and `article_service.py` → mongoengine models → MongoDB.
Where does configuration live?	`backend/.env` (secrets/URLs), `backend/config/db_connect.py` (MongoDB), `backend/.env.sample` (shape reference).
Where do tests live?	`backend/temp/tests/` (pytest).

If I only read 15 files to understand the project, these are the files

#	File	Why
1	`docs/PROJECT_DOCUMENTATION.md`	Everything in one place
2	`backend/app.py`	Entry point, factory, wiring
3	`backend/routes/api_routes.py`	API surface — what endpoints exist
4	`backend/controllers/api_controller.py`	Request handling — ask, ask-stream, persist
5	`backend/controllers/decorators.py`	JWT auth enforcement

6	`backend/services/qa_service.py`	Q&A orchestration and decision matrix
7	`backend/src/core/search_engine.py`	Hybrid search (BM25 + proximity + title boost)
8	`backend/src/core/bm25_scorer.py`	BM25 scoring
9	`backend/src/core/proximity.py`	Term proximity scoring
10	`backend/src/core/reranker.py`	RRF + MMR + rule boost
11	`backend/src/core/text_processor.py`	Tokenization pipeline
12	`backend/src/llm/rag_repository.py`	Retrieval + article promotion + Ollama client
13	`backend/models/user_model.py`	User schema + auth methods
14	`backend/models/message_model.py`	Message schema with article references
15	`frontend/src/pages/HomePage.jsx`	Main user-facing interface — search, toggle, streaming display
(bonus)	`backend/src/workflows/retrieval_workflow.py`	Connects search → rerank into one pipeline
(bonus)	`frontend/src/hooks/useAskStream.js`	SSE streaming — the most interesting frontend code

Architecture & Module Dependencies

Dependency Diagram



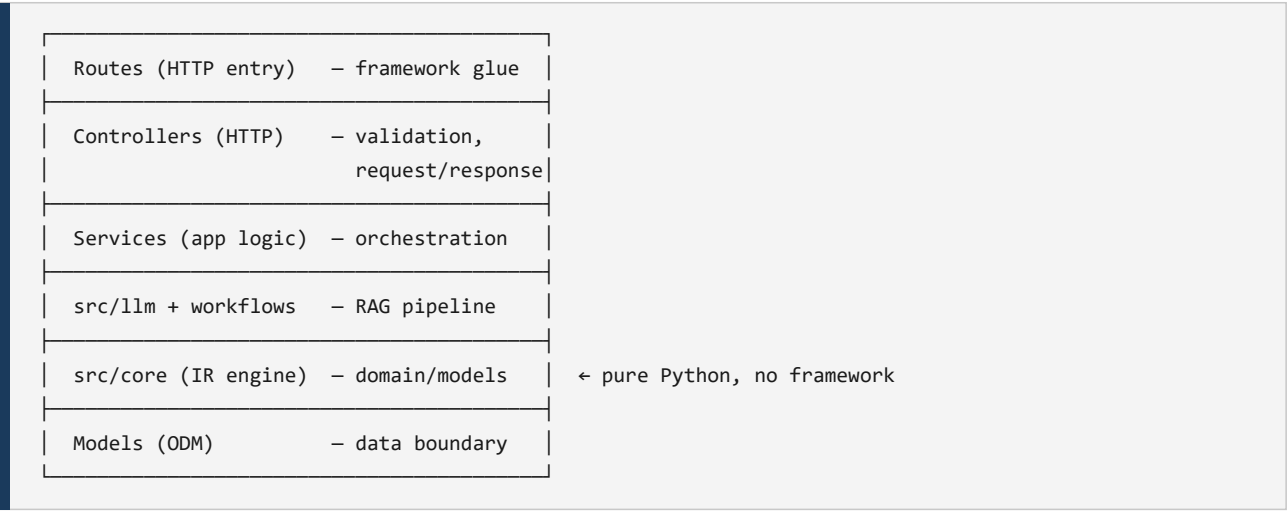
Why Each Dependency Exists

Edge	Reason
Routes → Controllers	Flask Blueprints wire URL paths to controller functions; thin pass-through layer
Controllers → Services	Controllers handle HTTP concerns (parse request, return jsonify); Services hold business logic

Controllers → Models	<code>`auth_controller`</code> and <code>`decorators`</code> need direct model access for JWT user lookups and <code>token_version</code> checks
Services → Workflows/LLM	<code>`QAService.answer_query()`</code> orchestrates the entire RAG pipeline — it's the application service
Services → Other Services	<code>`QAService`</code> calls <code>`ArticleService`</code> and <code>`MessageService`</code> for persistence
src/llm → src/workflows	<code>`RAGRepository.retrieve()`</code> delegates to <code>`RetrievalWorkflow`</code> for search+rerank
src/workflows → src/core	<code>`RetrievalWorkflow`</code> composes <code>SearchEngine</code> + <code>Reranker</code> ; <code>`IngestionWorkflow`</code> composes <code>IndexBuilder</code>
src/core/* → src/constants	<code>`TextProcessor`</code> needs contraction/stopword data for normalization
Frontend Pages → Components	Page composition: pages import feature components like <code>`MainSearchBar`</code> , <code>`Resultdisplay`</code>
Frontend Components → UI	Shared presentation primitives (Button, Card, Spinner) are consumed by feature components

Architectural Style

Layered Architecture with Clean Architecture influence in the backend.



The `src/core/` package is the cleanest domain boundary — zero imports from Flask, mongoengine, or any framework. It only depends on Python stdlib and spaCy. This is the hallmark of Clean Architecture: the domain/inner layer is framework-agnostic.

Tightly Coupled Modules

Module	Reason
<code>**`services/qa_service.py`**</code>	Importing hub — depends on <code>`engine_factory`</code> , <code>`reranker`</code> , <code>`retrieval_workflow`</code> , <code>`rag_repository`</code> , <code>`rag_workflow`</code> , <code>`rag_formatter`</code> , <code>`article_service`</code> , <code>`message_service`</code> . Nearly every backend package converges here.

<code>**src/llm/rag_repository.py*</code>	Spans two layers — imports from <code>src/workflows</code> and directly constructs <code>Ollama</code> clients
<code>**controllers/api_controller.py**</code>	Directly imports both <code>QAService</code> and <code>MessageService</code>

Loosely Coupled Modules

Module	Reason
<code>**src/core/bm25_scorer.py**</code>	Pure math function — no imports beyond <code>math</code> . Swap-in replacement.
<code>**src/core/reranker.py**</code>	Same — pure algorithm with zero local imports.
<code>**src/core/proximity.py**</code>	Self-contained pair scoring.
<code>**src/core/query_expander.py**</code>	Data → logic, no internal coupling.
<code>**src/llm/rag_formatter.py**</code>	Pure string builder — no dependencies on models, services, or workflows.
<code>**frontend/src/components/ui/***</code>	Every UI primitive is a self-contained pure presentational component.

Reusable Libraries

- `src/core/` — The entire IR engine (BM25, proximity, reranker, search engine, text processor) is a standalone library. It could be published as a Python package independently. It has no awareness of HTTP, databases, or LLMs.
- `src/constants/` — Contraction map and stopword list are reusable data artifacts.
- `frontend/src/components/ui/` — The 9 UI primitives (Button, Input, Card, etc.) are generic enough to extract into a shared component library.

Architectural Boundaries

Boundary	Separates	Nature
<code>**controllers/ ↔ services/**</code>	HTTP concerns vs. app logic	Clear — controllers never call <code>src/core/</code> directly
<code>**services/ ↔ src/workflows/^ src/llm/**</code>	App orchestration vs. domain pipeline	Porous — <code>RAGRepository</code> is imported directly by <code>QAService</code> , bypassing workflows
<code>**src/workflows/ ↔ src/core/**</code>	Pipeline composition vs. atomic algorithms	Clean — workflows compose; cores compute
<code>**src/core/ ↔ everything else**</code>	Domain IR logic vs. infrastructure	Strongest boundary — core has no framework imports
<code>**models/ ↔ services/**</code>	ODM schema vs. business logic	Moderate — services import models for CRUD but models don't import services
<code>**Backend ↔ Frontend**</code>	Server vs. client	Strict network boundary via REST API + JSON

Notable Issues

- `QAService` is an import god-object — it wires together 8+ modules. A dependency-injection container or factory pattern would reduce coupling.

1. LLM layer leaks across boundaries — `rag_repository.py` (in `src/llm/`) directly imports `RetrievalWorkflow` (in `src/workflows/`), meaning the LLM layer reaches down into the retrieval layer. A cleaner design would have `RAGWorkflow` own the composition.
1. `async def` in services is misleading — `MessageService` and `ArticleService` use `async def` but call sync mongoengine operations, making the `async` keyword a no-op.
1. Frontend `api/client.js` is stateful — module-level token variable mutated by `setToken()` creates implicit global state. This works in practice but is a subtle coupling.

Application Startup

Entry Point

```
python backend/app.py
```

The true entry point is `main()` at `backend/app.py:31`. Python executes module-level code as it loads `app.py`, then calls `main()` via the `if __name__ == "__main__":` guard at line 48.

Step-by-Step Startup Trace

0. Module Import Phase (Python interpreter, during import of `app.py`)

Before `main()` runs, importing `app.py` resolves all top-level imports. This triggers a cascade:

File Loaded	Side Effect	Objects Created
<code>`app.py`</code>	Imports downstream modules	None
<code>`config/log_config.py`</code>	Imports logging + flask	<code>`ContextFilter`</code> class defined
<code>`config/db_connect.py`</code>	Imports mongoengine	<code>`Database`</code> class defined
<code>`services/qa_service.py`</code>	Imports <code>`engine_factory`</code> , <code>`reranker`</code> , <code>`retrieval_workflow`</code> , <code>`rag_repository`</code> , <code>`rag_workflow`</code> , <code>`rag_formatter`</code> , <code>`article_service`</code> , <code>`message_service`</code>	Module-level <code>`_workflow = None`</code> (line 20)
<code>`routes/api_routes.py`</code>	Imports controller functions	<code>**`api_bp = Blueprint("api", __name__)**`</code> — Flask Blueprint object created at line 16
<code>`routes/auth_routes.py`</code>	Imports controller functions	<code>**`auth_bp = Blueprint("auth", __name__, url_prefix="/api/v1/auth")**`</code> — line 5
<code>`controllers/api_controller.py`</code>	Imports <code>QAService</code> , <code>MessageService</code>	Controller function objects defined
<code>`controllers/auth_controller.py`</code>	Imports <code>UserService</code> , <code>User</code> , <code>RoleEnum</code>	Controller function objects defined
<code>`controllers/decorators.py`</code>	Imports <code>jwt</code> , <code>User</code>	<code>`token_required`</code> decorator function defined
<code>`models/*.py`</code>	Define mongoengine Document classes	<code>`User`</code> , <code>`Message`</code> , <code>`ReferencedArticle`</code> classes; connection NOT yet open
<code>`src/core/*.py`</code>	Define pure algorithm classes	Class objects only; no pipeline assembled yet

`src/constants.py`	Define data dicts	`CONTRACTIONS_MAP` (57 entries), `STOPWORDS` (~120 words)
--------------------	-------------------	--

Key point: Only class/function/Blueprint objects exist at this point. No database connection, no pipeline, no inference engine.

1. load_dotenv(".env", override=True) — app.py:32

File: backend/app.py, line 32
Config: backend/.env (loaded by python-dotenv)

- Reads backend/.env into os.environ with override=True (file values win over existing env vars)
- Makes these variables available:
- OLLAMA_HOST (default: http://127.0.0.1:11434)
- OLLAMA_API_KEY (optional)
- OLLAMA_MODEL (default: qwen3:8b)
- MONGO_URI (default: mongodb://localhost:27017)
- MONGO_DB_NAME (default: ECIRAS)
- JWT_SECRET (required for auth)

Why: Applications needs environment variables before any initialization that depends on them (MongoDB connection string, model names, etc.).

2. setup_logging() — app.py:33

File: backend/config/log_config.py, line 25

- Creates logs/ directory (via os.makedirs(log_dir, exist_ok=True))
- Creates a console handler at INFO level
- Creates a rotating file handler at DEBUG level, writing to logs/backend.log (10 MB per file, 5 backups)
- Both handlers use a custom ContextFilter that injects HTTP method + route into log records (empty SYSTEM / - during startup)
- Sets the root logger to DEBUG so all child loggers inherit

Why: Must be early so that all subsequent log statements (logger.info(...), logger.warning(...)) have a destination.

Objects created:

- logging.StreamHandler (console)
- logging.handlers.RotatingFileHandler (file)
- ContextFilter instances (2 — one per handler)

3. init_workflow() — app.py:34

File: backend/services/qa_service.py, line 23

This is the most complex step. It assembles the entire RAG pipeline and stores it in the module-global _workflow variable.

3a. EngineFactory.from_artifacts(...) — qa_service.py:25

```
File: backend/src/core/engine_factory.py, line 42
Data: data/output/flattened_nepal_constitution.json
      data/output/tf_index.json
      data/output/pos_index.json
      data/output/doc_stats.json
      data/synonyms.json
```

Step-by-step inside this call:

1. Load flattened documents — engine_factory.py:66-67

- Reads data/output/flattened_nepal_constitution.json (~700 documents)
- Converts each dict to a Document dataclass (src/core/document.py)

1. Load pre-computed indexes — engine_factory.py:69-81

- tf_index.json — term → doc_id → term frequency
- pos_index.json — term → doc_id → list of positions
- doc_stats.json — {doc_lengths: {...}, avgdl: float}
- These are the outputs of the offline ingestion pipeline (run separately via `python -m preprocessing_scripts.run_ingestion`)

1. Create two TextProcessors — engine_factory.py:86-87

Variable	Lemmatization	Stopwords	Purpose
`bm25_proc`	ON	REMOVED	BM25 scoring + candidate generation
`prox_proc`	OFF	KEPT	Proximity pair matching

Note: At this point TextProcessor objects are created but not yet initialized with spaCy (that happens lazily on first `process_text()` call via `_get_nlp()`).

1. Load synonym expander (optional) — engine_factory.py:89-91

- If `synonyms_path` is provided (it is — data/synonyms.json), creates QueryExpander
- Constructor loads and parses 44 synonym groups, builds a lookup dict

1. Create scorers — engine_factory.py:93-94

- BM25Scorer(tf_index, doc_lengths, avgdl) — stores indexes, $N = \text{len}(\text{doc_lengths})$
- ProximityScorer(pos_index) — stores positional index

1. Create SearchEngine — engine_factory.py:96-105

- Constructor pre-tokenizes all document titles via `bm25_processor.process_text(doc.title)` for fast title-boost lookups (line 76 of `search_engine.py`)
- This is the first real work that happens at startup: ~700 titles are lemmatized via spaCy

Objects created:

- list[Document] — ~700 dataclass instances
- dict tf_index, pos_index, doc_stats — loaded from JSON
- TextProcessor × 2
- QueryExpander
- BM25Scorer
- ProximityScorer
- SearchEngine

3b. Reranker(engine.bm25_scorer.tf_index) — qa_service.py:29

```
File: backend/src/core/reranker.py, line 13
```

- Stores the BM25 tf_index for later use in RRF fusion and MMR cosine similarity
- Creates an empty `_vector_cache` dict (populated lazily on first `rerank()` call)

Reason: The Reranker needs the tf_index to build sparse TF vectors for MMR diversity calculations.

3c. RetrievalWorkflow(engine, reranker) — qa_service.py:30

File: backend/src/workflows/retrieval_workflow.py, line 5

- Wires SearchEngine + Reranker into a single retrieve() method
- Sets default_recall_k = 50, default_top_k = 8

Reason: A workflow that composes the two pipeline stages (search → rerank). This is the unit of work used by both the RAG pipeline and any future retrieval-only path.

3d. RAGRepository(retrieval_workflow) — qa_service.py:31

File: backend/src/llm/rag_repository.py, line 40

Constructor does:

1. Stores the retrieval workflow — rag_repository.py:46
2. Reads model name — rag_repository.py:47
 - OLLAMA_MODEL env var or falls back to "qwen3:8b"
1. Creates Ollama client — rag_repository.py:49-52
 - Reads OLLAMA_HOST (default http://127.0.0.1:11434)
 - Reads OLLAMA_API_KEY (optional; sets Bearer header if present)
 - Creates ollama.Client(host, headers)
 - Note: This just creates the client object — no actual HTTP request yet
1. Initializes connectivity cache — rag_repository.py:54-56
 - _ollama_available = None (lazy — first request will check)
1. Builds article lookup — rag_repository.py:60
 - _build_article_lookup() iterates all ~700 documents and groups them by article_no
 - For each article number:
 - If an article-level Document exists (e.g., articles with lettered sub_clauses) → use its text directly
 - If only clause/sub-clause Documents exist → concatenate all clause texts with \n---\n
 - Also tracks clause structure per article for later context truncation

Objects created:

- ollama.Client
- dict _article_lookup — ~200 article entries (depends on corpus)
- dict _clause_structure — subset of articles that have clause-level docs

3e. RAGFormatter() — qa_service.py:32

File: backend/src/llm/rag_formatter.py, line 1

- Stateless formatter object with three pure methods: format_context(), build_system_prompt(), build_user_prompt()
- No initialization work needed

3f. RAGWorkflow(repository, formatter) — qa_service.py:32

File: backend/src/llm/rag_workflow.py, line 23

- Stores the RAGRepository and RAGFormatter
- Sets max_context_articles = 8
- Assigned to module-global _workflow at qa_service.py:20

```
_workflow: Optional[RAGWorkflow] = None    # line 20
...
_workflow = RAGWorkflow(repository, RAGFormatter()) # line 32
```

Why: Eager initialization means every HTTP request gets predictable response times — no slow first request, no locking,

no singleton pattern. The ~1s startup cost is paid once at boot.

4. `get_spacy_pipeline()` — `app.py:35`

File: `backend/src/core/text_processor.py`, line 8

- This function is called again here (it was already called during `init_workflow()` when the first `process_text()` ran on ~700 titles)
 - The second call hits the `if _spacy_nlp is None` guard and returns the existing cached pipeline
 - If this were the first call:
 1. Tries `spacy.load("en_core_web_sm")` — the full English pipeline
 2. Falls back to `spacy.blank("en")` if the model is not installed (tokenization works, lemmas are identity)
- Objects created: (only on first call)
- `spacy.Language` pipeline (either `en_core_web_sm` or `blank en`)

5. `create_app()` — `app.py:37`

File: `backend/app.py`, line 19

5a. `Flask(__name__)` — `app.py:20`

Creates the Flask WSGI application instance. No special configuration (no secret key, no template folder — it's an API-only app).

Objects created:

- Flask app instance

5b. `CORS(app, supports_credentials=True)` — `app.py:21`

Wraps the Flask app with CORS middleware. `supports_credentials=True` allows cookies (used as JWT fallback).

Note: No origin restrictions are configured — `CORS(app)` with defaults means all origins are allowed. This is technical debt (flagged in `docs/PROJECT_DOCUMENTATION.md`).

5c. `Database().connect(...)` — `app.py:22-25`

File: `backend/config/db_connect.py`, line 15

Config: `MONGO_DB_NAME` (default "ECIRAS"), `MONGO_URI` (default "mongodb://localhost:27017")

Singleton pattern:

```
class Database:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

- `Database()` returns the singleton instance
- `.connect()` calls `mongoengine.connect(host, db, alias="default", ...)`
- `maxPoolSize=10`
- `minPoolSize=2`
- `connectTimeoutMS=5000`
- `serverSelectionTimeoutMS=5000`
- This establishes the connection pool to MongoDB
- Fail-fast: If MongoDB is unreachable, an exception is raised and the server will not start

Objects created:

- Database singleton (once, on first instantiation)
- mongoengine connection pool to ECIRAS database

5d. `app.register_blueprint(api_bp)` — `app.py:26`

File: `backend/routes/api_routes.py`, line 16

Registers 8 routes under the Flask app:

Method	Path	Auth	Decorator chain
GET	<code>/api/v1</code>	No	<code>`home_route()`</code> → <code>`home()`</code>
GET	<code>/api/v1/health</code>	No	<code>`health_route()`</code> → <code>`health()`</code>
POST	<code>/api/v1/ask</code>	JWT	<code>`@token_required`</code> → <code>`ask_route()`</code> → <code>`ask()`</code>
POST	<code>/api/v1/ask-stream</code>	JWT	<code>`@token_required`</code> → <code>`ask_stream_route()`</code> → <code>`ask_stream()`</code>
GET	<code>/api/v1/messages</code>	JWT	<code>`@token_required`</code> → <code>`list_messages_route()`</code> → <code>`list_messages()`</code>
GET	<code>/api/v1/messages/<id></code>	JWT	<code>`@token_required`</code> → <code>`get_message_route()`</code> → <code>`get_message()`</code>
DELETE	<code>/api/v1/messages/<id></code>	JWT	<code>`@token_required`</code> → <code>`delete_message_route()`</code> → <code>`delete_message()`</code>
DELETE	<code>/api/v1/messages</code>	JWT	<code>`@token_required`</code> → <code>`delete_all_messages_route()`</code> → <code>`delete_all_messages()`</code>

5e. `app.register_blueprint(auth_bp)` — `app.py:27`

File: `backend/routes/auth_routes.py`, line 5

Registers 4 routes under `/api/v1/auth`:

Method	Path	Auth	Decorator chain
POST	<code>/api/v1/auth/register</code>	No	<code>`register_user()`</code> → <code>`register()`</code>
POST	<code>/api/v1/auth/login</code>	No	<code>`login_user()`</code> → <code>`login()`</code>
POST	<code>/api/v1/auth/logout</code>	JWT	<code>`@token_required`</code> → <code>`logout_user()`</code> → <code>`logout()`</code>
GET	<code>/api/v1/auth/me</code>	JWT	<code>`@token_required`</code> → <code>`get_logged_in_user()`</code> → <code>`get_current_user()`</code>

6. `app.run(debug=False, threaded=True)` — `app.py:41`

File: `backend/app.py`, line 41

- Starts Werkzeug development server on `http://127.0.0.1:5000`
- `debug=False` — no reloader, no debugger
- `threaded=True` — each request gets its own thread
- The server blocks here until `KeyboardInterrupt` or crash

Why: This is the line that begins serving HTTP traffic. All initialization must complete before this point.

Startup Ordering Summary

Step	File	Duration	What Happens
0	backend/app.py (import)	~10ms	All modules imported, classes loaded
1	backend/app.py:32 (load_dotenv)	~5ms	.env → os.environ
2	backend/app.py:33 (setup_logging)	~5ms	Console + file log handlers
3a	src/core/engine_factory.py:42	~500ms	4 JSON files loaded, SearchEngine assembled
3b	src/core/reranker.py:13	~1ms	Reranker created (no data loaded)
3c	src/workflows/retrieval_workflow.py:5	~1ms	Composes engine + reranker
3d	src/llm/rag_repository.py:40	~200ms	Ollama client created, article lookup built
3e	src/llm/rag_formatter.py:1	~1ms	Stateless formatter
3f	services/qa_service.py:32	~1ms	RAGWorkflow stored in _workflow global
4	src/core/text_processor.py:8	~300ms	spaCy pipeline initialized (cached)
5a	backend/app.py:20	~5ms	Flask app created
5b	backend/app.py:21	~1ms	CORS wrapped
5c	config/db_connect.py:15	~200ms	MongoDB connection established
5d	backend/app.py:26	~1ms	API blueprint registered (8 routes)
5e	backend/app.py:27	~1ms	Auth blueprint registered (4 routes)
6	backend/app.py:41	–	Server starts listening (:5000)
Total startup time		~1.2-1.5s	

Dependency Injection Summary

The backend uses manual constructor injection (no DI framework). The assembly chain:

```
EngineFactory.from_artifacts()
├─ produces SearchEngine
│   └─ injected into RetrievalWorkflow(engine, reranker)
│       └─ injected into RAGRepository(retrieval_workflow)
│           └─ injected into RAGWorkflow(repository, formatter)
│               └─ stored in module global _workflow in qa_service.py
```

Each layer receives its dependencies explicitly via `__init__` parameters. No service locator, no inversion-of-control container.

What is NOT Initialized at Startup

Component	When Initialized	Where
Ollama connectivity check	First <code>`/ask?use_llm=true`</code> request	<code>`rag_repository.py:291`</code> (<code>`_ensure_ollama_checked()`</code>) — cached per process lifetime
spaCy pipeline (if not yet)	First <code>`process_text()`</code> call	<code>`text_processor.py:36`</code> (<code>`_get_nlp()`</code>) — lazy via module-level <code>`_spacy_nlp`</code>
Reranker vector cache	First <code>`rerank()`</code> call	<code>`reranker.py:29`</code> (<code>`_get_tf_vector()`</code>) — populated lazily per doc
Ingestion pipeline	Manual CLI invocation	<code>`python -m preprocessing_scripts.run_ingestion`</code>

Frontend Startup

The frontend is a separate process (Vite dev server):

```
cd frontend && npm run dev
→ Vite dev server on http://localhost:5173
→ Hot Module Replacement enabled
→ Proxies API requests to backend (configured in vite.config.js)
```

No connection to the backend is made until the user submits a query.

Request Lifecycle — POST /api/v1/ask (with LLM)

Representative Request

```
POST /api/v1/ask
Authorization: Bearer <JWT>
Content-Type: application/json

{"query": "What are the fundamental rights?", "use_llm": true}
```

Lifecycle Map

Client

POST /api/v1/ask (JWT Bearer + JSON body)

Flask Router

```
1. @token_required (decorator)
   | Extract Bearer token
   | jwt.decode(token, JWT_SECRET)
   | User lookup → token_version check ← MongoDB
   | request.user = payload
```

2. api_routes.py → ask_route() → ask()

api_controller.py : ask()

_parse_ask_request() → validate JSON, query ≤ 500 chars

QAService.answer_query(query, use_llm=True)

qa_service.py : answer_query()

_get_workflow() → module-global _workflow (RAGWorkflow)

workflow.ask(query, use_llm=True)

rag_workflow.py : RAGWorkflow.ask()

1. _prepare_articles(query)

repo.retrieve(query, top_k=8)

↓
rag_repository.py : RAGRepository.retrieve()

└ retrieval_workflow.retrieve(query)

↓
retrieval_workflow.py : retrieve()

└ engine.search(query, top_k=50)

↓
search_engine.py : SearchEngine

└ .search()

└ TextProcessor.process (BM25)

└ spaCy lemmatization

└ synonym_expander.expand()

└ _generate_candidates()

└ BM25Scorer.score()

└ title_boost computation

└ ProximityScorer.score()

└ hybrid scoring + sort

└ Returns top 30 scored documents

└ reranker.rerank(results, top_k=8)

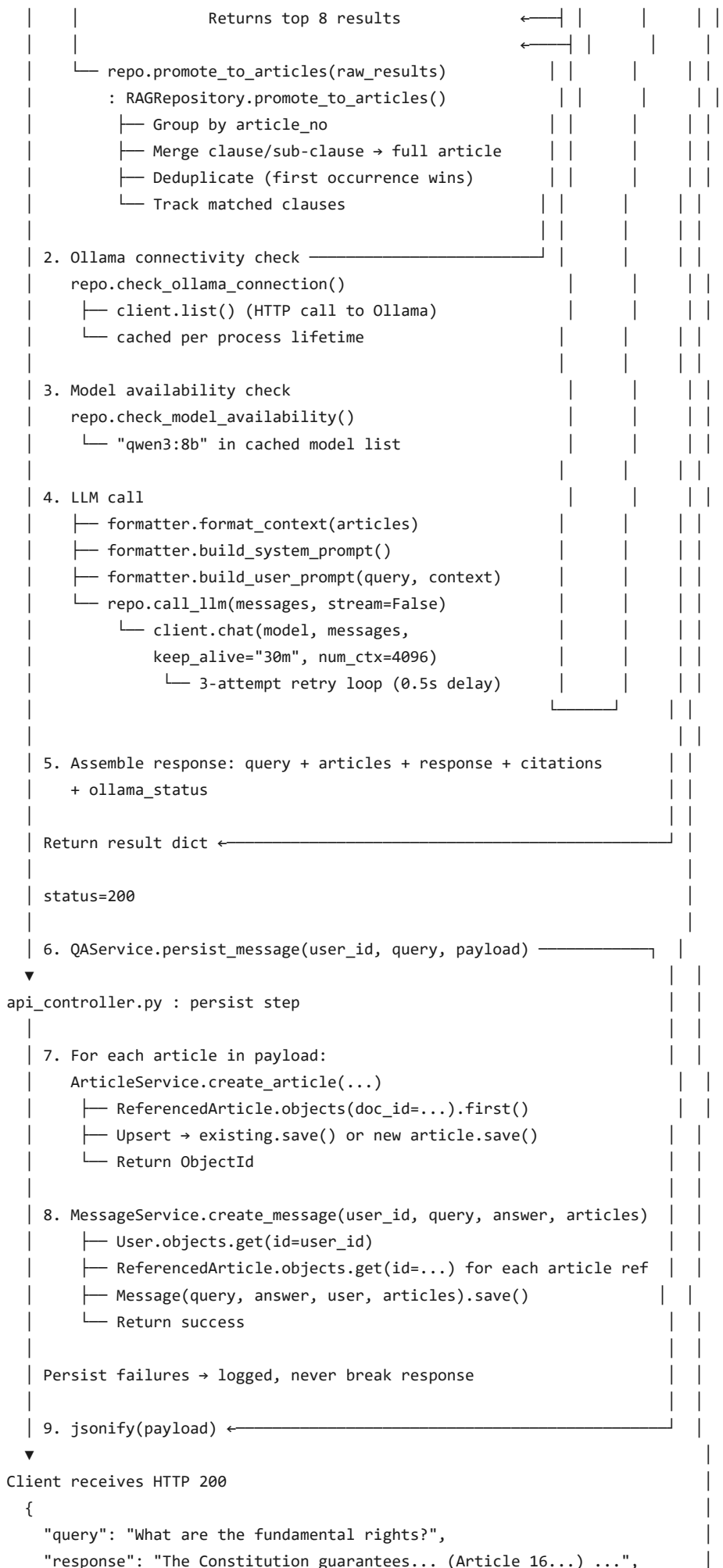
↓
reranker.py : Reranker.rerank()

└ _rrf_fuse() - RRF fusion

└ (k=60, BM25+prox+title ranks)

└ _mmr_diversify() - MMR (λ=0.5)

└ _apply_boost() - part/level



```
"articles": [... 8 article dicts ...],  
"citations": [... 8 citation objects ...],  
"ollama_status": { "connected": true, "model": "qwen3:8b",  
                  "model_available": true }  
}
```

Detailed Step-by-Step Trace

Phase 1 — Authentication (controllers/decorators.py:28-68)

File: backend/controllers/decorators.py Function: token_required(f) — decorator factory, returns decorated()

Step	File:Line	What Happens
1a	`decorators.py:31`	`request.headers.get('Authorization')` — reads `Authorization: Bearer <jwt>`
1b	`decorators.py:4-35`	Strips `Bearer ` prefix to extract the raw JWT string
1c	`decorators.py:6-37`	**Fallback:** if no header, reads `request.cookies.get('token')`
1d	`decorators.py:9-40`	If no token found anywhere → 401 `{ "error": "Token is missing!" }`
1e	`decorators.py:2-43`	Reads `JWT_SECRET` from environment → early 500 if unset
1f	`decorators.py:7`	`jwt.decode(token, JWT_SECRET, algorithms=['HS256'])` — verifies signature + expiry (12h)
1g	`decorators.py:9`	`_get_user(payload['user_id'])` — queries `User.objects(id=user_id).first()` via MongoDB
1h	`decorators.py:3-55`	**Token version check:** if `payload.token_version < user.token_version` → 401 (logout invalidates all prior JWTs)
1i	`decorators.py:7`	`request.user = payload` — attaches decoded JWT payload to Flask request context for downstream use
1j	`decorators.py:8-64`	Catches `ExpiredSignatureError` (→ 401), `InvalidTokenError` (→ 401), other exceptions (→ 500)

Why: Every /ask request requires authentication. The decorator validates the JWT and attaches the user identity to the request context.

MongoDB hit: One query to users collection for token_version check.

Phase 2 — Route Matching (routes/api_routes.py:28-31)

File: backend/routes/api_routes.py
Route: @api_bp.route("/api/v1/ask", methods=["POST"])
Handler: ask_route() → ask()

Flask matches the POST /api/v1/ask path against registered blueprints. The api_bp blueprint was registered at startup with no prefix, so /api/v1/ask matches line 28.

ask_route() simply calls ask() — a thin passthrough.

Phase 3 — Request Validation (controllers/api_controller.py:40-87)

File: backend/controllers/api_controller.py Function: ask() (line 62)

Step	File:Line	What Happens
3a	`api_controller.py:65`	`_parse_ask_request()` validates: JSON content-type (line 42), parseable JSON (line 45-47), `query` is a string (line 53), query $\leq$ 500 chars (line 56-57)
3b	`api_controller.py:70`	Reads `use_llm` from body, defaults to `False`
3c	`api_controller.py:73`	Calls `QAService.answer_query(query, use_llm=True)` — the main orchestration

Input: request object (Flask) with query="What are the fundamental rights?", use_llm=true Output: Tuple (payload dict, status_code int) Failure modes: Bad content-type → 400, missing query → 400, too long → 400

Phase 4 — Service Orchestration (services/qa_service.py:38-87)

File: backend/services/qa_service.py Function: QAService.answer_query() (line 84)

```
@staticmethod
def answer_query(query: str, use_llm: bool = False) -> tuple[dict, int]:
    workflow = QAService._get_workflow()
    return workflow.ask(query, use_llm=use_llm), 200
```

- Retrieves the module-global _workflow (RAGWorkflow instance assembled at startup)
- Delegates to RAGWorkflow.ask()
- Always returns status 200 (errors are encoded in the response body for graceful degradation)

Why: QAService is a thin passthrough. It was originally designed to hold the pipeline initialization and persistence. At request time it simply hands control to the workflow.

Phase 5 — RAG Workflow (src/llm/rag_workflow.py:53-120)

File: backend/src/llm/rag_workflow.py Function: RAGWorkflow.ask() (line 53)

5a. Article Retrieval + Promotion (rag_workflow.py:45-51)

_prepare_articles(query):

1. repo.retrieve(query, top_k=8) — delegates to retrieval pipeline (Phase 6 below)
2. repo.promote_to_articles(retrieved) — merges clause-level results into full articles (Phase 7 below)
3. Copies full_text ↔ text for downstream formatting

5b. LLM Decision Matrix (rag_workflow.py:70-119)

Condition	Code Path	Output
`use_llm=False`	Line 70-71	Returns `{query, articles}` immediately
Ollama unreachable	Lines 73-80	Returns `{query, articles, ollama_status: {connected: false}}`
Model missing	Lines 82-92	Returns `{query, articles, ollama_status: {connected: true, model_available: false}}`
LLM succeeds	Lines 94-109	Returns `{query, articles, response, citations, ollama_status}`
LLM fails after retries	Lines 110-113	Returns `{query, articles, response (error text), error}`

For the happy path with use_llm=true:

1. Check connectivity — repo.check_ollama_connection() (lazy, cached per process)
2. Check model availability — repo.check_model_availability() (cached model list)
3. Format prompt — rag_formatter.py:
 - format_context(promoted_articles) — builds a string with citation + title + text + score per article
 - build_system_prompt() — returns strict-grounding instructions
 - build_user_prompt(query, context) — wraps query + context in final prompt
1. Call LLM — repo.call_llm(messages, stream=False) with 3-attempt retry
2. Assemble response — extracts response.message.content, builds citations list
-

Phase 6 — Retrieval Pipeline (deep dive)

This is the most computationally intensive path.

6a. RAGRepository.retrieve(query, top_k=8) — rag_repository.py:213-220

Straight delegation: calls self.retrieval.retrieve(query, top_k).

6b. RetrievalWorkflow.retrieve(query, top_k=8) — retrieval_workflow.py:23-46

```
def retrieve(self, query, top_k=None, boost_rules=None):
    recall_k = self.default_recall_k          # 50
    final_k = top_k or self.default_top_k     # 8

    initial_results = self.engine.search(query, top_k=recall_k) # Step 6c
    if not initial_results:
        return []

    return self.reranker.rerank(initial_results, top_k=final_k, # Step 6d
```

Why two phases? Phase 1 (SearchEngine) casts a wide net for recall. Phase 2 (Reranker) refines for precision and diversity.

6c. SearchEngine.search(query, top_k=50) — search_engine.py:79-127

```
Input: "What are the fundamental rights?"
Output: 30 scored document dicts
```

S t e p	Function	What Happens
6 c. 1	`bm25_processor.process_text(query)`	Lemmatize + stopwords removal → `["fundamental", "right"]`
6 c. 2	`synonym_expander.expand(bm25_tokens, raw_query)`	Expands to include synonyms like `["fundamental", "right", "entitlement", "prerogative", ...]`
6 c. 3	`proximity_processor.process_text(query)`	No lemmatization, keeps stopwords → `["what", "are", "the", "fundamental", "rights"]`
6 c. 4	`ProximityScorer.generate_query_pairs(raw_tokens)`	5 tokens ≤ 5 → all-pairs: 10 pairs (what-are, what-the, what-fundamental, ...)

6 c. 5	<code>`_generate_candidates(bm25_tokens)`</code>	Union of <code>`tf_index[token].keys()`</code> for each expanded token → set of doc IDs
6 c. 6	Loop: <code>`_score_document(doc, ...)`</code> for each candidate	BM25 + title_match_count × 5.0 + 1.0 × proximity_score
6 c. 7	Sort by <code>`final_score`</code> DESC, take top 50	Returns 50 results → but <code>`search_engine.py:127`</code> takes <code>`scored[:top_k]`</code> where <code>`top_k=50`</code> (recall_k), so all candidates pass

Inside `_score_document` (`search_engine.py:142-179`):

```
score = BM25(bm25_tokens, doc_id)
        + len(set(bm25_tokens) & title_tokens[doc_id]) × 5.0
        + 1.0 × ProximityScorer.score(doc_id, query_pairs, max_window=30)
```

BM25 formula (`bm25_scorer.py:27-41`):

```
score = Σi IDF(ti) × tf(ti,D) × (k1+1) / (tf(ti,D) + k1 × (1-b+b×|D|/avgdl))
where k1=1.5, b=1.0
IDF(t) = ln((N - df(t) + 0.5) / (df(t) + 0.5) + 1)
```

Proximity formula (`proximity.py:67-103`):

```
score = 1/n × Σ 1/(d(p)+1)2
for each pair p within max_window=30 tokens
d(p) = minimum ordered distance between term1 and term2
```

6d. `Reranker.rerank(results, top_k=8)` — `reranker.py:164-180`

Input: 30 scored docs from SearchEngine
Output: 8 reranked docs

Stage	Function	What Happens
6d.1	<code>`_rrf_fuse()`</code>	Computes 3 separate rankings (BM25, proximity, title-match), fuses via <code>`RRF = Σ 1/(60 + rank)`</code>
6d.2	Sort by RRF score DESC	Results reordered
6d.3	<code>`_mmr_diversify()`</code>	<code>`λ=0.5`</code> : picks highest RRF first, then iteratively selects candidates <code>max `λ×score - (1-λ)×max_similarity`</code>
6d.4	<code>`_apply_boost()`</code>	Multiplies <code>`score × boost × part_rules[part_no] × level_rules[level]`</code>
6d.5	<code>`results[:top_k]`</code>	Takes top 8

Why RRF+MMR? RRF combines three independent signals without tuning weights. MMR ensures diversity — the top-8 results won't all be from Part 3 (Fundamental Rights) even if the query is about rights.

Phase 7 — Article Promotion (`rag_repository.py:175-208`)

File: `backend/src/llm/rag_repository.py` Function: `promote_to_articles()`

The reranker returns individual document-level results (articles, clauses, sub-clauses). This function merges them into full articles:

1. Scans results to build `matched_clauses_per_article` — a dict of `article_no` → set of matched clause numbers
2. Iterates results (already sorted by score descending):
 - For each unique `article_no`, looks up the pre-built article text from `_article_lookup`
 - Adds matched clause tracking
 - Skips duplicate `article_no` entries (first occurrence keeps its score)
1. Returns deduplicated, full-article results

Why: Users want to see full articles, not fragments. Clauses are merged so the user reads complete provisions.

Phase 8 — Persistence (`api_controller.py:76-77`)

File: `backend/controllers/api_controller.py` Function: inline in `ask()` (line 76-77)

```
if status_code == 200:
    user_id = request.user.get("user_id")
    QAService.persist_message(user_id, query, payload)
```

8a. `QAService.persist_message()` — `qa_service.py:43-81`

File: `backend/services/qa_service.py`

For each article in `payload["articles"]`:

1. `ArticleService.create_article(...)` — `article_service.py:10-73`
 - Queries `ReferencedArticle.objects(doc_id=...).first()`
 - If exists → updates text, scores, matched terms, saves
 - If new → creates `ReferencedArticle(...).save()`
 - Returns `{success, data: {id: ObjectId}}`
1. `MessageService.create_message(user_id, query, answer, article_ids)` — `message_service.py:16-72`
 - `User.objects.get(id=user_id)` — resolves the user reference
 - `ReferencedArticle.objects.get(id=...)` for each article id — resolves article references
 - `Message(query, answer, user, articles).save()` — creates the message document

Fire-and-forget: All persistence failures are caught and logged. They never affect the HTTP response.

Why: Message persistence must never block the user from getting their answer. If MongoDB has a transient failure, the user still gets their results; the conversation is simply not recorded.

Phase 9 — Response (`api_controller.py:84`)

```
return jsonify(payload), status_code
```

Response shape (success, with LLM):


```

{
  "query": "What are the fundamental rights?",
  "response": "The Constitution of Nepal guarantees several fu
ndamental rights... (Part 3, Article 16)...",
  "articles": [
    {
      "doc_id": "31",
      "part_no": 3,
      "article_no": 31,
      "title": "Right relating to education",
      "citation": "Part 3, Article 31",
      "content": "(1) Every citizen shall have the right to access to education...",
      "text": "Part 3 Article 31\nRight relating to education\nClause 1...",
      "level": "article",
      "score": 8.47,
      "bm25_score": 6.12,
      "proximity_score": 2.35,
      "title_match_count": 2,
      "matched_terms": ["right", "education"],
      "exact_matched_terms": ["rights"],
      "boost_multiplier": 0.98,
      "matched_clauses": ["1", "2", "3"]
    },
    ...
  ],
  "citations": [
    {"article": "Part 3, Article 31", "title": "Right relating to education", "doc_id": "31"},
    ...
  ],
  "ollama_status": {
    "connected": true,
    "model": "qwen3:8b",
    "model_available": true
  }
}

```

Content-Type: application/json Status: 200 OK (errors are encoded in body, never 500 for LLM failures)

Failure Mode Matrix

Failure Point	Behavior	HTTP Status	User Sees
Missing/invalid JWT	Decorator rejects	401	`{"error": "Token is missing!"}`
Bad JSON body	`_parse_ask_request()`	400	`{"error": "Invalid JSON payload."}`
Query > 500 chars	`_parse_ask_request()`	400	`{"error": "Query is too long..."}`
Search returns nothing	RAGWorkflow	200	`articles: []`
Ollama unreachable	`check_ollama_connection()`	200	`articles` + `ollama_status.connected=false`
Model missing	`check_model_availability()`	200	`articles` + `ollama_status.model_available=false`

LLM fails 3× retry	<code>`call_llm()` catch</code>	200	<code>`response` (error text) + `error` field</code>
Article persist fails	<code>`ArticleService` catch</code>	200	Answer intact; failure logged
Message persist fails	<code>`MessageService` catch</code>	200	Answer intact; failure logged
Unhandled exception	<code>`ask()`</code> outer <code>try/except</code>	500	<code>`{"error": "An error occurred..."}`</code>
MongoDB connection lost (mid-request)	<code>`token_required`</code> → <code>`_get_user`</code>	401	Token validation passes (skips version check), but downstream CRUD fails

Key Design Properties

- Graceful degradation at every layer — LLM, MongoDB, and search all have independent failure modes that degrade rather than crash
- No locking or contention — no shared mutable state during request processing; all pipeline objects are read-only after startup
- Synchronous orchestration — no `async/await`, no queue, no background jobs. Request-response is synchronous within the thread
- Stateless IR engine — SearchEngine and Reranker are pure computation with no side effects
- Persistence is best-effort — fire-and-forget after the response is computed

Domain Model

Business Context

We build a constitutional Q&A system for the Constitution of Nepal (2072 / 2015). Citizens, legal researchers, and students ask questions in plain English about constitutional provisions, and the system returns the most relevant articles with citations — optionally augmented by an LLM-generated answer grounded strictly in those articles.

This is an information retrieval system with a RAG layer, not a legal advice platform. The system retrieves what the constitution says; it does not interpret, argue, or apply law to specific cases.

Core Domain Entities

User

A person who registers, logs in, and asks questions.

User

- fullname (3-50 chars, required)
- email (unique, required)
- password_hash (bcrypt, 60 chars)
- role (user | admin)
- token_version (integer, incremented on logout)

Business rules:

- Email must be unique — no two users can share an email
- Password is never stored in plaintext; only the bcrypt hash is kept
- token_version is a monotonic counter: incrementing it invalidates all existing JWTs for that user
- Role defaults to user; admin routes are not yet exposed

Identity: identified by a MongoDB ObjectId; authenticated via JWT

Message

A single Q&A exchange between a user and the system.

Message

- query (string, required, ≤500 chars)
- answer (string, optional – empty if LLM was not used)
- user (reference → User, required)
- articles (list of references → ReferencedArticle)
- created_at
- updated_at

Business rules:

- Every message belongs to exactly one user (ownership)
- Deleting a user CASCADE-deletes their messages
- A message can reference zero or more articles (retrieval-only mode has articles but no answer)
- answer is empty when use_llm=false
- The query is the user's original question — never modified after creation

Identity: MongoDB ObjectId

ReferencedArticle

A constitutional provision that was retrieved and presented to the user as part of an answer. This is a snapshot — the

article's text and scores are frozen at query time.

```
ReferencedArticle
├─ title (e.g., "Right relating to education")
├─ citation (e.g., "Part 3, Article 31")
├─ doc_id (unique, from the flattened corpus)
├─ relevance_score (combined final score after reranking)
├─ bm25_score
├─ proximity_score
├─ title_match_count
├─ article_no
├─ clause_no
├─ subclause_id
├─ level (article | clause | subclause)
├─ part_no
├─ text (truncated context for LLM)
├─ full_text (complete provision text)
├─ content (cleaned body, without enriched-text headers)
├─ matched_terms (lemmatized query terms that matched)
├─ exact_matched_terms (raw query terms that matched)
├─ created_at
└─ updated_at
```

Business rules:

- doc_id is unique across the corpus — used for upsert deduplication
- relevance_score is the canonical ranking signal after all reranking stages
- Matched terms drive frontend highlighting in ArticleCard.jsx
- Articles can be shared across messages (many-to-many via reference list)
- Deleting an article NULLIFIES the reference (messages keep working, the article data remains in the referenced_articles collection)

Identity: MongoDB ObjectId

Document (IR Engine Internal)

The IR engine's representation of a single row in the flattened constitution corpus. This is an internal domain entity — it is never exposed to the API or frontend.

```
Document (dataclass)
├─ doc_id (unique string)
├─ part_no
├─ article_no
├─ title
├─ text (enriched with Part/Article/Clause headers)
├─ citation (human-readable, e.g. "Part 3, Article 16")
├─ level (part | article | clause | subclause)
├─ clause_no (optional)
├─ subclause_id (optional)
├─ is_primary
├─ parent_id
├─ raw_text (optional – original text without enrichment)
├─ citation_normalized (optional)
└─ boost (float, default 1.0 – per-document scoring multiplier)
```

Business rules:

- One Document per line in the flattened JSON corpus (~700 documents)
- boost is authored per document and influences the rule-based reranking stage

- level determines how the document is treated during article promotion (clause/sub-clause docs get merged into full article texts)
- doc_id ties back to the ReferencedArticle.doc_id for persistence

Supporting Domain Concepts

SearchEngine + Reranker (The Retrieval Pipeline)

Not entities — these are domain services that execute the core business logic.

- SearchEngine — accepts a query, returns scored candidate documents using BM25 + title boost + proximity scoring
- Reranker — refines candidate list through RRF fusion, MMR diversity, and rule-based boost
- RetrievalWorkflow — composes SearchEngine + Reranker into a single pipeline step

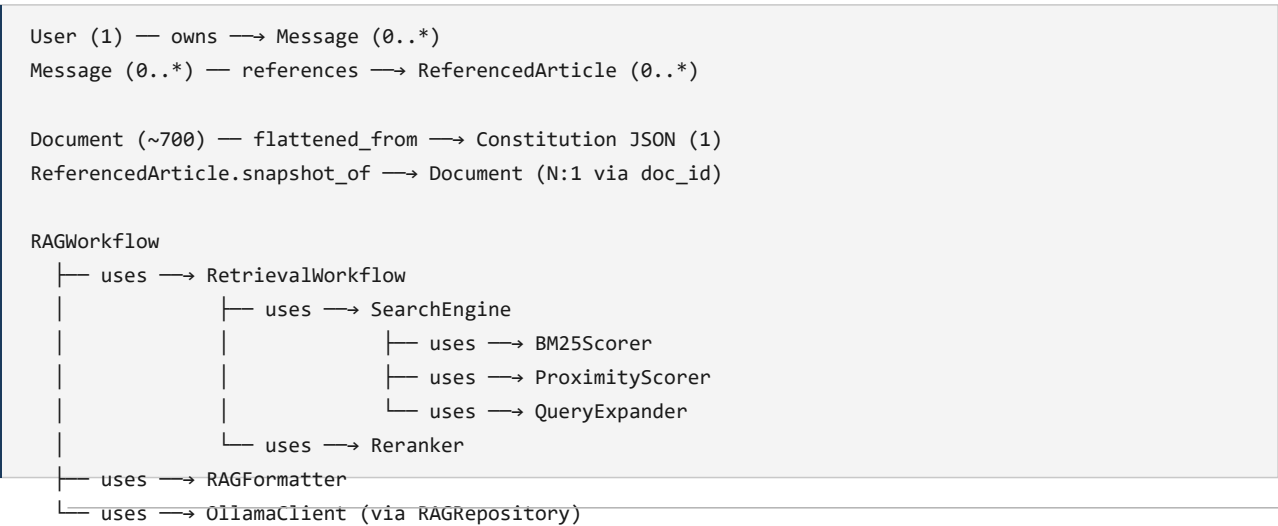
RAGWorkflow

The orchestrator that wires retrieval, article promotion, and LLM generation into the Q&A answer.

QueryExpander

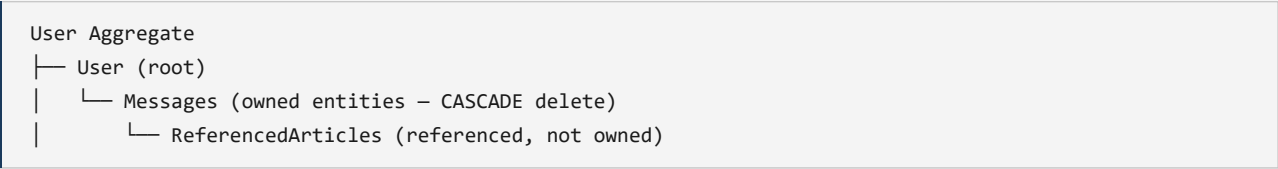
A domain service that expands query tokens with synonyms (44 groups) to improve recall across legal terminology variants (e.g., "right" ↔ "entitlement" ↔ "prerogative").

Relationships



Aggregates

Aggregate Root: User



- Deleting a User deletes all their Messages
- Messages survive if their ReferencedArticles are deleted (NULLIFY)

Aggregate Root: ReferencedArticle (standalone)

- No child entities
- Referenced by zero or more Messages
- Lifecycle independent of any User

Business Rules (Invariants)

Rule	Enforced At	Rationale
Query $\leq$ 500 characters	Controller (`_parse_ask_request`)	Prevents abuse and keeps latency predictable
JWT must match user's current `token_version`	Decorator (`token_required`)	Logout must invalidate all existing sessions immediately
LLM answer must cite article(s)	RAGFormatter prompts (not programmatically enforced)	Legal answers need traceability to source text
LLM must say "not found" if context lacks answer	System prompt instruction (not programmatically enforced)	Prevents hallucination of non-constitutional content
Article scores are frozen at query time	`ReferencedArticle.to_json()` captures scores	Historical accuracy — old messages reflect old ranking
Email uniqueness	MongoDB unique index on `User.email`	No duplicate accounts
Synonym expansion only for BM25, not proximity	`SearchEngine.search()` code logic	Proximity needs original ordering; synonyms would corrupt it
Two text processors (BM25 + proximity)	EngineFactory	BM25 needs lemmatized + stopword-free; proximity needs raw tokens with stopwords preserved
Short queries use all term pairs; long queries use adjacent only	`ProximityScorer.generate_query_pairs()`	$O(n^2)$ blowup protection for long queries
spaCy `en_core_web_sm` preferred, blank `en` fallback	`get_spacy_pipeline()`	Graceful degradation if model not installed

Terminology

Term	Definition
Article	A numbered section of the Constitution (e.g., Article 16 — "Right to live with dignity")
Part	A group of related articles (e.g., Part 3 — "Fundamental Rights")
Clause	A numbered subsection within an article (e.g., Article 16(1))
Sub-clause	A lettered sub-division within a clause (e.g., Article 16(1)(a))
Level	One of `part`, `article`, `clause`, `subclause` — determines how the document is scored and promoted
Flattened corpus	The nested constitution JSON transformed into a flat list of ~700 Documents, one per leaf node
Article promotion	The process of merging clause/sub-clause search results into full article texts for user display
Score	The combined relevance signal after BM25 + proximity + title boost + RRF + MMR + rule boost
Citation	Human-readable reference string (e.g., "Part 3, Article 31")

Matched terms	Lemmatized query tokens that had non-zero tf in the BM25 index for a given document
Enriched text	Document text with "Part X Article Y\nClause Z" headers prepended by the flattening script
Token version	A counter on each User document; incrementing it invalidates all existing JWTs for that user

What the Domain is NOT

- Not a legal reasoning system — no case law, no precedent, no statutory interpretation
- Not a document management system — the constitution corpus is static, not user-editable
- Not a citation graph — articles are standalone, not linked to each other
- Not a chatbot — each query is independent, no conversation state beyond history persistence
- Not an embedding/vector search system — all retrieval is algorithmic (BM25 + proximity + RRF/MMR); no dense embeddings

Important Classes & Modules

Ranked by architectural importance (not code volume).

1. RAGWorkflow — backend/src/llm/rag_workflow.py

Field	Value
Responsibility	Central Q&A orchestrator — retrieves articles, calls LLM, assembles response
Collaborators	<code>`RAGRepository`</code> , <code>`RAGFormatter`</code>
Why it exists	Single entry point that both <code>`answer_query()`</code> and <code>`answer_query_streaming()`</code> delegate to. Owns the decision matrix (LLM on/off, error recovery).
Design pattern	Facade — exposes <code>`ask()`</code> and <code>`ask_streaming()`</code> that hide retrieval, promotion, prompt building, and LLM retry
Complexity	6/10 — branching logic across LLM availability states
Used by	Every Q&A request — <code>`QAService.answer_query()`</code> and <code>`answer_query_streaming()`</code>

2. SearchEngine — backend/src/core/search_engine.py

Field	Value
Responsibility	Multi-signal retrieval: BM25 + title boost + proximity scoring → top-k candidates
Collaborators	<code>`BM25Scorer`</code> , <code>`ProximityScorer`</code> , <code>`TextProcessor`</code> (×2), <code>`QueryExpander`</code> , <code>`Document`</code>
Why it exists	This is the heart of the IR engine. Implements the entire Phase 1 retrieval pipeline. Pure domain logic — zero framework dependencies.
Design pattern	Strategy — scoring signals are injected as separate objects; Template Method — <code>`search()`</code> defines the pipeline skeleton
Complexity	7/10 — orchestrates 3 scoring signals, synonym expansion, candidate generation
Used by	<code>`RetrievalWorkflow.retrieve()`</code> on every query

3. RAGRepository — backend/src/llm/rag_repository.py

Field	Value
Responsibility	Bridges retrieval + LLM worlds. Owns Ollama client, connectivity checks, retry logic, article lookup, and context truncation.
Collaborators	<code>`RetrievalWorkflow`</code> , <code>`ollama.Client`</code>

Why it exists	Separates LLM concerns (client management, retry, connectivity caching) from both the retrieval engine and the workflow orchestrator.
Design pattern	**Repository** — abstracts the data source (LLM + retrieval) behind a uniform interface
Complexity	8/10 — manages Ollama client lifecycle, retry loop, API version variance in model list responses, article lookup construction
Used by	<code>`RAGWorkflow`</code> on every LLM-involved request

4. Reranker — backend/src/core/reranker.py

Field	Value
Responsibility	Three-stage reranking: RRF signal fusion → MMR diversity → rule-based boost
Collaborators	<code>`BM25Scorer.tf_index`</code> (via constructor — for cosine similarity)
Why it exists	Phase 2 of retrieval. Raw <code>SearchEngine</code> results are high-recall; Reranker adds precision and diversity without ML. Pure math — zero imports beyond <code>`math`</code> .
Design pattern	**Pipeline** — three composable stages (<code>`_rrf_fuse`</code> , <code>`_mmr_diversify`</code> , <code>`_apply_boost`</code>) with a single public <code>`rerank()`</code> entry point
Complexity	9/10 — MMR requires per-request $O(n^2)$ cosine similarity computations on sparse TF vectors; the vector cache adds tricky state management
Used by	<code>`RetrievalWorkflow.retrieve()`</code> on every query

5. QAService — backend/services/qa_service.py

Field	Value
Responsibility	Application service that owns pipeline initialization and persistence. Exposes <code>`answer_query()`</code> and <code>`persist_message()`</code> .
Collaborators	<code>`RAGWorkflow`</code> , <code>`ArticleService`</code> , <code>`MessageService`</code> , <code>`EngineFactory`</code> , <code>`Reranker`</code>
Why it exists	Boundary between Flask controllers and the domain layer. Also holds the module-global <code>`_workflow`</code> singleton, making initialization explicit.
Design pattern	**Facade** — hides the entire pipeline assembly behind static methods; **Singleton** (for <code>`_workflow`</code> global)
Complexity	4/10 — mostly delegation and wiring
Used by	<code>`api_controller`</code> on every Q&A request

6. EngineFactory — backend/src/core/engine_factory.py

Field	Value
-------	-------

Responsibility	Assembles a fully-wired `SearchEngine` from disk artifacts (4 JSON files + synonyms)
Collaborators	All `src.core` modules — `Document`, `TextProcessor`, `BM25Scorer`, `ProximityScorer`, `SearchEngine`, `QueryExpander`
Why it exists	Encapsulates the complex construction logic. Without it, callers would need to load 4 JSON files, create 2 TextProcessors, 2 scorers, and wire a SearchEngine manually.
Design pattern	**Factory** (static) — the purest form of Factory Method in the codebase
Complexity	3/10 — linear assembly with error handling for missing files
Used by	Called once at startup by `init_workflow()`, and by the standalone `rag_workflow.py` demo

7. BM25Scorer — backend/src/core/bm25_scorer.py

Field	Value
Responsibility	Compute BM25 scores and identify matched terms for a document
Collaborators	None (pure calculation)
Why it exists	The primary ranking algorithm. `k1=1.5`, `b=1.0` (full length normalization — unusual but intentional for legal documents of varying length).
Design pattern	**Strategy** — the BM25 algorithm is encapsulated behind `score()` / `idf()` / `matched_terms()`
Complexity	3/10 — standard BM25 formula with edge-case guards (df=0, doc_len=0, tf=0)
Used by	`SearchEngine.score_document()` for every candidate on every query

8. RetrievalWorkflow — backend/src/workflows/retrieval_workflow.py

Field	Value
Responsibility	Composes SearchEngine + Reranker into a single `retrieve(query)` → `list[dict]` step
Collaborators	`SearchEngine`, `Reranker`
Why it exists	Defines the canonical retrieval pipeline: recall_k=50 → search → rerank → top_k=8. This is the unit reused by both `RAGRepository` and any future standalone search path.
Design pattern	**Facade** — hides the two-phase pipeline behind `retrieve()`
Complecity	2/10 — thin composition

Used by	<code>`RAGRepository.retrieve()`</code> on every query
--------------------	--

9. TextProcessor — backend/src/core/text_processor.py

Field	Value
Responsibility	NLP pipeline: normalize → optionally lemmatize → optionally remove stopwords
Collaborators	<code>`CONTRACTIONS_MAP`</code> (constants), <code>`STOPWORDS`</code> (constants), spaCy pipeline
Why it exists	Two instances exist at runtime (BM25 processor and proximity processor) with different configurations. Centralizes text normalization so all tokenization decisions live in one file.
Design pattern	**Strategy** — two configured instances with different lemmatize/stopword settings; **Lazy Initialization** — spaCy pipeline loaded on first <code>`process_text()`</code> call
Complexity	4/10 — regex contraction expansion, spaCy fallback logic
Used by	<code>`SearchEngine`</code> (both processors), <code>`IndexBuilder`</code> , <code>`EngineFactory`</code>

10. ProximityScorer — backend/src/core/proximity.py

Field	Value
Responsibility	Score document relevance by how close query term pairs appear in the positional index
Collaborators	None (pure algorithm with injected positional index)
Why it exists	Adds term-proximity signal that BM25 cannot capture. "right to education" and "education right" get different proximity scores even though BM25 treats them identically.
Design pattern	**Strategy** — <code>`score()`</code> takes a list of pairs and returns a float; pair generation heuristic (all-pairs ≤5 tokens, adjacent only >5) is encapsulated in <code>`generate_query_pairs()`</code>
Complexity	5/10 — two-pointer minimum-distance algorithm, quadratic inverse score formula, max_window cutoff
Used by	<code>`SearchEngine.score_document()`</code> for every candidate on every query

11. RAGFormatter — backend/src/llm/rag_formatter.py

Field	Value
Responsibility	Build LLM prompts: context formatting, system instructions, user query wrapper
Collaborators	None (pure string builder)

Why it exists	Separates prompt engineering from orchestration logic. All LLM instruction changes happen here without touching retrieval or controller code.
Design pattern	Builder — constructs multi-part prompts via dedicated methods
Complexity	2/10 — string concatenation and template logic
Used by	<code>`RAGWorkflow.ask()`</code> and <code>`ask_streaming()`</code>

12. QueryExpander — backend/src/core/query_expander.py

Field	Value
Responsibility	Expand query tokens with 44 synonym groups for improved recall
Collaborators	None (loaded from <code>`data/synonyms.json`</code>)
Why it exists	Legal terminology has high variance: "arrest/detention/custody", "right/entitlement/prerogative". Synonym expansion bridges the gap between user vocabulary and constitutional text vocabulary.
Design pattern	Decorator — wraps token list and returns an expanded version
Complexity	4/10 — multi-word phrase detection, group-based lookup construction, deduplication
Used by	<code>`SearchEngine.search()`</code> for every query (if <code>synonyms_path</code> was provided at construction)

13. User (Model) — backend/models/user_model.py

Field	Value
Responsibility	MongoDB document model for user accounts. Handles password hashing and JSON serialization.
Collaborators	mongoengine, bcrypt
Why it exists	The authentication user. Every Q&A request requires a valid User with matching token_version.
Design pattern	**Active Record** (mongoengine Document) — combines data and persistence in one class
Complexity	2/10 — standard user model with password methods
Used by	<code>`UserService`</code> , <code>`auth_controller`</code> , <code>`decorators.py`</code> , <code>`MessageService`</code>

14. Message (Model) — backend/models/message_model.py

Field	Value
Responsibility	MongoDB document model for Q&A exchanges with article references
Collaborators	mongoengine, <code>`User`</code> (reference), <code>`ReferencedArticle`</code> (reference list)

Why it exists	Persists every Q&A exchange so users can browse history. Links users to articles.
Design pattern	**Active Record** — combines data and persistence
Complexity	2/10 — references + timestamps
Used by	`MessageService`

15. ReferencedArticle (Model) — backend/models/referenced_article_model.py

Field	Value
Responsibility	MongoDB document model for a constitutional provision snapshot at query time
Collaborators	mongoengine
Why it exists	Freezes the retrieval state so historical messages remain accurate even if the ranking algorithm changes. Also stores matched terms for frontend highlighting.
Design pattern	**Active Record** — combines data and persistence
Complexity	2/10 — 22 fields, body-cleaning regex
Used by	`ArticleService`, `MessageService`

16. IndexBuilder — backend/src/core/index_builder.py

Field	Value
Responsibility	Build TF index, positional index, and document statistics from a flat document list
Collaborators	`Document`, `TextProcessor`
Why it exists	Offline pipeline. Run once during ingestion to produce the three JSON indexes that `EngineFactory` needs at startup.
Design pattern	**Builder** — `build_all_indexes()` returns three indexes at once
Complexity	3/10 — multi-pass counting
Used by	`IngestionWorkflow` (offline), never at request time

17. Database — backend/config/db_connect.py

Field	Value
Responsibility	Singleton MongoDB connection manager

**Collaborators* *	mongoengine
Why it exists	Ensures exactly one connection pool is created per process. Fail-fast on startup if MongoDB is unreachable.
Design pattern	**Singleton** — `_instance` class variable with `__new__` override
Complexity	1/10 — connection wrapper
Used by	Called once at startup in `create_app()`

18. token_required (Decorator) — backend/controllers/decorators.py

Field	Value
Responsibility	JWT authentication gate — extracts token, validates signature, checks token_version
Collaborators	PyJWT, `User` (model)
Why it exists	Centralizes all auth logic so controllers don't duplicate it. Supports Bearer header + cookie fallback.
Design pattern	**Decorator** — wraps route handlers with pre-request validation
Complexity	4/10 — multi-source token extraction, token_version invalidation, DB-availability fallback
Used by	All protected routes in `api_routes.py` and `auth_routes.py`

19. Document (Dataclass) — backend/src/core/document.py

Field	Value
Responsibility	Typed data container for a single row in the flattened constitution corpus
Collaborators	None (plain dataclass)
Why it exists	Gives the IR engine a consistent typed representation of provisions. All 700+ corpus entries are `Document` instances.
Design pattern	**DTO / Data Transfer Object** — pure data with no behavior
Complexity	1/10 — 14 fields, no methods
Used by	`EngineFactory`, `SearchEngine`, `IndexBuilder`, `IngestionWorkflow` — every class that touches corpus data

20. RAGFormatter (Frontend: ui/ Primitives as a Group)

Field	Value
Responsibility	9 reusable UI primitives: Button, Input, Toggle, Card, Alert, Badge, Spinner, Pagination, Dialog

Collaborators	React, Tailwind CSS
Why it exists	Consistent, accessible presentation layer. Every page and feature component delegates visual rendering to these primitives.
Design pattern	**Composite** — pages compose feature components which compose primitives
Complexity	2/10 each — pure presentational components with variant/size props
Used by	Every page in the frontend

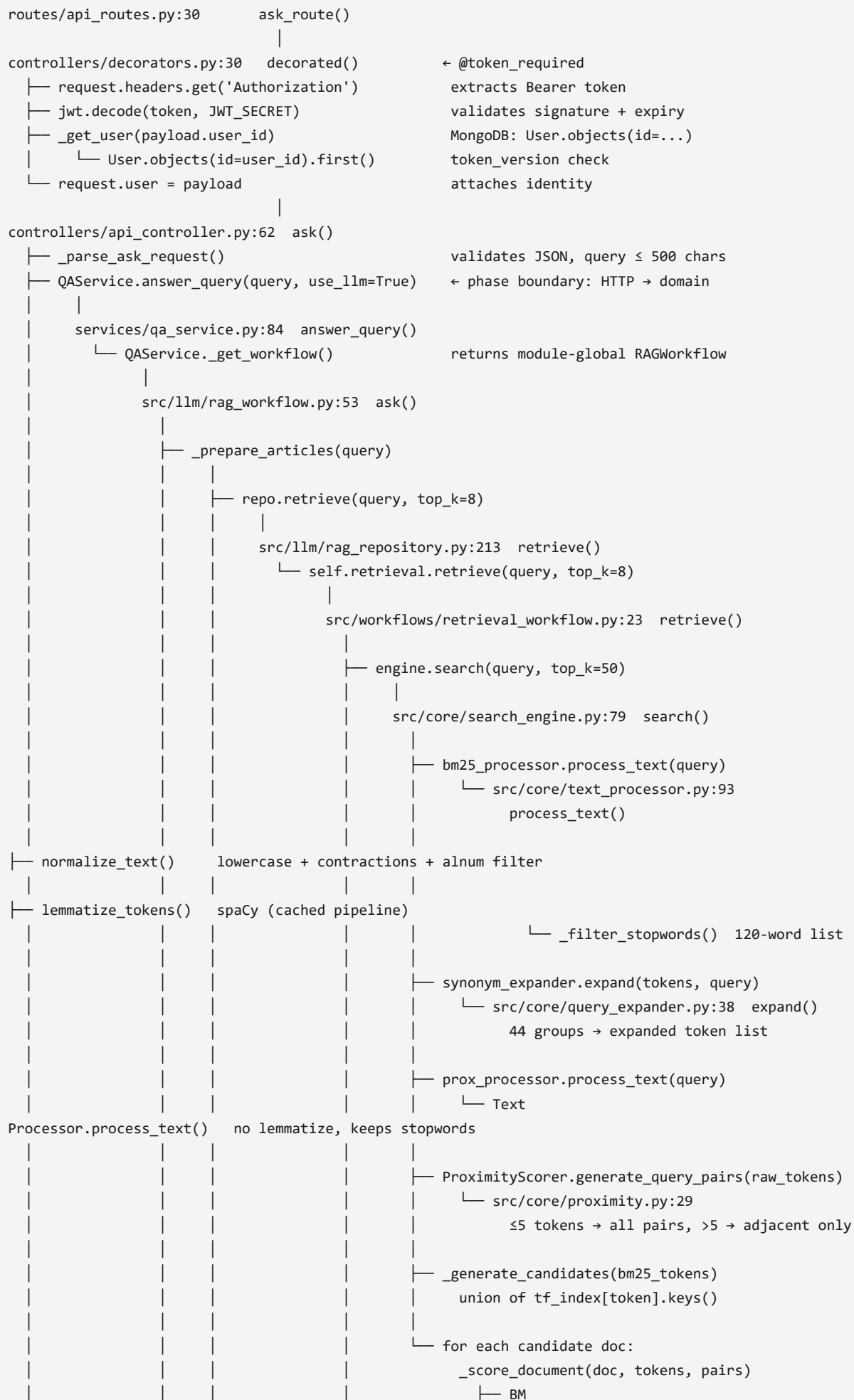
Honorable Mentions

Module	Rank	Why It Didn't Make Top 20
`UserService`	21	Thin CRUD wrapper over User model
`MessageService`	22	Same — CRUD with pagination
`ArticleService`	23	Same — upsert by doc_id
`IngestionWorkflow`	24	Offline-only, never runs at request time
`AuthProvider` (frontend)	25	Important but simpler than backend classes
`useAskStream` (hook)	26	SSE streaming wrapper — clever but small
`FlattenConstitution` (script)	27	Offline preprocessing, not runtime

Call Graph — Most Important Execution Paths

Path 1: Synchronous Q&A — POST /api/v1/ask

The primary path, exercised on every search or answer request.



```

25Scorer.score(tokens, doc_id) ← src/core/bm25_scorer.py:27
    |
    | └─ title_match_count × 5.0
    | └─ Pr
proximityScorer.score(doc_id, pairs) ← src/core/proximity.py:67
    |
    | └─ final = bm25 + title_bonus + 1.0 × prox
    |
    | └─ sort by final DESC → return top 50
    |
    | └─ reranker.rerank(initial_results, top_k=8)
    |
    |   src/core/reranker.py:164 rerank()
    |   |
    |   | └─ _rrf_fuse(results)
    |   |   rank by BM25 + rank by proximity + rank by title
    |   |   rrf =  $\sum 1/(60 + \text{rank})$  for each signal
    |   |
    |   | └─ _mmr_diversify(results)
    |   |    $\lambda=0.5$ , cosine similarity on sparse TF vectors
    |   |   iteratively selects max-mmr candidate
    |   |   └─ _cosine_similarity(vec_a, vec_b)
    |   |   └─ _g
et_tf_vector(doc_id) ← lazy-built cache
    |
    |   └─ _apply_boost(results)
    |       score ×
boost × part_rules[part] × level_rules[level]
    |
    |   └─ repo.promote_to_articles(raw_results)
    |       |
    |       | src/llm/rag_repository.py:175 promote_to_articles()
    |       |   └─ group by article_no
    |       |   └─ merge clause docs → full article text
    |       |   └─ deduplicate (first occurrence wins)
    |       |   └─ track matched clauses per article
    |
    |   └─ repo.check_ollama_connection() ← cached, may be no-op
    |       └─ self.client.list() HTTP GET to Ollama
    |
    |   └─ repo.check_model_availability()
    |       └─ "qwen3:8b" in self._available_models
    |
    |   └─ formatter.format_context(articles) ← src/llm/rag_formatter.py:5
    |   └─ formatter.build_system_prompt() ← src/llm/rag_formatter.py:19
    |   └─ formatter.build_user_prompt(query, ctx) ← src/llm/rag_formatter.py:44
    |
    |   └─ repo.call_llm(messages, stream=False)
    |       |
    |       | src/llm/rag_repository.py:320 call_llm()
    |       |   └─ for attempt in 1..3: ← retry loop
    |       |       client.chat(model, messages,
    |       |           keep_alive="30m",
    |       |           options={"num_ctx": 4096})
    |       |           ← HTTP POST to Ollama /api/chat
QAService.persist_message(user_id, query, payload)
    |
    |   services/qa_service.py:43 persist_message()
    |
    |   └─ for each article in payload:

```

```

|
| ArticleService.create_article(...)
|
| services/article_service.py:11 create_article()
|   └─ ReferencedArticle.objects(doc_id=...).first()
|   └─ existing.save() OR ReferencedArticle(...).save()
|   └─ return {success, data}
|
└─ MessageService.create_message(user_id, query, answer, article_ids)
    |
    services/message_service.py:16 create_message()
    └─ User.objects.get(id=user_id)
    └─ ReferencedArticle.objects.get(id=...) for each ref
    └─ Message(query, answer, user, articles).save()
└─ jsonify(payload), 200

```

← phase boundary: domain → HTTP

Path 2: Streaming Q&A — POST /api/v1/ask-stream

Same retrieval path, different response delivery.

```
controllers/api_controller.py:116 ask_stream()
├─ _parse_ask_request()           same validation
├─ QAService.answer_query_streaming(query, use_llm=True)
│   └─ services/qa_service.py:90 answer_query_streaming()
│       └─ workflow.ask_streaming(query, use_llm=True)
│           └─ src/llm/rag_workflow.py:122 ask_streaming() ← GENERATOR
│               └─ _prepare_articles(query)   same retrieval as sync path
│                   └─ repo.retrieve → ... → search → rerank → promote
│               └─ yield {"type": "articles", articles: [...]}
│                   └─ ← SSE event sent to client immediately
│               └─ check_ollama_connection()
│               └─ check_model_availability()
│               └─ format_context / build_system_prompt / build_user_prompt
│               └─ repo.call_llm(messages, stream=True)
│                   └─ rag_repository.py:320 call_llm()
│                       └─ client.chat(model, messages, stream=True)
│                           └─ ← Ollama returns generator
│                               for part in response: ← token loop
│                                   yield {"type": "token", "content": ...}
│                                       └─ ← SSE event per token
└─ Response(stream_with_context(_stream_events(...)),
    mime_type="text/event-stream")
    └─ controllers/api_controller.py:90 _stream_events() ← consumer of upstream generator
        └─ for event in events:
            └─ if type == "articles": store articles_data
            └─ if type == "token":   append to full_answer
            └─ if type == "done":
                └─ QAService.persist_message(...) persistence after stream ends
                └─ yield f"data: {json.dumps(event)}\n\n" ← SSE-serialized event
        └─ Response closes when generator is exhausted
```

Path 3: Login — POST /api/v1/auth/login

```

routes/auth_routes.py:14      login_user()
|
|
controllers/auth_controller.py:42 login()
|
|— request.get_json(), validate email + password
|
|— UserService.authenticate(email, password)
|   |
|   |— services/user_service.py:20 authenticate()
|   |   |— User.objects.get(email=email)           MongoDB query
|   |   |— user.check_password(password)          bcrypt.checkpw
|   |   |— return user
|   |
|— jwt.encode(payload={user_id, token_version, exp}, JWT_SECRET)
|
|— jsonify({token, user})
  + set_cookie('token', ..., httpOnly=True, SameSite='Strict')

```

Path 4: Registration — POST /api/v1/auth/register

```

controllers/auth_controller.py:8 register()
|
|— validate fullname (3-50), email (regex), password (≥6), role
|
|— UserService.create_user(fullname, email, password, role)
|   |
|   |— services/user_service.py:8 create_user()
|   |   |— User(fullname, email, role)
|   |   |— user.set_password(password)           bcrypt.gensalt + hashpw
|   |   |— user.save()                          MongoDB insert
|   |   |— return user.to_json()
|   |
|— jsonify({success, data: user})

```

Path 5: Chat History — GET /api/v1/messages

```

controllers/api_controller.py:143 list_messages()
|
|— request.args.get("limit", 20, type=int)
|— request.args.get("skip", 0, type=int)
|
|— MessageService.get_user_messages(user_id, limit, skip)
|   |
|   |— services/message_service.py:75 get_user_messages()
|   |   |— User.objects.get(id=user_id)
|   |   |— Message.objects(user=user)
|   |       .order_by('-created_at')
|   |       .skip(skip)
|   |       .limit(limit)
|   |   |— return {success, data: [msg.to_json()], pagination: {total, limit, skip, has_more}}

```

Path 6: Ingestion (Offline) — python -m preprocessing_scripts.run_ingestion

```

preprocessing_scripts/run_ingestion.py
├── flatten_constitution.py:flatten_constitution()
│   ├── reads data/nepal_constitution_new.json
│   │   ├── yields ~700 Document objects
│   │   └── writes data/output/flattened_nepal_constitution.json
└── IngestionWorkflow.build_indexes(flat_path, output_dir)
    ├── src/workflows/ingestion_workflow.py
    │   ├── load flat JSON → list[Document]
    │   ├── IndexBuilder(bm25_processor, prox_processor)
    │   │   ├── src/core/index_builder.py
    │   │   │   ├── build_tf_index(documents)
    │   │   │   │   ├── per doc: BM25 process → count term frequencies
    │   │   │   ├── build_positional_index(documents)
    │   │   │   │   ├── per doc: prox process → record positions
    │   │   │   └── compute_doc_stats(documents)
    │   │   │       ├── doc lengths + avgdl
    │   └── save_json(tf_index, "data/output/tf_index.json")
    │       save_json(pos_index, "data/output/pos_index.json")
    │       save_json(doc_stats, "data/output/doc_stats.json")

```

Cross-Cutting Concerns

Async Boundaries

Boundary	Type	Details
<code>`answer_query_streaming()`</code>	**Generator** (not async)	Uses Python generators with <code>`yield`</code> , not <code>`async def`</code> . The streaming pipeline is synchronous — each token blocks the thread.
<code>`_stream_events()`</code>	**Generator consumer**	Reads from an upstream generator, wraps events in SSE format. No true async I/O.
<code>`ArticleService.create_article()`</code> <code>async def`</code>	**Misleading async**	Declared <code>`async def`</code> but calls sync mongoengine internally. The <code>`async`</code> keyword is a no-op — there is no <code>`await`</code> anywhere.

Retry Loops

Location	Attempts	Delay	Trigger
<code>`RAGRepository.call_llm()`</code>	3	0.5s	Ollama HTTP failure or timeout
<code>`_perform_ollama_check()`</code>	1	—	Checked on first LLM request (cached afterward)

Callbacks

None. The codebase uses no callback-based patterns. All orchestration is synchronous and sequential.

Background Jobs

None. Every operation runs inline in the request thread. The only approximation is the fire-and-forget persistence in `QAService.persist_message()`` — MongoDB failures are logged but never break the HTTP response.

Recursion

None. All loops are iterative (for, while) or generator-based (yield from).

Call Graph Summary Statistics

Metric	Value
Deepest call chain (sync <code>`/ask`</code>)	~25 nested calls
Distinct functions invoked per <code>`/ask`</code>	~40
Network calls per <code>`/ask`</code> (LLM)	1–3 (Ollama /api/chat with retry)
Network calls per <code>`/ask`</code> (no LLM)	0
MongoDB queries per <code>`/ask`</code>	3 (token_version user lookup + N article upserts + 1 message create)
Generators	2 (<code>`ask_streaming`</code> , <code>`_stream_events`</code>)
Retry loops	1 (<code>`call_llm`</code>)
Recursion depth	0
Async/await usage	0 (all sync)
Background threads	0

Data Flow — The User Query

Chosen Data: A Single User Query

We trace "What are the fundamental rights?" from the moment the user types it until it reaches storage as a persisted message.

Flow Map



Step-by-Step Data Transformations

Step 1 — User Input (Frontend)

File: frontend/src/components/mainsearchbar.jsx Object: React state variable Transformation: Raw keystrokes → controlled input value


```
const [query, setQuery] = useState("");
// User types "What are the fundamental rights?"
// → query = "What are the fundamental rights?"
```

Purpose: Capture user input before submission.

Step 2 — HTTP Serialization (Frontend)

File: frontend/src/api/client.js Object: apiClient() → fetch options Transformation: React state → JSON HTTP body

```
// mainsearchbar.jsx calls:
apiClient("/api/v1/ask", {
  method: "POST",
  body: JSON.stringify({ query: "What are the fundamental rights?", use_llm: true }),
})
// apiClient.js adds Authorization: Bearer <jwt> header, 100s timeout
```

Purpose: Serialize the query into an HTTP POST request with JWT auth.

Step 3 — HTTP Deserialization (Backend)

File: backend/controllers/api_controller.py:40-59 Function: _parse_ask_request() Transformation: Flask request object → Python string

Input	`request` (Flask) — JSON body bytes
Validation	`request.is_json`, `request.get_json(silent=True)`, `data.get("query")`, type check, ≤500 chars
Output	`data = {"query": "What are the fundamental rights?", "use_llm": true}`
Failures	400 with `{ "error": "..." }` for missing content-type, unparseable JSON, missing query, wrong type, or too long

Purpose: Extract and validate the query from the HTTP request.

Step 4 — BM25 Tokenization

File: backend/src/core/text_processor.py:93-110 Function: TextProcessor.process_text(text) — called via SearchEngine.search() Object: TextProcessor instance configured with use_lemmatization=True, remove_stopwords=True

Input	`"What are the fundamental rights?"`
`normalize_text()`	→ `"what are the fundamental rights"` (lowercase, contractions expanded, non-alphanumeric stripped)
`lemmatize_tokens()` via spaCy	→ `["what", "be", "the", "fundamental", "right"]`
`_filter_stopwords()`	→ `["fundamental", "right"]` (what/be/the removed)
Output	`["fundamental", "right"]`

Purpose: Produce clean, lemmatized tokens for BM25 scoring. Stopwords are removed because they add noise to IDF.

Step 5 — Synonym Expansion

File: backend/src/core/query_expander.py:38-65 Function: QueryExpander.expand(tokens, raw_query)

Input	`["fundamental", "right"]`
-------	----------------------------

Lookup	`"fundamental"` → no synonym group; `"right"` → group with `entitlement`, `prerogative`, etc.
Expansion	`["fundamental", "right", "entitlement", "prerogative"]`
Multi-word guard	Multi-word phrases only included if present in raw query string
Output	`["fundamental", "right", "entitlement", "prerogative"]`

Purpose: Improve recall by matching documents that use synonym variants (e.g., "prerogative" instead of "right").

Step 6 — Proximity Tokenization

File: backend/src/core/text_processor.py:93-110 Function: TextProcessor.process_text(text) — TextProcessor with use_lemmatization=False, remove_stopwords=False

Input	`"What are the fundamental rights?"`
`normalize_text()`	→ `"what are the fundamental rights"`
No lemmatization	tokens pass through unchanged
No stopword removal	all tokens kept
Output	`["what", "are", "the", "fundamental", "rights"]`

Purpose: Preserve original word order and stopwords for proximity scoring. "right to education" ≠ "education right" — ordering matters.

Step 7 — Proximity Pair Generation

File: backend/src/core/proximity.py:29-33 Function: ProximityScorer.generate_query_pairs(tokens)

Input	`["what", "are", "the", "fundamental", "rights"]`
Length check	5 tokens → ≤5, so use **all-pairs**
Pairs generated	10 pairs: (what,are), (what,the), (what,fundamental), (what,rights), (are,the), (are,fundamental), (are,rights), (the,fundamental), (the,rights), (fundamental,rights)
Output	`[("what","are"), ("what","the"), ..., ("fundamental","rights")]`

Purpose: For short queries (≤5 tokens), all cross-term pairs are meaningful. For longer queries, only adjacent pairs are used to avoid $O(n^2)$ blowup.

Step 8 — Candidate Generation

File: backend/src/core/search_engine.py:134-140 Function: SearchEngine.generate_candidates(bm25_tokens)

Input	`["fundamental", "right", "entitlement", "prerogative"]`
Operation	For each token, look up `tf_index[token].keys()` → union all doc IDs
Output	`set` of ~200-400 document IDs (from ~700 total) containing at least one expanded token

Purpose: Narrow the corpus from 700 documents to a candidate set of documents that contain any query-related term.

Step 9 — BM25 Scoring

File: backend/src/core/bm25_scorer.py:27-41 Function: BM25Scorer.score(query_tokens, doc_id)

Applied once per candidate document.

Input	<code>`["fundamental", "right", "entitlement", "prerogative"]`, doc_id="31"</code>
Per term	<code>`tf = tf_index[term][doc_id]`; if tf=0 skip; `idf = ln((N-df+0.5)/(df+0.5)+1)`</code>
Per doc	<code>`doc_len = doc_lengths[doc_id]`, `avgdl` from stats</code>
Formula	<code>`score = Σ idf(t) × tf(t) × (1.5+1) / (tf(t) + 1.5 × (1-1.0+1.0 × doc_len/avgdl))`</code>
Output	<code>`float` — e.g., `6.12`</code>

Purpose: Compute term-frequency-based relevance using the BM25 algorithm. $k_1=1.5$ controls TF saturation, $b=1.0$ applies full length normalization.

Step 10 — Title Boost

File: backend/src/core/search_engine.py:168 Inline in `_score_document()`

Input	<code>`["fundamental", "right", "entitlement", "prerogative"]` ∩ `title_tokens[doc_id]`</code>
Operation	Count intersection size; multiply by 5.0
Example	For Article 31 "Right relating to education": title tokens include <code>`"right"`</code> → <code>`title_match_count=1`</code> → <code>`boosted_bm25 = 6.12 + 1 × 5.0 = 11.12`</code>
Output	<code>`float` — BM25 + title bonus</code>

Purpose: Documents whose titles contain query terms get a significant boost. "Right relating to education" ranks higher for "rights" queries because the title directly matches.

Step 11 — Proximity Scoring

File: backend/src/core/proximity.py:67-103 Function: `ProximityScorer.score(doc_id, pairs, max_window=30, ordered=True)`

Input	<code>doc_id="31", 10 query pairs</code>
Per pair	Look up positions of both terms in <code>`pos_index`</code> ; compute minimum ordered distance via two-pointer sweep
If distance ≤ 30	<code>`score += 1/(distance+1)^2`</code>
Average	<code>`total / valid_pairs`</code> (if no valid pairs, returns 0.0)
Output	<code>`float` — e.g., `2.35`</code>

Purpose: Reward documents where query terms appear close together. "fundamental rights" appearing in sequence scores higher than "fundamental" on page 1 and "rights" on page 5.

Step 12 — Hybrid Score Fusion

File: backend/src/core/search_engine.py:115-127 Inline in `search()` loop

Input	BM25 score + title bonus + proximity score
Formula	<code>`final = boosted_bm25 + 1.0 × proximity_score`</code>
Example	<code>`11.12 + 2.35 = 13.47`</code>

Sort	All scored documents sorted DESC by `final`
Output	Top 50 scored results as `list[dict]` with full metadata

Purpose: Combine three independent relevance signals into a single sortable score.

Step 13 — RRF Fusion

File: backend/src/core/reranker.py:54-84 Function: Reranker._rrf_fuse(results)

Input	50 scored documents from SearchEngine
Rank 1	Sort by `bm25_score` DESC → each doc gets BM25 rank
Rank 2	Sort by `proximity_score` DESC → each doc gets proximity rank
Rank 3	Sort by `title_match_count` DESC → each doc gets title rank
Fusion	$\text{rrf_score} = 1/(60+\text{bm25_rank}) + 1/(60+\text{prox_rank}) + 1/(60+\text{title_rank})$
Output	Same 50 documents, now with `rrf_score` field, sorted by RRF DESC

Purpose: Fuse three independent rankings without tuning weights. RRF automatically balances signals — a document that ranks moderately on all three signals may outrank one that ranks very high on only one.

Step 14 — MMR Diversity

File: backend/src/core/reranker.py:89-126 Function: Reranker._mmr_diversify(results)

Input	50 documents sorted by RRF
Seed	Pick highest-RRF document first
Loop	For each remaining candidate: $\text{mmr} = 0.5 \times \text{rrf_score} - 0.5 \times \text{max_similarity}(\text{to already selected})$
Similarity	Cosine similarity on sparse BM25 TF vectors
Pick	Move highest-MMR candidate to selected list
Output	50 documents reordered for diversity

Purpose: Prevent all top-8 results from being about the same topic. If the query is "fundamental rights", MMR ensures you also see results from other parts of the constitution, not just Part 3.

Step 15 — Rule-Based Boost

File: backend/src/core/reranker.py:132-159 Function: Reranker._apply_boost(results)

Input	50 diversified documents
Per doc	$\text{final_score} = \text{score} \times \text{doc.boost} \times \text{part_rules}[\text{part_no}] \times \text{level_rules}[\text{level}]$
Default multipliers	`part: 1.0, article: 0.98, clause: 0.95, subclause: 0.90`
Output	Top 8 documents taken after boost

Purpose: Authorial and hierarchical boosts. Part-level rules can promote specific parts of the constitution. Level rules ensure full articles rank above clauses/sub-clauses by default.

Step 16 — Article Promotion

File: backend/src/llm/rag_repository.py:175-208 Function: RAGRepository.promote_to_articles(results)

Input	8 document-level results (possibly clauses/sub-clauses)
Group	Aggregate by `article_no`
Merge	For articles stored as clauses → concatenate with `n---n`; for articles with direct text → use as-is
Dedup	First occurrence of each `article_no` keeps its score
Track	`matched_clauses` list per article for context truncation
Output	8 promoted article dicts with `content`, `citation`, `matched_clauses`

Purpose: Users should see full articles, not fragments. A search matching "Clause 2 of Article 31" should return the entire Article 31.

Step 17 — LLM Context Formatting

File: backend/src/llm/rag_formatter.py:5-16 Function: RAGFormatter.format_context(articles)

Input	8 article dicts with `citation`, `title`, `text`, `score`
Per article	`"{citation}: {title}\n{text}\n(Relevance Score: {score:.2f})\n"`
Output	Single string with all 8 articles concatenated, separated by blank lines

Purpose: Convert structured article data into a flat text block the LLM can consume in its context window.

Step 18 — LLM Prompt Assembly

File: backend/src/llm/rag_formatter.py:19-55 Functions: build_system_prompt(), build_user_prompt(query, context)

Input	`query`, formatted context string
System prompt	Grounding instructions: cite articles, answer from context only, adapt style by question type, decline if not found
User prompt	`"Context:\n{context}\n\nQuestion:\n{query}\n\nTask:\nFind the exact answer..."`
Messages	`[{"role": "system", "content": system_prompt}, {"role": "user", "content": user_prompt}]`
Output	`list[dict]` — 2 messages ready for Ollama API

Purpose: Structure the LLM conversation with strict grounding instructions and the retrieved articles as context.

Step 19 — LLM Call

File: backend/src/llm/rag_repository.py:320-338 Function: RAGRepository.call_llm(messages, stream=False)

Input	2 messages, model=`"qwen3:8b"`, stream=False`
Transport	HTTP POST to `ollama.Client.chat()` at `http://127.0.0.1:11434/api/chat`
Config	`keep_alive="30m", options={"num_ctx": 4096}`
Retry	3 attempts with 0.5s delay between failures
Output	`response.message.content` (string) — e.g., `"The Constitution of Nepal guarantees..."`

Purpose: Generate a natural language answer grounded in the retrieved articles.

Step 20 — Response Assembly

File: backend/src/llm/rag_workflow.py:104-119 Inline in ask()

Input	query, promoted articles, LLM response
Build	<code>`result = {"query": query, "articles": [...], "response": "...", "citations": [...], "ollama_status": {...}}`</code>
<code>`articles`</code>	Filtered through <code>`_build_article_dict()`</code> — only fields the frontend needs
<code>`citations`</code>	<code>`[{"article": citation, "title": title, "doc_id": doc_id}]`</code> for each article
Output	<code>`dict`</code> — the complete JSON payload

Purpose: Assemble the final response payload that the frontend will render.

Step 21 — HTTP Serialization (Backend)

File: backend/controllers/api_controller.py:84 Inline in ask()

Input	Python dict
Flask	<code>`jsonify(payload)`</code> → serializes to JSON string, sets Content-Type: application/json
Output	HTTP response with status 200 and JSON body

Purpose: Send the structured response back to the client over HTTP.

Step 22 — MongoDB Persistence (Articles)

File: backend/services/article_service.py:11-73 Function: ArticleService.create_article(...) Collection: referenced_articles

Input	Each article dict from the response payload
Lookup	<code>`ReferencedArticle.objects(doc_id=doc_id).first()`</code>
If exists	Update: title, citation, scores, text, matched_terms → <code>`save()`</code>
If new	<code>`ReferencedArticle(...)`</code> → <code>`save()`</code> — create new document
Output	<code>`{"success": true, "data": {"id": ObjectId, ...}}`</code>

Transformation: ArticleService.create_article(article_no=31, title="Right relating to education", score=8.47, ...) → MongoDB document in referenced_articles collection.

Step 23 — MongoDB Persistence (Message)

File: backend/services/message_service.py:16-72 Function: MessageService.create_message(...) Collection: messages

Input	user_id, query, answer, list of article ObjectIds
Resolve	<code>`User.objects.get(id=user_id)`</code>
Resolve	<code>`ReferencedArticle.objects.get(id=...)`</code> for each article id
Create	<code>`Message(query=query, answer=answer, user=user, articles=refs).save()`</code>
Output	<code>`{"success": true, "data": message.to_json()}`</code>

Transformation: MessageService.create_message(user_id, query="What are the fundamental rights?", answer="The Constitution guarantees...", articles=[ObjectId("...")]) → MongoDB document in messages collection.

Step 24 — HTTP Deserialization (Frontend)

File: frontend/src/api/client.js Function: apiClient() — response handler

```
const data = await response.json();
```

Transformation: HTTP response body (JSON bytes) → JavaScript object.

Step 25 — State Update & Render (Frontend)

File: frontend/src/components/mainsearchbar.jsx

The response object flows into React state:

```
setResult(data); // data = {query, response, articles, citations, ollama_status}
```

This triggers re-render of Resultdisplay, which passes articles to ArticleCard, which uses HighlightText to mark matched terms.

Transformation: JavaScript object → DOM nodes via React rendering.

Data Shape Summary

Stage	Shape	Size
User input	`"What are the fundamental rights?"`	~35 chars
HTTP request	`{"query": "What...", "use_llm": true}`	~60 bytes
BM25 tokens	`["fundamental", "right"]`	2 tokens
Expanded tokens	`["fundamental", "right", "entitlement", "prerogative"]`	4 tokens
Proximity tokens	`["what", "are", "the", "fundamental", "rights"]`	5 tokens
Query pairs	`[("what", "are"), ..., ("fundamental", "rights")]`	10 pairs
Candidates	`set` of doc IDs	~200-400 IDs
Per-doc score tuple	`(final, bm25, prox, title_count, doc, matched, exact_matched)`	7 fields
Phase 1 results	`list[dict]` of 50 scored docs	~50 KB
Phase 2 results	`list[dict]` of 8 reranked docs	~8 KB
Promoted articles	`list[dict]` of 8 full-article dicts	~16 KB
LLM context	Single formatted string	~4096 tokens
LLM response	Natural language string	~100-500 words
Final JSON payload	`{query, response, articles[8], citations[8], ollama_status}`	~20 KB
HTTP response	JSON body	~20 KB
MongoDB `referenced_articles`	8 documents	~4 KB each
MongoDB `messages`	1 document with 8 refs	~2 KB
React state	JavaScript object	~20 KB
Rendered DOM	Answer markdown + 8 article cards	~200 DOM nodes

State Management

State Inventory

State Type	Location	What It Stores	Lifetime
Database	MongoDB `ECIRAS`	users, messages, referenced_articles	Permanent
Global (module-level)	`services/qa_service.py:_workflow`	Singleton RAGWorkflow instance	Process lifetime
Global (module-level)	`src/core/text_processor.py:_spacy_nlp`	spaCy NLP pipeline	Process lifetime
Singleton (class)	`config/db_connect.py:Database_instance`	MongoDB connection singleton	Process lifetime
In-memory cache	`src/core/reranker.py:_vector_cache`	Sparse TF vectors for MMR	Process lifetime
In-memory cache	`src/llm/rag_repository.py:_ollama_available`	Ollama connectivity status	Process lifetime (1st request)
In-memory cache	`src/llm/rag_repository.py:_available_models`	Ollama model list	Process lifetime (1st request)
In-memory cache	`src/llm/rag_repository.py:_article_lookup`	Pre-built article → text mapping	Process lifetime (startup)
In-memory cache	`src/llm/rag_repository.py:_clause_structure`	Pre-built clause structure mapping	Process lifetime (startup)
HTTP	JWT payload on `request.user`	Authenticated user identity	Per-request
Local	React `useState`	Query input, results, loading flags	Per-component mount
Local	React `useRef`	AbortController for streaming	Per-component mount
Persistent (localStorage)	Frontend `localStorage`	JWT token	Until logout or expiry
Persistent (cookie)	Frontend httpOnly cookie	JWT token (fallback)	Until logout or expiry

Database State (MongoDB — ECIRAS)

Collection: users

Field	Type	Mutability
`fullname`	String	Write-once (registration)
`email`	String	Write-once
`password_hash`	String	Write-once
`role`	Enum	Write-once

`token_version`	Int	**Incremented on every logout**
`created_at`	DateTime	Auto-set
`updated_at`	DateTime	Auto-updated on save

Collection: messages

Field	Type	Mutability
`query`	String	Write-once
`answer`	String	Write-once
`user`	Reference	Write-once
`articles`	List\[Reference\]	Write-once
`created_at`	DateTime	Auto-set
`updated_at`	DateTime	Auto-updated on save

Collection: referenced_articles

Field	Type	Mutability
`doc_id`	String	**Upsert key** — existing doc updated, new doc created
All score fields	Float	Updated on re-query with same `doc_id`
`matched_terms`	List\[String\]	Updated on re-query
Timestamps	DateTime	Auto-managed

Consistency Rules

- Token version invalidation: token_version is the sole source of truth for session validity. When a user logs out, token_version increments. All JWTs carry the version at issue time. On every protected request, the decorator compares jwt.token_version against user.token_version. If the DB version is higher, the JWT is rejected. This is a version-based invalidation, not a blacklist.
- User deletion: CASCADE-deletes all owned Messages (reverse_delete_rule=2).
- Article deletion: NULLIFIES the reference in Messages (reverse_delete_rule=3) — messages survive without the article data.
- No transactions: Each save is atomic, but there is no multi-document transaction. If the server crashes between ArticleService.create_article() and MessageService.create_message(), the orphan article documents remain. This is accepted — articles are immutable snapshots and orphan data has no user-facing impact.

Global / Module-Level State

workflow — services/qa_service.py:20

```
_workflow: Optional[RAGWorkflow] = None
```

- Initialized: Once at startup by init_workflow()
- Accessed: On every /ask and /ask-stream request via _get_workflow()
- Mutability: Read-only after initialization
- Why global: Avoids reconstructing the entire RAG pipeline (EngineFactory → SearchEngine → Reranker → RetrievalWorkflow → RAGRepository → RAGFormatter → RAGWorkflow) on every request. Startup takes

~1s; constructing per-request would make every query slow.

[_spacy_nlp — src/core/text_processor.py:6](#)

```
_spacy_nlp = None
```

- Initialized: Lazy — on first call to `get_spacy_pipeline()` (which happens during `EngineFactory.from_artifacts()` when title tokens are pre-processed)
 - Accessed: By every `TextProcessor.process_text()` call
 - Mutability: Read-only after initialization
 - Why global: spaCy pipeline loading is expensive (~300ms) and memory-heavy (~50MB). One instance per process is the standard pattern.
-

Singleton Pattern

[Database — config/db_connect.py:7-14](#)

```
class Database:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

- Thread safety: Not guarded (`__new__` is not synchronized). In a multi-threaded WSGI server, two threads could pass the `None` check simultaneously and create two instances. In practice, `Database().connect()` is called once during startup (single-threaded), so this never races.
 - Connection pool: `mongoengine.connect()` manages its own pool (`maxPoolSize=10`, `minPoolSize=2`). The `Database` singleton just ensures `connect()` is called exactly once.
 - Why singleton: Prevents duplicate connection pools to the same database.
-

In-Memory Caches

[Reranker._vector_cache — src/core/reranker.py:22](#)

```
self._vector_cache: dict[str, dict[str, int]] = {}
```

- What: Sparse BM25 TF vectors, keyed by `doc_id`. Built lazily on first `_get_tf_vector(doc_id)` call.
- Size: One dict per unique document seen during reranking. Each dict has entries only for terms present in that document. Bounded by corpus size (~700 docs).
- Consistency: Build once, never invalidated. The BM25 index is static (loaded at startup from disk artifacts), so cached vectors are always consistent.
- Why cache: Building a sparse TF vector requires scanning the full `tf_index` (tens of thousands of terms) for each document. Building on every MMR iteration would be $O(\text{candidates} \times \text{terms})$. Caching makes it $O(\text{terms})$ total per request.

[RAGRepository._ollama_available / _available_models — rag_repository.py:54-56](#)

```
self._ollama_available: Optional[bool] = None
self._available_models: list[str] = []
self._connection_status: str = "Not checked yet."
```

- Initialized: `None` — checked lazily on first LLM request
- Mutation: First `_ensure_ollama_checked()` call sets these and never re-checks
- Why cache: Ollama availability doesn't change during a process lifetime. Checking `client.list()` on every

request would add ~200ms to every LLM call.

- Tradeoff: If Ollama is started after the first LLM request, it won't be discovered until the process restarts. This is acceptable for a development tool.

RAGRepository._article_lookup / _clause_structure — rag_repository.py:58-59

```
self._article_lookup: dict[str, dict] = {}
self._clause_structure: dict[str, dict] = {}
```

- Built: In RAGRepository.__init__() via _build_article_lookup()
- Content: Maps article_no → full article text (merged from clauses), and clause structure for context truncation
- Mutability: Read-only after construction
- Why cache: Building the lookup requires iterating all ~700 documents. Doing this per-request would be wasteful.

Per-Request State

request.user — Flask request context

```
request.user = payload # set by @token_required decorator
```

- What: Decoded JWT payload containing user_id, token_version, exp
- Lifetime: Single HTTP request. Flask tears down the request context after the response is sent.
- Accessed by: Controllers (ask(), list_messages(), etc.) to identify the authenticated user.
- Thread safety: Flask's request context is thread-local — each thread sees its own request.

Frontend State

React Component State

Component	State Variable	Type	Purpose
`mainsearchbar.jsx`	`query`	string	Controlled input value
`mainsearchbar.jsx`	`result`	object \	null
`mainsearchbar.jsx`	`loading`	boolean	Show spinner during request
`mainsearchbar.jsx`	`error`	string \	null
`mainsearchbar.jsx`	`streaming`	boolean	Whether SSE is active
`mainsearchbar.jsx`	`cancelled`	boolean	User cancelled streaming
`mainsearchbar.jsx`	`streamingAnswer`	string	Accumulated tokens during stream
`ArticleCard.jsx`	`expanded`	boolean	Card accordion toggle
`LoginPage.jsx`	`email`, `password`	string	Form state
`HistoryPage.jsx`	`messages`	array	Paginated message list
`HistoryPage.jsx`	`page`	number	Current pagination page
`MessageDetailPage.jsx`	`message`	object \	null

React Refs

Component	Ref	Purpose
-----------	-----	---------

`mainsearchbar.jsx`	`abortRef` (AbortController)	Cancels in-flight streaming request
`Dialog.jsx`	`dialogRef`	Focus trap for modal dialog

Auth Context

File: frontend/src/context/AuthProvider.jsx

State	Storage	Purpose
`token`	`localStorage` + React state	JWT for Authorization header
`user`	React state	User profile (fullname, email, id)
`loading`	React state	Initialization flag (checks localStorage on mount)

Persistence: On login, setToken() writes to both localStorage and React state. On logout, both are cleared.

Consistency: The JWT is the source of truth. The localStorage value is read on app mount to restore session across page refreshes. If the JWT expires or is invalidated, the server returns 401 and the api/client.js interceptor clears localStorage and redirects to /login.

URL State

State	Mechanism	Purpose
Current route	`react-router-dom` (BrowserRouter)	Page navigation, history
`message_id`	URL param `/messages/:id`	Identify which message to load

State Consistency Summary

Concern	Mechanism
Auth invalidation	Token version comparison (DB is truth, JWT carries version snapshot)
No dirty reads	All DB writes precede response; persistence is fire-and-forget after response assembled
Cache staleness	`_ollama_available` never re-checks (intentional — Ollama is static per session)
Thread safety (global)	`_workflow`, `_spacy_nlp`, caches are read-only after init; no writes during request handling
Thread safety (singleton)	`Database.__new__` unsynchronized but safe — called once during startup
Frontend session	JWT in `localStorage` — single source of truth; cookie is fallback
Race conditions	None — every request is synchronous within its thread; no shared mutable state is written during requests

State Diagram: Lifecycle of a JWT

```
Registration
↓
User created → token_version = 0
↓
Login
↓
JWT issued with {user_id, token_version=0, exp=12h}
JWT → localStorage (frontend)
JWT → httpOnly cookie (backend)
↓
Every /ask request:
  JWT presented → jwt.decode() → payload.token_version=0
  User lookup → user.token_version=0 → match → allowed
↓
Logout
↓
User.token_version incremented → 1
↓
Next /ask request:
  JWT presented → payload.token_version=0
  User lookup → user.token_version=1 → mismatch → 401
↓
Frontend clears localStorage, redirects to /login
```

Consistency property: The logout is immediately effective for all existing JWTs because the version comparison hits the database on every request. There is no propagation delay, no token blacklist, no expiry wait.

Design Patterns

1. Factory (Static) — EngineFactory.from_artifacts()

Aspect	Detail
Where	<code>`backend/src/core/engine_factory.py:42`</code>
What*	<code>`EngineFactory.from_artifacts(docs_path, index_dir, synonyms_path)`</code> returns a fully-wired <code>`SearchEngine`</code>
Why chosen*	Construction of a <code>`SearchEngine`</code> requires loading 4 JSON files from disk, creating 2 <code>`TextProcessor`</code> instances with different configurations, optionally loading a <code>`QueryExpander`</code> , and wiring 2 scorers. Exposing this in a constructor would force every caller to know about file formats and index structure. A factory encapsulates the entire assembly.
Benefits**	Callers get a ready-to-use engine with one call. File-not-found errors are caught and reported with clear messages. Construction logic is centralized, not duplicated across tests, CLI scripts, and <code>`init_workflow()`</code> .
Drawbacks**	Static factory cannot be mocked or subclassed. Tests that need a <code>`SearchEngine`</code> must either provide real files or bypass the factory entirely.

2. Factory (Constructor Injection) — RAGRepository.__init__()

Aspect	Detail
Where	<code>`backend/src/llm/rag_repository.py:40`</code>
What**	<code>`RAGRepository`</code> constructor reads <code>`OLLAMA_HOST`</code> , <code>`OLLAMA_API_KEY`</code> , <code>`OLLAMA_MODEL`</code> from env vars and creates the <code>`ollama.Client`</code> internally
Why chosen**	The Ollama client configuration is entirely environment-driven. Rather than forcing callers to pass host/api_key/model every time, the constructor reads them from the environment. Reduces boilerplate at every call site.

Benefits	Simple for callers — just pass the <code>RetrievalWorkflow</code> . Environment changes don't require code changes.
Drawbacks	Hidden dependency on environment variables makes testing harder (must set env vars or mock at the <code>ollama.Client</code> layer). The constructor has side effects (reading env vars) that can't be bypassed.

3. Strategy — Scoring Algorithms

Aspect	Detail
Where	<code>backend/src/core/bm25_scorer.py` (BM25Scorer), <code>backend/src/core/proximity.py` (ProximityScorer), <code>backend/src/core/reranker.py` (Reranker — RRF, MMR, boost)</code></code></code>
What	Each scoring algorithm is a self-contained class implementing a specific computation. They are composed by <code>SearchEngine</code> and <code>RetrievalWorkflow</code> interchangeably.
Why choose	Each scoring signal (BM25, proximity, RRF fusion, MMR diversity, rule boost) is independently testable, replaceable, and tunable. Adding a new signal (e.g., embedding similarity) means writing a new scorer class without touching existing ones.
Benefits	High cohesion, low coupling. Each scorer has a single responsibility. BM25 can be swapped for BM25L or BM25+ by replacing one class.
Drawbacks	No common interface — each scorer has its own signature (<code>score(tokens, doc_id)</code> vs <code>score(doc_id, pairs)</code> vs <code>rerank(results)</code>). A formal interface would improve substitutability.

4. Template Method — `SearchEngine.search()`

Aspect	Detail
Where	<code>backend/src/core/search_engine.py:79`</code>
What	<code>search()</code> defines the invariant pipeline skeleton: tokenize → expand → generate candidates → score each → sort → format. Subclasses or callers inject the scoring strategy.
Why choose	Every search follows the same high-level sequence. The pipeline structure is fixed, but the scoring function could vary. The template method keeps the sequence in one place while allowing the scoring details to vary.
Benefits	The search algorithm is easy to follow — you read one method to understand the entire flow. New scoring signals slot in without changing the pipeline structure.
Drawbacks	Not a classic template method (no abstract subclass hooks). The scoring is determined by constructor-injected scorers rather than overridden methods. This is more **Strategy + Template Method** hybrid.

5. Facade — `QAService`, `RAGWorkflow`, `RetrievalWorkflow`

Aspect	Detail
Where	<code>`services/qa_service.py:35`,</code> <code>`src/llm/rag_workflow.py:20`,</code> <code>`src/workflows/retrieval_workflow.py:5`</code>
What	Each facade hides a complex subsystem behind a simple API:
- <code>`QAService`</code> → hides <code>EngineFactory</code> , <code>Reranker</code> , <code>RetrievalWorkflow</code> , <code>RAGRepository</code> , <code>RAGFormatter</code> , <code>ArticleService</code> , <code>MessageService</code>	
- <code>`RAGWorkflow`</code> → hides <code>RAGRepository</code> (retrieval + Ollama), <code>RAGFormatter</code>	
- <code>`RetrievalWorkflow`</code> → hides <code>SearchEngine</code> + <code>Reranker</code>	
Why chosen	The inner pipeline has many moving parts. Without facades, controllers would need to wire 8+ objects manually. Each facade defines a clear subsystem boundary and reduces cognitive load.
Benefits	Controllers are thin. The pipeline is layered: <code>`ask()`</code> → <code>`QAService`</code> → <code>`RAGWorkflow`</code> → <code>`RAGRepository`</code> → <code>`RetrievalWorkflow`</code> → <code>`SearchEngine`</code> + <code>`Reranker`</code> . Each layer hides the complexity below it.
Drawbacks	The facade layers add indirection. Debugging a query requires tracing through 5-6 delegation calls before reaching actual computation. The <code>`QAService`</code> facade is particularly thick — it both assembles the pipeline and handles persistence.

6. Repository — RAGRepository

Aspect	Detail
Where	<code>`backend/src/llm/rag_repository.py:33`</code>
What	<code>`RAGRepository`</code> abstracts two data sources: the retrieval pipeline (delegated to <code>`RetrievalWorkflow`</code>) and the LLM (Ollama client). Callers interact with <code>`retrieve()`</code> and <code>`call_llm()`</code> without knowing how either is implemented.
Why chosen	The Q&A workflow needs articles and an LLM answer. The repository pattern separates "what data is needed" from "where it comes from." If Ollama is replaced with an OpenAI API call, only <code>`RAGRepository.call_llm()`</code> changes.
Benefits	Clear data-access boundary. The repository owns connectivity caching, retry logic, and article lookup — none of which leak into the workflow.
Drawbacks	The name "Repository" is stretched — it doesn't mediate between domain and data mapping as in DDD. It mixes two different data sources (search + LLM) in one class. A stricter design would split into <code>`RetrievalRepository`</code> and <code>`LLMRepository`</code> .

7. Active Record — MongoDB Models

Aspect	Detail
Where	<code>`backend/models/user_model.py:12` (User), `message_model.py:7` (Message), `referenced_article_model.py:22` (ReferencedArticle)</code>
What	Each model class combines data fields, persistence methods (<code>`save()`</code>), query logic (<code>`objects.get()`</code>), and business methods (<code>`set_password()`</code> , <code>`check_password()`</code> , <code>`to_json()`</code>) in a single class.
Why chosen	mongoengine is an Active Record ODM by design. For a project with simple CRUD and few model relationships, Active Record avoids the ceremony of a separate data-mapper layer.
Benefits	Rapid development — one class defines schema, queries, and behavior. No repository interfaces, no unit-of-work, no database abstraction layer.
Drawbacks	Violates Single Responsibility Principle. <code>`User`</code> handles persistence, password hashing, serialization, and validation. Testing business logic requires a running MongoDB (or extensive mocking). The models have no interface — swapping MongoDB for another database would require rewriting every model.

8. Singleton — Database

Aspect	Detail
Where	<code>`backend/config/db_connect.py:7`</code>
What	<code>`Database._instance`</code> class variable with <code>`__new__`</code> guard ensures exactly one instance per process
Why chosen	MongoDB connection pools must not be duplicated. The singleton guarantees <code>`mongoengine.connect()`</code> is called exactly once.
Benefits	Simple, well-understood, trivially correct for this use case.
Drawbacks	The <code>`__new__`</code> implementation is not thread-safe (two threads could pass the <code>`None`</code> check simultaneously). Safe in practice because <code>`connect()`</code> is called once during startup before any request threads exist. Harder to mock in tests compared to dependency injection.

9. Lazy Initialization — spaCy Pipeline, Ollama Check, TF Vector Cache

Aspect	Detail
Where	<code>`text_processor.py:8` (`get_spacy_pipeline`), `rag_repository.py:291` (`_ensure_ollama_checked`), `reranker.py:29` (`_get_tf_vector`)</code>
What	Expensive resources are initialized on first use, not at construction time.
Why chosen	The spaCy pipeline (~300ms, ~50MB) should not load if the IR engine is used only for offline ingestion. The Ollama check (~200ms) should not run if the user never enables LLM. TF vectors for MMR should not be built for documents that never appear in reranking. Lazy init defers cost until value is certain.
Benefits	Faster startup, lower memory when features are unused. The Ollama check is especially important — without lazy init, an unavailable Ollama would cause a visible startup warning or crash.
Drawbacks	First request pays the initialization cost (transparent to the user but visible in logs). Stateful lazy init (<code>_spacy_nlp`, `_ollama_available`)</code> is module-level global, complicating test isolation.

10. Decorator — @token_required

Aspect	Detail
Where	<code>`backend/controllers/decorators.py:28`</code>
What	<code>`@token_required`</code> wraps route handlers with JWT validation logic
Why chosen	Authentication is a cross-cutting concern. Applying it as a decorator keeps route handlers clean of auth code and makes the auth requirement visible at the route definition.
Benefits	Eight protected routes share one auth implementation. The decorator extracts token from two sources (header + cookie), decodes JWT, checks token_version, and attaches <code>request.user`</code> . Controllers just read <code>request.user.get("user_id")`</code> .
Drawbacks	The decorator hits MongoDB on every request (<code>_get_user`</code> for token_version check). For high-traffic endpoints, this adds latency. A token-blacklist cache could avoid the DB hit but introduces complexity.

11. DTO (Data Transfer Object) — Document

Aspect	Detail
Where	<code>`backend/src/core/document.py:4`</code>
What	<code>`Document`</code> is a <code>@dataclass</code> with 14 fields and no behavior. It carries data between the ingestion pipeline, the search engine, and the reranker.
Why chosen	A typed data object is safer than passing raw dicts. The dataclass gives field validation, IDE autocompletion, and explicit field names.
Benefits	Type safety without boilerplate. The dataclass is trivially constructable from JSON (<code>Document(**item)`</code>).
Drawbacks	Not used everywhere — the SearchEngine returns <code>list[dict]</code> , not <code>list[Document]</code> . The dataclass is passed into the engine but not out. Inconsistent application of the pattern.

12. Pipeline — Reranker.rerank()

Aspect	Detail
Where	<code>`backend/src/core/reranker.py:164`</code>
What	<code>`rerank()`</code> runs three stages sequentially: RRF fusion → MMR diversity → rule boost. Each stage is a private method.
Why chosen	Three independent transformations applied in sequence. The pipeline makes the ordering explicit and allows adding/removing stages without changing the overall structure.
Benefits	The three stages are independently testable. Each stage has a clear input and output. Adding a fourth stage (e.g., semantic reranking) means writing one method and adding one call.
Drawbacks	Data is mutated in place (dicts are modified and returned) rather than creating immutable copies. This is fine within a single request but would be problematic if results were shared across threads.

13. Builder — RAGFormatter

Aspect	Detail
Where	<code>`backend/src/llm/rag_formatter.py:1`</code>
What	Three methods (<code>format_context()</code> , <code>build_system_prompt()</code> , <code>build_user_prompt()</code>) construct different parts of the LLM prompt independently.
Why chosen	The LLM prompt has three distinct parts (context, system instructions, user query). Builder separates construction of each part, making prompt changes isolated and testable.
Benefits	The system prompt can be updated without touching context formatting. The user prompt template can be changed without affecting system instructions.

Draw backs	No director object — callers orchestrate the three builds manually. This is fine for two call sites (<code>ask()</code> and <code>ask_streaming()</code>) but would benefit from a director if more callers were added.
-----------------------	---

14. Adapter (Implicit) — RAGRepository wrapping ollama.Client

Aspect	Detail
Where	<code>`backend/src/llm/rag_repository.py:52`</code>
What	<code>`RAGRepository`</code> wraps <code>`ollama.Client`</code> and provides retry logic, connectivity caching, and a simpler interface (<code>`call_llm(messages)`</code> vs <code>`client.chat(model, messages, ...)`</code>).
Why chosen	The raw Ollama SDK exposes too many details (model name, context window, <code>keep_alive</code>). The adapter provides a domain-specific interface that callers (<code>`RAGWorkflow`</code>) can use without knowing about Ollama.
Benefits	Switching from Ollama to OpenAI requires changing only the adapter internals. The retry logic and connectivity caching are in one place.
Draw backs	The adapter is tightly coupled to the Ollama response format (<code>`response.message.content`</code>). A different LLM provider would likely return a different response shape.

15. Module-Level Singleton — `_workflow` in `qa_service.py`

Aspect	Detail
Where	<code>`backend/services/qa_service.py:20`</code>
What	<code>`_workflow: Optional[RAGWorkflow] = None`</code> — the entire RAG pipeline is assembled once and stored as a module-level variable
Why chosen	Alternative to a class-based singleton. There must be exactly one pipeline per process. A module variable is simpler than a class Singleton with <code>`__new__`</code> and works the same way in practice.
Benefits	Dead simple — one global, one setter (<code>`init_workflow()`</code>), one getter (<code>`_get_workflow()`</code>). No locks, no class boilerplate.
Draw backs	Module-level state is invisible to callers. Tests must explicitly reset the global between test cases. If <code>`init_workflow()`</code> raises, the module is in an undefined state.

16. Composite — Frontend UI Primitives

Aspect	Detail
Where	<code>`frontend/src/components/ui/`</code> (9 components)
What	Pages compose feature components (<code>`MainSearchBar`</code> , <code>`Resultdisplay`</code>), which compose UI primitives (<code>`Button`</code> , <code>`Card`</code> , <code>`Spinner`</code> , etc.). Components form a tree.

**Why chosen* *	Natural in React — every component is a composable unit. The UI primitives provide a consistent baseline (all buttons look the same) while allowing feature components to vary layout.
Benefits	Visual consistency without CSS duplication. Primitives handle accessibility (aria attributes, focus management, keyboard events).
Drawbacks	The composition is React's default, not a deliberate pattern choice. The "composite" label describes what React already does, not a novel design decision.

Pattern Density Summary

Pattern	Location	Strictness
Factory (Static)	<code>`EngineFactory.from_artifacts()`</code>	
Factory (Constructor)	<code>`RAGRepository.__init__()`</code> (env vars)	
Strategy	BM25Scorer, ProximityScorer, Reranker stages	
Template Method	<code>`SearchEngine.search()`</code>	
Facade	QAService, RAGWorkflow, RetrievalWorkflow	
Repository	RAGRepository	
Active Record	User, Message, ReferencedArticle (mongoengine)	
Singleton	Database	
Lazy Initialization	spaCy, Ollama check, TF vector cache	
Decorator	@token_required	
DTO	Document dataclass	
Pipeline	Reranker (RRF → MMR → boost)	
Builder	RAGFormatter	
Adapter	RAGRepository wrapping ollama.Client	
Module Singleton	<code>`_workflow`</code> in qa_service.py	
Composite	Frontend UI primitives	

Not used: Event Bus, CQRS, Observer, Chain of Responsibility, Proxy, Flyweight, Visitor, Interpreter, Mediator, Memento, State.

Code Reading Guide

Goal: Maximize understanding with minimal jumping. Each file builds on the previous one.

Phase 1 — Surface Layer (4 files)

Understand what the system does and how to run it.

1. [docs/understanding/01_SYSTEM_OVERVIEW.md](#)
 2. [docs/understanding/02_REPOSITORY_TOUR.md](#)
 3. [backend/app.py](#)
 4. [frontend/src/App.jsx](#)
-

Phase 2 — Request Flow (4 files)

Understand how a request travels through the system.

5. [backend/routes/api_routes.py](#)
 6. [backend/controllers/api_controller.py](#)
 7. [backend/controllers/decorators.py](#)
 8. [backend/services/qa_service.py](#)
-

Phase 3 — Domain Core (6 files)

Understand the IR engine — the heart of the system.

9. [backend/src/core/document.py](#)
 10. [backend/src/core/text_processor.py](#)
 11. [backend/src/core/bm25_scorer.py](#)
 12. [backend/src/core/proximity.py](#)
 13. [backend/src/core/search_engine.py](#)
 14. [backend/src/core/reranker.py](#)
-

Phase 4 — Workflow Composition (3 files)

Understand how the pieces fit together.

15. [backend/src/workflows/retrieval_workflow.py](#)
 16. [backend/src/llm/rag_repository.py](#)
 17. [backend/src/llm/rag_workflow.py](#)
-

Phase 5 — Persistence & Auth (3 files)

Understand how data is stored and users are managed.

- 18. [backend/services/article_service.py](#)
 - 19. [backend/services/message_service.py](#)
 - 20. [backend/controllers/auth_controller.py](#)
-

Phase 6 — Frontend (4 files)

Understand the UI surface.

- 21. [frontend/src/api/client.js](#)
 - 22. [frontend/src/context/AuthProvider.jsx](#)
 - 23. [frontend/src/hooks/useAskStream.js](#)
 - 24. [frontend/src/components/mainsearchbar.jsx](#)
-

Phase 7 — Deeper Dives (4 files)

For readers who want to understand the edges.

- 25. [backend/src/core/engine_factory.py](#)
 - 26. [backend/src/core/query_expander.py](#)
 - 27. [backend/src/core/index_builder.py](#)
 - 28. [backend/src/llm/rag_formatter.py](#)
-

Suggested Reading Order

Phase 1 – Surface

- └─ 01_SYSTEM_OVERVIEW.md
- └─ 02_REPOSITORY_TOUR.md
- └─ backend/app.py
- └─ frontend/src/App.jsx

Phase 2 – Request Flow

- └─ backend/routes/api_routes.py
- └─ backend/controllers/api_controller.py
- └─ backend/controllers/decorators.py
- └─ backend/services/qa_service.py

Phase 3 – Domain Core

- └─ backend/src/core/document.py
- └─ backend/src/core/text_processor.py
- └─ backend/src/core/bm25_scorer.py
- └─ backend/src/core/proximity.py
- └─ backend/src/core/search_engine.py
- └─ backend/src/core/reranker.py

Phase 4 – Workflow Composition

- └─ backend/src/workflows/retrieval_workflow.py
- └─ backend/src/llm/rag_repository.py
- └─ backend/src/llm/rag_workflow.py

Phase 5 – Persistence & Auth

- └─ backend/services/article_service.py
- └─ backend/services/message_service.py
- └─ backend/controllers/auth_controller.py

Phase 6 – Frontend

- └─ frontend/src/api/client.js
- └─ frontend/src/context/AuthProvider.jsx
- └─ frontend/src/hooks/useAskStream.js
- └─ frontend/src/components/mainsearchbar.jsx

Total 28 files across 7 phases, progressing from 5-minute overviews to 30-minute deep dives.

- └─ backend/src/core/engine_factory.py
- └─ backend/src/core/query_expander.py
- └─ backend/src/core/index_builder.py
- └─ backend/src/llm/rag_formatter.py

Deep Dive — File-by-File Analysis

Each file analyzed below follows the same structure: Purpose → Dependencies → Public API → Internal Structure → Main Algorithms → Important Functions → Design Decisions → Common Pitfalls.

Say "next" to proceed to the next file.

File 1: backend/src/core/document.py (22 lines)

Purpose: Typed data container for a single constitutional provision in the flattened corpus. Every one of the ~700 documents in the corpus is an instance of this dataclass.

Dependencies: None — pure Python dataclasses.

Public API: The Document dataclass with 14 fields. All fields are positional constructor args.

Internal structure:

```
doc_id: str          - Unique key (e.g., "1_1_2" for Part 1, Article 1, Clause 2)
part_no: str         - Part number (as string, e.g., "1", "3")
article_no: str      - Article number
title: str           - Article title (e.g., "Right relating to education")
text: str            - Enriched provision text (includes header prefix)
citation: str         - Display citation (e.g., "Part 3, Article 31")
level: str           - "part", "article", "clause", or "subclause"
clause_no: Optional[str] - Clause identifier within an article
subclause_id: Optional[str]
is_primary: bool     - Is this the primary representation of the article?
parent_id: Optional[str]
raw_text: Optional[str] - Original un-enriched text (if available)
citation_normalized: Optional[str]
boost: float         - Document-level relevance boost multiplier (default 1.0)
```

Main algorithms: None — pure data.

Important functions: None beyond the constructor.

Design decisions:

- level as a string rather than an enum — simpler but allows invalid values at runtime.
- boost has default 1.0 — documents are equal unless explicitly weighted.
- raw_text and citation_normalized are Optional — they may be missing from older corpus formats.

Common pitfalls:

- doc_id uniqueness is not enforced by the dataclass; duplicates in the corpus list cause silent overwrites during index building.
 - Mixing string and optional types for part_no/article_no means consumers must handle type coercion (e.g., in sorting).
-

File 2: backend/src/core/text_processor.py (110 lines)

Purpose: NLP pipeline for normalizing, lemmatizing, and filtering text. Two instances exist at runtime with different configurations (BM25 processor vs proximity processor).

Dependencies: constants/contraction_map.py (57 contractions), constants/stopwords.py (~120 stopwords), spaCy (en_core_web_sm with fallback to blank("en")).

Public API:

- process_text(text: str) -> list[str] — full pipeline: normalize → lemmatize (optional) → stopword removal (optional)

- `normalize_text(text: str) -> str` — lowercase, expand contractions, alphanumeric+whitespace only
- `lemmatize_tokens(tokens: list[str]) -> list[str]` — spaCy lemmatization
- `_filter_stopwords(tokens)`, `_expand_contractions(text)` — internal steps

Internal structure:

```
process_text():
  |— normalize_text() → lowercase, contractions, char filter
  |   |— _expand_contractions() → regex substitution from CONTRACTIONS_MAP
  |   |   |— ''.join(ch for ch if isalnum or isspace)
  |— split()
  |— [lemmatize_tokens()] → spaCy pipeline, pron→text fallback
  |— [_filter_stopwords()] → set lookup
```

Main algorithms:

- Contraction expansion: Compiled regex of 57 contractions sorted by descending length. `\b(couldn't've|...)\b` → lookup in map. Sorting by length ensures longer contractions match before their prefixes.
- Lemmatization fallback: If `en_core_web_sm` is not installed, uses `spacy.blank("en")` — tokenization still works but lemmas are identity (no actual lemmatization).

Important functions:

- `get_spacy_pipeline()` — module-level singleton. Loads once, caches in `_spacy_nlp`. This is called at app startup explicitly in `app.py:get_spacy_pipeline()` to avoid slow first-request latency.

Design decisions:

- Two-processor architecture: BM25 processor (lemmatize ON, stopwords REMOVED) vs proximity processor (lemmatize OFF, stopwords KEPT). This is the single most important design choice — it recognizes that BM25 benefits from conflation ("rights"→"right") while proximity matching needs exact token positions including stopwords to capture phrase structure.
- -pron- lemmas are mapped back to the original token text. spaCy uses -pron- for pronouns; mapping back preserves the original word.
- Module-level `_spacy_nlp` singleton avoids reloading the spaCy model per request or per `TextProcessor` instance.

Common pitfalls:

- `_expand_contractions` uses `str.replace` operations via regex, not a simple dict lookup. If `CONTRACTIONS_MAP` values contain punctuation, the subsequent `isalnum` filter may strip them.
- spaCy model download is an implicit dependency. If the user runs without `en_core_web_sm`, lemmatization silently degrades (tokens pass through unchanged).
- The static `_pattern` cache on `TextProcessor._build_contraction_pattern()` — it's shared across all instances, which is correct but unusual.

File 3: `backend/src/core/bm25_scorer.py` (46 lines)

Purpose: Compute BM25 relevance scores between a query and a document. Primary ranking signal.

Dependencies: None — pure Python `math.log`.

Public API:

- `score(query_tokens, doc_id) -> float` — BM25 score for a document given query tokens
- `matched_terms(query_tokens, doc_id) -> list[str]` — which query terms exist in the document
- `idf(term) -> float` — inverse document frequency

Internal structure: The constructor takes:

- `tf_index: dict[term → dict[doc_id → term_frequency]]`

- doc_lengths: dict[doc_id → token_count]
- avgdl: average document length across the corpus
- k1 (1.5), b (1.0)

Main algorithms:

BM25 formula:

$$\text{score}(D, Q) = \sum \text{idf}(t) * (\text{tf}(t,D) * (k1+1)) / (\text{tf}(t,D) + k1 * (1 - b + b * |D|/\text{avgdl}))$$

$$\text{idf}(t) = \log((N - \text{df}(t) + 0.5) / (\text{df}(t) + 0.5) + 1)$$

All components are standard BM25 Okapi. Notable: b=1.0 means full document length normalization — the uncommon choice that penalizes long documents proportionally.

Important functions:

- score() — iterates query terms, skips terms with tf=0, accumulates with IDF
- idf() — edge-case guard: if df==0 returns 0.0 (unseen terms contribute nothing)

Design decisions:

- b=1.0 rather than the conventional 0.75. Legal documents vary hugely in length (titles: 3 words, multi-clause articles: 500+ words). Full normalization prevents verbose articles from dominating short dense provisions purely by term volume.
- No get_scores() or batch scoring — each document is scored individually in SearchEngine. This is $O(n * m)$ where n = candidates, m = query terms, but with recall_k=30 it's fast enough.

Common pitfalls:

- doc_len=0 returns 0.0 — division by zero guard.
- tf=0 is skipped, not added as zero — avoids unnecessary IDF lookups.
- matched_terms() uses get(doc_id, 0) > 0 — checks presence, not count.

File 4: backend/src/core/proximity.py (119 lines)

Purpose: Score document relevance by how close query term pairs appear in the document text. Captures phrase-level signal that BM25 cannot see.

Dependencies: json, pathlib.Path (for load/save helpers).

Public API:

- score(doc_id, pairs, max_window=30, ordered=True) -> float — average quadratic inverse proximity score
- generate_query_pairs(tokens) -> list[tuple[str,str]] — pair generation heuristic
- generate_all_pairs(tokens), generate_adjacent_pairs(tokens) — pair generators
- load_index(path), save_index(path) — serialization

Internal structure: The positional index is a nested dict: term → doc_id → list[positions]. Each position is a 0-indexed token offset in the document.

Main algorithms:

Pair generation heuristic:

- ≤ 5 query tokens: all unordered pairs ($O(n^2/2)$)
- > 5 query tokens: adjacent pairs only ($O(n-1)$)

This prevents $O(n^2)$ blowup for long queries where distant pairs contribute negligible signal.

Minimum ordered distance (two-pointer sweep):

```

i=0, j=0
while i < len(pos1) and j < len(pos2):
    if pos1[i] < pos2[j]:
        dist = pos2[j] - pos1[i]
        i++
    else:
        j++

```

Finds the minimum distance where term1 appears before term2. For unordered, take $\text{abs}(\text{pos1}[i] - \text{pos2}[j])$ and advance the smaller pointer.

Score per pair: $1 / (\text{distance} + 1)^2$ — quadratic inverse: close pairs contribute much more than far pairs. A pair at distance 0 scores 1.0; distance 9 scores 0.01.

Average over all valid pairs (pairs where both terms appear in the document within max_window tokens).

Important functions:

- `_min_ordered_distance()` — the core algorithm. Used by default (`ordered=true`).
- `_min_distance()` — unordered variant, used when `ordered=false`.

Design decisions:

- Ordered distance: By default, term1 must appear before term2. This captures phrase order naturally: "right to education" matches, "education right" does not (unless both orders exist in the text).
- Max window = 30 tokens: Pairs separated by more than 30 tokens are discarded. In legal text, related terms rarely appear more than 30 tokens apart within a clause.
- Pair heuristic threshold at 5 tokens: Empirical choice. Short queries benefit from full cross-product; beyond 5, adjacent pairs are sufficient.

Common pitfalls:

- Equal terms in a pair (`term1 == term2`) are skipped — prevents self-pair scoring.
- If a term is missing from the positional index, the pair is silently skipped (no error).
- `max_window` check happens after distance computation — for very distant pairs, this is wasted work.

File 5: backend/src/core/query_expander.py (71 lines)

Purpose: Expand query tokens with synonyms from 44 legal synonym groups to improve recall.

Dependencies: json (loads data/synonyms.json).

Public API:

- `expand(tokens, raw_query="")` -> `list[str]` — expanded token list preserving order and uniqueness

Internal structure: Synonym groups are loaded as `data["groups"]` — each group is a list of terms (e.g., `["arrest", "detention", "custody"]`).

`_build_lookup()` creates:

- `self.lookup`: `word` → `group_idx` → `set[synonyms]`
- `self.multi_word_entries`: `group_idx` → `list[multi-word phrases]`

Main algorithms:

Expansion:

```

for each token:
    add token to result (if not seen)
    if token in lookup:
        for each group containing token:
            if group has multi-word phrases and raw_query doesn't contain them:
                skip this group (don't expand multi-word without evidence)
            add all synonyms from group to result (if not seen)

```

Multi-word phrase guard: If a synonym group contains phrases like "right to information", but the user's raw query doesn't contain any such phrase, the group is not expanded. This prevents over-expansion (e.g., expanding "right" to include "right to information" when the user just asked about "rights").

Design decisions:

- Only applied to BM25 tokens (not proximity tokens). Synonyms improve recall for BM25 scoring but would break the positional signal if applied to proximity matching.
- Deduplication in `_add_if_new()` uses a set for $O(1)$ lookup while maintaining insertion order.
- Multi-word phrase check is against the **raw query**, not the tokenized version — phrases with stopwords like "right to information" must match the unprocessed query.

Common pitfalls:

- `_normalize()` applies the same alpha-only filter as `TextProcessor.normalize_text()` — if a synonym contains digits or special chars, they are stripped.
 - Synonym expansion can increase the token count significantly (one token may expand to 10+ synonyms). This expands the candidate set but also increases scoring work.
-

File 6: backend/src/core/engine_factory.py (105 lines)

Purpose: Assembles a fully-wired SearchEngine from disk artifacts.

Dependencies: All `src.core` modules — `Document`, `TextProcessor`, `BM25Scorer`, `ProximityScorer`, `SearchEngine`, `QueryExpander`.

Public API:

- `EngineFactory.from_artifacts(documents_path, index_dir, proximity_weight=1.0, title_boost=5.0, synonyms_path=None) -> SearchEngine`

Internal structure:

1. Load flattened documents JSON → list of `Document` objects
2. Load `tf_index.json`, `pos_index.json`, `doc_stats.json` from `index_dir`
3. Extract `doc_lengths` and `avgdl` from stats
4. Create two `TextProcessor` instances (BM25 config, proximity config)
5. Optionally create `QueryExpander` from synonyms
6. Create `BM25Scorer` and `ProximityScorer`
7. Construct and return `SearchEngine`

Main algorithms: Linear assembly — no complex algorithms.

Important functions:

- `_load_json(path, label)` — loads JSON with error messages for `FileNotFoundError` and `JSONDecodeError`. Used by both the documents file and the three index files.

Design decisions:

- Static factory pattern — all methods are static. No state, no need for instantiation. This is the purest form of Factory Method in the codebase.
- Errors are propagated (not swallowed). If any index file is missing, the server fails at startup rather than at request time.
- The caller is responsible for providing correct paths — no path discovery or convention-based lookup.

Common pitfalls:

- If the documents file contains duplicate `doc_id` values, `Document` objects are created but duplicates are not detected.
- The `synonyms_path` is optional; without it, no synonym expansion occurs and the system queries the raw BM25 tokens only.
- The `SearchEngine` constructor parameter order must match — 10 positional arguments.

File 7: backend/src/core/search_engine.py (204 lines)

Purpose: The central retrieval pipeline — orchestrates BM25 scoring, title boosting, proximity scoring, candidate generation, and result formatting.

Dependencies: BM25Scorer, ProximityScorer, TextProcessor, QueryExpander, Document.

Public API:

- `search(query, top_k=None, proximity_weight=None, title_boost=None) -> list[dict]`

Internal structure:

```
search(query):
    1. base_tokens = bm25_processor.process_text(query)
       bm25_tokens = base_tokens + synonym_expander.expand(base_tokens)
    2. raw_tokens = proximity_processor.process_text(query) # no lemmatization, no stopword removal
    3. query_pairs = ProximityScorer.generate_query_pairs(raw_tokens)
    4. candidates = union of tf_index[t].keys() for each t in bm25_tokens
    5. for each doc where doc.doc_id in candidates:
        bm25 = BM25Scorer.score(bm25_tokens, doc.doc_id)
        if bm25 == 0: skip
        title_bonus = len(set(bm25_tokens) & title_tokens[doc.doc_id]) * 5.0
        prox = ProximityScorer.score(doc.doc_id, query_pairs, max_window=30, ordered=True)
        final = bm25 + title_bonus + 1.0 * prox
    6. sort DESC, return top-k
```

Important functions:

- `_score_document()` — computes the triple-signal score for one document. Returns a tuple of (final, bm25, prox, title_matches, doc, matched_terms, exact_matched_terms).
- `_generate_candidates()` — set union of all doc_ids that contain any query token in the BM25 index.
- `_prepare_proximity_query()` — delegates to proximity processor (no lemmatization, stopwords preserved).
- `_format_results()` — converts internal tuples into API-ready dicts.

Design decisions:

- Title tokens pre-computed in `__init__` (`self.title_tokens`). For every doc, title is tokenized with the BM25 processor once at startup. This avoids per-query tokenization of all document titles.
- Synonym expansion only on BM25 tokens. Proximity tokens are never expanded — synonyms would create artificial pairs that don't exist in the text.
- Matched terms tracked at two levels: `matched_terms` (lemmatized BM25 tokens) for backend analysis, `exact_matched_terms` (original query tokens) for frontend highlighting.
- Zero-score results are excluded (line 122: `if result[0] > 0`). This means documents that match via synonyms but have zero BM25 score are excluded — though this shouldn't happen since synonym expansion only adds tokens already in the BM25 index.

Common pitfalls:

- Title tokens are pre-computed with the BM25 processor. If `synonym_expander` is present, it's applied to the query during search but NOT to the title tokens. So title boost is computed on base tokens + synonyms, matched against pre-lemmatized title tokens.
- The `top_k` parameter in `search()` is typically `recall_k` (30-50), not the final number of results — final truncation happens in `Reranker`.
- `_score_document` returns 0.0 for all signals if BM25 score is 0.0 — early exit skip.

File 8: backend/src/core/reranker.py (180 lines)

Purpose: Three-stage reranking: RRF signal fusion → MMR diversity → rule-based boost. Pure math — no ML or

embeddings.

Dependencies: math (for sqrt).

Public API:

- rerank(results, top_k=8, boost_rules=None) -> list[dict]

Internal structure:

```
rerank():  
    res = _rrf_fuse(results)  
    res = _mmr_diversify(res)  
    res = _apply_boost(res, boost_rules)  
    return res[:top_k]
```

Stage 1 — RRF Fusion (_rrf_fuse):

- Rank the top recall_k results independently by bm25_score, proximity_score, and title_match_count.
- For each result: $\text{rrf_score} = 1/(k+\text{rank_bm25}) + 1/(k+\text{rank_prox}) + 1/(k+\text{rank_title})$
- $k=60$ (inverse rank penalty — larger k means ranks matter less).
- Sort by rrf_score DESC.

Stage 2 — MMR Diversity (_mmr_diversify):

- Start with the highest-RRF result.
- For each remaining candidate: $\text{mmr} = \lambda \cdot \text{score} - (1-\lambda) \cdot \text{max_similarity_to_selected}$
- Cosine similarity on BM25 TF vectors (sparse, computed from tf_index).
- $\lambda=0.5$ — equal weight for relevance and diversity.
- Pick the highest-MMR candidate, add to selected, repeat.

Stage 3 — Rule-Based Boost (_apply_boost):

- $\text{final_score} = \text{original_score} \times \text{boost} \times \text{part_rules}[\text{part_no}] \times \text{level_rules}[\text{level}]$
- Default level multipliers: part=1.0, article=0.98, clause=0.95, subclause=0.90.
- boost comes from the Document (default 1.0).
- part_rules can be injected to boost/suppress specific parts (e.g., Part 3 on Fundamental Rights).

Important functions:

- _get_tf_vector(doc_id) — builds/caches sparse TF vectors for cosine similarity. Cache lasts the lifetime of the Reranker instance.
- _cosine_similarity(vec_a, vec_b) — dot product over intersection, normalized by L2 norms.
- _ranked(signal_key) — helper that produces [(rank, doc_id), ...] sorted descending by a signal.

Design decisions:

- rrf_k=60 is higher than the typical default of 60. This means the fusion is less sensitive to individual ranks — consistent with having only 3 signals (vs. many systems that fuse dozens of rankers).
- MMR with BM25 cosine similarity rather than embedding similarity. This is a deliberate choice to avoid a dependency on embedding models. BM25 TF vectors provide a reasonable measure of topical overlap.
- Vector cache (_vector_cache) is unbounded — a long-running server processes queries across different users/documents, and the cache accumulates all doc_id TF vectors ever requested. For 700 documents this is negligible, but if the corpus grows it could become a memory concern.
- Static _apply_boost method — can be called without a Reranker instance if needed (though currently it isn't).

Common pitfalls:

- The _vector_cache is never cleared. For a static corpus of 700 docs this is fine, but if documents are added at runtime (which they aren't), stale vectors could accumulate.
- _mmr_diversify re-computes TF vectors for all candidates on every iteration of the while loop. The helper variables vec_candidates and vec_selected are re-computed each iteration because the candidate list

shrinks.

- If `rff_score` key is present (set by RRF) it's used for MMR; otherwise falls back to `score`. This is a safety fallback for when `rerank()` is called with results that already have `rff_score`.

File 9: backend/src/llm/rag_repository.py (337 lines)

Purpose: Data-access layer for the LLM module. Owns the Ollama client, connectivity checks, retry logic, article promotion, and context truncation.

Dependencies: `ollama.Client`, `RetrievalWorkflow`, `re`, `time`, `logging`.

Public API:

- `retrieve(query, top_k, boost_rules) -> list[dict]` — delegates to `RetrievalWorkflow`
- `promote_to_articles(results) -> list[dict]` — clause→article promotion
- `check_ollama_connection() -> (bool, str)` — cached connectivity
- `check_model_availability(model_name) -> (bool, str, list)` — cached model check
- `call_llm(messages, stream) -> response` — LLM call with 3-attempt retry
- `build_truncated_text(article) -> str` — context for LLM (matched clauses only)

Internal structure:

```
__init__():
- store retrieval_workflow, model, Ollama Client (with optional API key header)
- _ollama_available = None (lazy, first request triggers check)
- _available_models = []
- _build_article_lookup() ← called in constructor, builds lookup from documents
```

Main algorithms:

Article Promotion (`promote_to_articles`):

1. Iterate results, collect `matched_clauses` per `article_no` (set of `clause_no` values)
2. Deduplicate by `article_no` (first occurrence wins — preserves original ranking order)
3. For each unique article:
 - If an article-level Document exists → use its text directly
 - If only clause-level docs exist → concatenate with `\n---\n` separator
 - Build Citation string (e.g., "Part 3, Article 31")

Article Lookup (`_build_article_lookup`): Groups all documents by `article_no`. Two cases:

- Articles with their own Document (`level="article"`) — typically articles with lettered sub-clauses where the full text is already aggregated.
- Articles stored as individual clause Documents — concatenate all clause texts together.

Context Truncation (`build_truncated_text`): If matched clauses are tracked, returns only the header + matched clause texts rather than the full article. This saves LLM context window for the most relevant portions.

Ollama Retry (`call_llm`): 3 attempts with 0.5s delay. Creates `keep_alive=30m` and `num_ctx=4096`. All exceptions are caught and retried.

Important functions:

- `_build_article_lookup()` — builds `_article_lookup` (`article_no` → full article) and `_clause_structure` (`article_no` → clause map). This is called once in the constructor and never rebuilt.
- `promote_to_articles()` — the main article promotion entry point. Called on every query.
- `call_llm()` — Ollama call with retry. Raises the last exception if all 3 attempts fail.
- `_extract_model_names()` — handles two response shapes from the Ollama API (dict vs object), making the code resilient to API version changes.
- `_ensure_ollama_checked()` — lazy initialization with caching. Only checks once per process lifetime (or re-checks if the first check failed).

Design decisions:

- Article lookup is built in the constructor from RetrievalWorkflow.engine.documents. This ties RAGRepository to having access to the full document list. The _build_article_lookup method has a guard that checks if self.retrieval.engine.documents exists.
- Ollama connectivity is cached per process lifetime. The first query triggers the check; subsequent queries use the cached result. This avoids a /api/tags call on every request but means that if Ollama restarts, the server won't detect it until the process restarts.
- Context truncation only works for articles with tracked clause structure. For single-block articles or articles where no clause was matched, the full text is returned.
- promote_to_articles handles the "no article lookup" case gracefully — if _article_lookup is empty, it returns the raw results unchanged.

Common pitfalls:

- The _article_lookup is built once. If new documents are added to the corpus at runtime (they aren't), the lookup would be stale.
- _build_promoted_item receives a result dict and optionally enriches it with article data from the lookup. If article is None, the result's fields are used as-is — this means partial data can flow through.
- The _clean_body function strips enriched-text headers using _ENRICHED_RE — this regex must match the output format of flatten_constitution.py. If the format changes, the body cleaning breaks silently.

File 10: backend/src/llm/rag_workflow.py (216 lines)

Purpose: Thin orchestrator that composes RAGRepository + RAGFormatter into the public Q&A entry points. Handles the LLM decision matrix and response assembly.

Dependencies: RAGRepository, RAGFormatter.

Public API:

- ask(query, use_llm=False) -> dict — synchronous Q&A
- ask_streaming(query, use_llm=True) -> generator — streaming Q&A (SSE event dicts)
- retrieve(query, top_k=None) -> list[dict] — convenience wrapper

Internal structure:

```
ask(query, use_llm):
    1. _prepare_articles(query) → retrieve + promote
    2. If !use_llm → return {query, articles}
    3. check_ollama_connection() → if not connected → return {..., ollama_status: connected=false}
    4. check_model_availability() → if not available → return {..., model_available=false}
    5. format_context(promoted_articles)
    6. build_system_prompt() + build_user_prompt(query, context)
    7. call_llm(messages, stream=False)
    8. return {query, articles, response, citations, ollama_status}
```

Main algorithms:

Decision matrix:

Condition	Response includes
`use_llm=false`	query + articles
LLM disconnected	query + articles + ollama_status.connected=false
Model missing	query + articles + ollama_status.model_available=false
LLM succeeds	query + articles + response + citations + ollama_status
LLM fails after retries	query + articles + response (error text) + error field

Streaming variant: Same logic but yields SSE event dicts:

- {"type": "articles", "articles": [...]} — immediately
- {"type": "token", "content": "partial"} — per LLM chunk
- {"type": "done"} — completion
- {"type": "error", "content": "..."} — on failure

Important functions:

- `_prepare_articles(query)` — retrieve + promote, also sets `full_text = text` for persistence
- `_build_article_dict(article)` — filters to only the fields defined in `_ARTICLE_FIELDS`

Design decisions:

- `_ARTICLE_FIELDS` constant at module level defines the exact response contract. Adding or removing fields here changes the API response shape.
- Articles are retrieved with `max_context_articles = 8`, then all 8 are sent to the LLM in context. The `DEFAULT_MAX_CONTEXT` is 8 — matching the `DEFAULT_TOP_K` from retrieval.
- Standalone `main()` demo function runs 3 preset questions. Useful for testing without the Flask server.

Common pitfalls:

- `_prepare_articles` sets `art["full_text"] = art["text"]` — the same string assigned to two keys. The text field is later truncated by `build_truncated_text()` in the formatter, but `full_text` retains the original. This is intentional for persistence but can confuse readers.
 - The standalone `main()` has hardcoded paths (`data/output/...`) — only works when run from the backend directory.
-

File 11: backend/src/llm/rag_formatter.py (55 lines)

Purpose: Build LLM prompts — context formatting, system instructions, user query wrapper.

Dependencies: None — pure string builder.

Public API:

- `format_context(articles) -> str` — concatenates article citations and texts
- `build_system_prompt() -> str` — system-level instructions for the LLM
- `build_user_prompt(query, context) -> str` — user message with context and question

Internal structure:

- `format_context`: Joins `{citation}: {title}\n{text}` for each article, with relevance score appended.
- `build_system_prompt`: A multi-line instruction block that defines 6 answer-style adaptation rules (What→explain, Who→identify, When→date, etc.), plus grounding constraints ("Answer ONLY using the Context").
- `build_user_prompt`: Template with `{context}` and `{query}` placeholders.

Design decisions:

- All methods are static — no state, no instance needed.
- The system prompt is the most critical piece: it enforces strict grounding (no hallucination) and style adaptation by question type. This is where the "Explainable" in the project name is implemented.
- Relevance scores are included in the context — the LLM can see which articles are most relevant.

Common pitfalls:

- The score in `format_context` comes from `article["score"]` — this is the final score after reranking, promotion, and boost, not any raw score.
 - If `use_llm=false`, the formatter is never called. It's only invoked for LLM-involved answers.
-

File 12: backend/app.py (49 lines)

Purpose: Flask application factory and server entry point.

Dependencies: Flask, flask-cors, python-dotenv, Database, log_config, init_workflow, blueprints, get_spacy_pipeline.

Public API: create_app() -> Flask, main().

Internal structure:

```
main():
    load_dotenv()
    setup_logging()
    init_workflow()          ← builds SearchEngine + Reranker + RAGWorkflow
    get_spacy_pipeline()     ← loads spaCy model (singleton)
    app = create_app()       ← Flask(), CORS(), Database().connect(), register_blueprints
    app.run()
```

Important functions:

- create_app() — factory. Creates Flask, enables CORS, connects MongoDB, registers blueprints.
- main() — entry point. Loads env, sets up logging, initializes workflow, starts spaCy, creates app, runs.

Design decisions:

- Eager initialization: init_workflow() and get_spacy_pipeline() run before the server starts. This adds ~1s startup latency but ensures zero slow-first-request penalty.
- create_app() is separate from main() — the factory pattern allows testing with different configurations and importing the app in WSGI servers.
- CORS(app) with no restrictions — no origin checking, no allowed methods. This is a production readiness gap.
- threaded=True — Flask's built-in threading for concurrent request handling. No async workers.

Common pitfalls:

- No environment validation — if JWT_SECRET is missing, it only fails when the first authenticated request arrives, not at startup.
- MongoDB connection is established in create_app() but the database singleton is loaded from config/db_connect.py — if MongoDB is unreachable, the app fails to start (by design).

File 13: backend/controllers/decorators.py (69 lines)

Purpose: JWT authentication decorator. Extracts token, validates signature, checks token_version.

Dependencies: jwt (PyJWT), flask.request, mongoengine.connection, User model.

Public API:

- token_required(f) — decorator for protected routes

Internal structure:

```
token_required:
    1. Extract token from:
        a. Authorization: Bearer <token> header (primary)
        b. token cookie (fallback)
    2. If no token → 401
    3. If JWT_SECRET not configured → 500
    4. jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
    5. _get_user(user_id) → lookup user
        a. If user exists and payload.token_version < user.token_version → 401 (invalidated)
        b. If user is None (DB unavailable) → skip version check (pass)
    6. Attach payload to request.user
    7. Handle ExpiredSignatureError → 401
    8. Handle InvalidTokenError → 401
```

Important functions:

- `_get_user(user_id)` — tries to access the database; if `ConnectionFailure` is raised, returns `None` (graceful degradation — authentication still works, just skips `token_version` check).

Design decisions:

- Token version invalidation: On logout, `user.token_version` is incremented. All existing JWTs with the old `token_version` become invalid immediately. This is replay-attack protection.
- Cookie fallback: If the `Authorization` header is missing, checks for a token cookie. This allows both SPA (header-based) and server-rendered (cookie-based) clients.
- DB-unavailable fallback: If the database is unreachable, the decorator skips the `token_version` check but still validates the JWT signature. This means logged-in users can continue using the system even if MongoDB is temporarily down (but new logins would fail).
- Double-fault handling: `_get_user` wraps the lookup in a bare `except Exception`, so any DB error becomes a `None` return rather than a 500.

Common pitfalls:

- The `Bearer` prefix check is case-sensitive. `Authorization: bearer <token>` (lowercase 'b') would fail the `startswith('Bearer ')` check and fall through to the cookie check.
- If both the header and cookie are absent, the decorator returns 401 immediately — no fallback to any other auth method.
- The cookie `SameSite=Strict` means the cookie is not sent on cross-origin requests, which is correct for APIs but can cause issues in embedded contexts.

File 14: backend/controllers/api_controller.py (182 lines)

Purpose: HTTP handlers for all Q&A and message management endpoints. Input validation, response assembly, persistence.

Dependencies: `MessageService`, `QAService`, `flask`.

Public API (handlers):

- `home()` — GET `/api/v1/` — endpoint list
- `health()` — GET `/api/v1/health` — liveness
- `ask()` — POST `/api/v1/ask` — synchronous Q&A
- `ask_stream()` — POST `/api/v1/ask-stream` — streaming Q&A via SSE
- `list_messages()` — GET `/api/v1/messages` — paginated history
- `get_message(message_id)` — GET `/api/v1/messages/<id>` — single message
- `delete_message(message_id)` — DELETE `/api/v1/messages/<id>`
- `delete_all_messages()` — DELETE `/api/v1/messages`

Internal structure:

```
ask():
    _parse_ask_request() → validate (JSON, query, length)
    → QAService.answer_query(query, use_llm)
    → if status==200: QAService.persist_message(user_id, query, payload)
    → return jsonify(payload), status_code

ask_stream():
    _parse_ask_request()
    → QAService.answer_query_streaming(query, use_llm)
    → Response(stream_with_context(_stream_events(...)), mimetype=text/event-stream)
```

Important functions:

- `_parse_ask_request()` — shared validation for both `/ask` and `/ask-stream`. Returns (data, error) tuple.
- `_stream_events()` — generator that wraps the `QAService` streaming generator, accumulates token events

for persistence, and calls `persist_message` on the "done" event.

Design decisions:

- `_persist_message()` is fire-and-forget — failures are logged but never break the response. This ensures the user always gets their answer even if MongoDB is slow or unavailable.
- The same `_parse_ask_request` function is used for both endpoints, ensuring consistent validation.
- `ask_stream` defaults `use_llm=True` (different from `ask` which defaults to `False`). This makes sense: streaming is primarily for LLM answers; non-LLM retrieval is fast enough to be synchronous.
- Ownership checks in `get_message` and `delete_message` compare `result["data"]["user"]["id"]` to `request.user.get("user_id")` — string comparison.

Common pitfalls:

- `_stream_events` accumulates the full answer in memory before persisting. For very long answers with many tokens, this is fine; for indefinite streaming, it could be a memory concern.
- The "X-Accel-Buffering": "no" header is nginx-specific. Behind other proxies, SSE may be buffered and delayed.
- Error handling wraps the entire handler in a broad `except Exception` that returns 500 — no distinction between validation errors (4xx) and server errors (5xx).

File 15: backend/services/qa_service.py (92 lines)

Purpose: Application service — owns pipeline initialization and persistence. Bridge between Flask controllers and domain layer.

Dependencies: `EngineFactory`, `Reranker`, `RetrievalWorkflow`, `RAGRepository`, `RAGWorkflow`, `ArticleService`, `MessageService`.

Public API:

- `init_workflow()` — global pipeline initialization (called once at startup)
- `QAService.answer_query(query, use_llm) -> (dict, int)` — Q&A response
- `QAService.answer_query_streaming(query, use_llm) -> generator`
- `QAService.persist_message(user_id, query, payload) -> None`

Internal structure:

```
init_workflow():
    engine = EngineFactory.from_artifacts(docs, index, synonyms)
    reranker = Reranker(engine.bm25_scorer.tf_index)
    retrieval_workflow = RetrievalWorkflow(engine, reranker)
    repository = RAGRepository(retrieval_workflow)
    _workflow = RAGWorkflow(repository, RAGFormatter())

persist_message():
    for each article → ArticleService.create_article() → upsert by doc_id
    collect article ObjectIds
    → MessageService.create_message() → save query + answer + article refs
```

Design decisions:

- Module-global `_workflow` singleton: Initialized once in `init_workflow()`, accessed via `_get_workflow()`. No locks, no lazy init — if `init_workflow()` hasn't been called, `_workflow` is `None` and the server crashes on first request (by design).
- Eager initialization means predictable response times — no slow cold start.
- `publish_message` is called from two places (in `ask` after response, in `_stream_events` after completion for streaming). Both fire-and-forget.

Common pitfalls:

- `ArticleService.create_article()` is called with `article.get("content")` but the `content` key may not exist in the

article dict (it's set by `_build_promoted_item` but it depends on `_clean_body`). This is why there are fallback patterns throughout.

- The `_DEFAULT_*` paths are relative to the backend directory — they only work when the server is started from `backend/`.
-

File 16: `backend/services/article_service.py` (134 lines)

Purpose: CRUD for ReferencedArticle MongoDB collection. Implements upsert-by-doc_id pattern.

Dependencies: ReferencedArticle model, mongoengine.

Public API:

- `create_article(...)` -> dict — create or update (upsert by doc_id)
- `get_article(article_id)` -> dict
- `list_articles()` -> dict
- `delete_article(article_id)` -> dict

Main algorithms:

- Upsert: Query by doc_id. If exists → update fields. If not → create new.
- Updated fields: `content`, `text`, `full_text`, `matched_terms`, `exact_matched_terms`, `relevance_score`, `bm25_score`, `proximity_score`, `title_match_count`.

Design decisions:

- Upsert-by-doc_id ensures deduplication: if the same constitution article appears in multiple queries, the existing document is updated rather than duplicated.
- All methods return a standard dict with success, error, and data keys — consistent with the rest of the service layer.
- `matched_terms` defaults to [] if None — prevents storing null values in MongoDB.

Common pitfalls:

- The constructor signature has 16 parameters, most of which are optional. Callers must match the parameter names carefully.
 - `ValidationError` is caught generically — no detail about which field failed validation.
 - Uses synchronous mongoengine calls within async def method signatures (see technical debt).
-

File 17: `backend/services/message_service.py` (195 lines)

Purpose: CRUD for Q&A messages with pagination, ownership, and search.

Dependencies: User, ReferencedArticle, Message models, mongoengine.

Public API:

- `create_message(user_id, query, answer, articles=[])` -> dict
- `get_user_messages(user_id, limit=20, skip=0)` -> dict (paginated)
- `get_message(message_id)` -> dict
- `update_message_answer(message_id, new_answer)` -> dict
- `search_messages(user_id, search_term)` -> dict
- `delete_message(message_id)` -> dict
- `delete_user_messages(user_id)` -> dict

Internal structure:

- `create_message`: Lookup user → lookup each article ref by id → create Message with article references → save.
- `get_user_messages`: Query `Message.objects(user=user).order_by('-created_at').skip(skip).limit(limit)` + count for pagination metadata.

Design decisions:

- Article references are stored as a list of ObjectId references. The `to_json()` method populates these on read.
- `search_messages` uses `query__icontains` which is a case-insensitive contains search in mongoengine. Note: the search term is regex-escaped, so it matches literally.
- Pagination returns `has_more: skip + limit < total_count` for the frontend to know if there are more pages.

Common pitfalls:

- `create_message` imports the article reference, but if an article id is invalid, it logs a warning and skips it — the message is still created without that article reference.
 - Uses `async def` with synchronous mongoengine calls. Misleading but functionally correct.
 - The `re.escape` in `search_messages` means the user can't use regex patterns in search (which is correct — search should be literal).
-

File 18: frontend/src/App.jsx (58 lines)

Purpose: Frontend root — routing and auth provider setup.

Dependencies: `react-router-dom`, `AuthProvider`, `Navbar`, `ProtectedRoute`, all page components.

Public API: The `App` component (exported default).

Internal structure:

```
BrowserRouter
  AuthProvider
    Navbar
    div#main-content
      Routes
        /login → LoginPage
        /register → RegisterPage
        / → ProtectedRoute → HomePage
        /history → ProtectedRoute → HistoryPage
        /history/:id → ProtectedRoute → MessageDetailPage
        /about → AboutPage
        /how-it-works → HowItWorksPage
        * → NotFoundPage
```

Design decisions:

- `AuthProvider` wraps all routes including `Navbar` — auth state is available everywhere.
- `ProtectedRoute` wraps the authenticated pages. Unauthenticated users are redirected to `/login`.
- `/about` and `/how-it-works` are public (no auth required).
- `Navbar` is rendered outside the route switch — it's always visible.

Common pitfalls:

- The `#main-content` div is used for layout but has no explicit styling — layout depends on parent containers.
 - Route changes are handled by `react-router-dom v7` — no page reloads.
-

File 19: frontend/src/hooks/useAskStream.js

Purpose: Custom hook for consuming the SSE streaming endpoint.

Dependencies: `ApiClient` from `api/client.js`.

Public API: `useAskStream()` returns `{ articles, response, loading, error, startStream, cancel }`

Internal structure:

```

startStream(query, useLlm):
  POST /api/v1/ask-stream via fetch (with Authorization header)
  → ReadableStream.getReader() → while loop reading chunks
  → parse "data: {...}\n\n" SSE lines
  → dispatch by event type: articles → setArticles, token → appendResponse, done → setLoading(false)
  → support AbortController for cancellation

```

Design decisions:

- Uses the Fetch API's ReadableStream directly rather than an EventSource wrapper. This gives more control over the SSE parsing and allows custom HTTP headers (Authorization).
- AbortController allows the user to cancel an in-flight stream — the Cancel button in MainSearchBar triggers this.
- State is managed with useState inside the hook — the consuming component re-renders on each state update.

Common pitfalls:

- The hook assumes data: {...}\n\n SSE format (single-line events). Multi-line SSE data would break the line-by-line parsing.
- If the connection drops mid-stream, the error state is set and loading becomes false — the user sees partial results.

File 20: backend/controllers/auth_controller.py (108 lines)

Purpose: User registration, login, logout, and profile endpoints.

Dependencies: UserService, User model, flask.

Public API (handlers):

- register() — POST /api/v1/auth/register
- login() — POST /api/v1/auth/login
- logout() — POST /api/v1/auth/logout
- get_current_user() — GET /api/v1/auth/me

Main algorithms:

Login:

```

UserService.authenticate_user(email, password)
  → User.objects.get(email=email)
  → check_password(password)
  → jwt.encode({user_id, email, token_version}, JWT_SECRET, algorithm='HS256')
  → set httpOnly cookie + return JSON with token

```

Logout:

```

user.token_version += 1
user.save()
→ All existing JWTs become invalid
→ Clear cookie

```

Design decisions:

- Token version invalidation: logout increments User.token_version. Any JWT with a lower token_version is rejected by the @token_required decorator. This ensures logout is irreversible even if the JWT hasn't expired.
- Dual response: Return token in JSON body AND set it as httpOnly cookie. The frontend uses the body token; the decorator checks both.
- SameSite=Strict on the cookie — prevents CSRF attacks but means the cookie is not sent on cross-origin

navigation.

- `register()` returns 201 on success, 400 on validation errors.

Common pitfalls:

- `login()` calls `current_app.config.get('PRODUCTION', False)` — production flag must be set properly for secure cookies.
- Error handling: if `User.objects.get(id=user_id)` fails in `logout()`, the exception is caught and a 500 is returned without invalidating the token.
- `get_current_user()` uses `UserService.get_user(user_id)` which returns a dict with `success`, `data`, `message` keys — the controller returns this directly, so the response shape is `{success: true, data: {...}, message: "..."}` rather than a flat user object.

Function-by-Function: backend/src/core/search_engine.py

This is the central retrieval pipeline — the single most important file in the IR engine. Every Q&A request flows through `SearchEngine.search()`.

SearchEngine.__init__(...)

Example:

```
# title_tokens for a document
doc.doc_id = "3_31"
doc.title = "Right relating to education"
# After __init__:
self.title_tokens["3_31"] = ["right", "relate", "education"]
# (lemmatized, stopwords removed)
```

SearchEngine.search(query, top_k, proximity_weight, title_boost)

Example:

```
engine.search("right to education", top_k=5)
# Returns top 5 scored dicts, e.g.:
# [
#   {"doc_id": "3_31", "title": "Right relating to education", "score": 45.2, ...},
#   {"doc_id": "3_30", "title": "Right regarding freedom", "score": 32.1, ...},
# ]
```

SearchEngine._prepare_proximity_query(query)

SearchEngine._generate_candidates(bm25_tokens)

Example:

```
# For query "fundamental right":
# tf_index["fundamental"] = {"1_1": 2, "3_31": 1, ...}
# tf_index["right"] = {"3_31": 1, "3_30": 2, ...}
# → candidates = {"1_1", "3_31", "3_30", ...}
```

SearchEngine._score_document(doc, bm25_tokens, original_tokens, query_pairs, title_boost, proximity_weight)

Example:

```
# For doc "3_31" (Right relating to education), query "right to education":
# - BM25 score ≈ 12.5 (has "right" and "education")
# - title_match_count = 2 ("right", "education" both in title)
# - title_boost = 5.0
# - proximity score = 0.85 ("right" ... "education" close in text)
# - final = 12.5 + 2*5.0 + 1.0*0.85 = 23.35
```

SearchEngine._format_results(scored_docs)

Reranker.rerank(results, top_k, boost_rules)

Reranker._rfr_fuse(results)

Example:

```
# 3 results with their signals:
# Doc A: BM25=10, Prox=0.2, Title=2
# Doc B: BM25=8, Prox=0.8, Title=0
# Doc C: BM25=6, Prox=0.5, Title=1
#
# Ranks by BM25: A(1), B(2), C(3)
# Ranks by Prox: B(1), C(2), A(3)
# Ranks by Title: A(1), C(2), B(3)
#
# k=60:
# A: 1/61 + 1/63 + 1/61 = 0.0492
# B: 1/62 + 1/61 + 1/63 = 0.0488
# C: 1/63 + 1/62 + 1/62 = 0.0480
```

Reranker._mmr_diversify(results)

Example:

```
# After MMR, instead of [A, B, C] (all about fundamental rights),
# might get [A, D, B] where D is about a different topic
# but still relevant:
# MMR(A) = 0.5*0.0492 - 0.5*0 = 0.0246 (first, always picked)
# MMR(B) = 0.5*0.0488 - 0.5*0.3 = 0.0094 (similar to A)
# MMR(D) = 0.5*0.0470 - 0.5*0.05 = 0.0210 (less similar, gets picked second)
```

Reranker._apply_boost(results, boost_rules)

Example:

```
# Default level rules: article=0.98, clause=0.95, subclause=0.90
# Document: boost=1.0, part_no="3", level="article"
# No part rules override
# multiplier = 1.0 * 1.0 * 0.98 = 0.98
# If part_boost = {"3": 1.2} (boost Fundamental Rights):
# multiplier = 1.0 * 1.2 * 0.98 = 1.176
```

Reranker._get_tf_vector(doc_id)

Reranker._cosine_similarity(vec_a, vec_b)

Example:

```
# Doc A: {"right": 2, "education": 1, "fundamental": 1}
# Doc B: {"right": 1, "freedom": 2, "speech": 1}
# intersection = {"right"}
# dot = 2*1 = 2
# norm(A) = sqrt(4+1+1) = sqrt(6) ≈ 2.45
# norm(B) = sqrt(1+4+1) = sqrt(6) ≈ 2.45
# cos_sim = 2 / (2.45*2.45) = 2/6 ≈ 0.333
```

Code Review — Staff Engineer Review

Reviewing the three most important modules: `search_engine.py`, `reranker.py`, and `rag_repository.py`.

search_engine.py — Review

Why it is written this way

The `SearchEngine` follows a pipeline pattern — each step (`tokenize` → `expand` → `generate candidates` → `score` → `format`) is a separate method. The constructor pre-computes what can be pre-computed (title tokens) and accepts all dependencies via injection.

Alternative considered: Making each signal scorer a mixin that `SearchEngine` inherits from. Rejected because it would couple the scoring algorithms to the orchestration — current design keeps each scorer independently testable.

Tradeoffs

Decision	Pro	Con
Pre-computed title tokens	Fast per-query title boost (O(1) lookup)	~700 docs × 5 tokens = negligible memory, but doesn't adapt to runtime changes (not needed)
Candidate set = union of all term postings	High recall — no document is missed if it has any query term	Large candidate sets for common terms (e.g., "right" → hundreds of docs scored)
Single <code>`_score_document`</code> returns 7-element tuple	One pass over candidates, all signals computed	Unreadable magic-index destructuring in <code>`_format_results`</code> (tuple[5] for Document, tuple[6] for matched_terms, etc.)
<code>`query`</code> → two separate tokenizations (BM25 + proximity)	Optimal configuration for each signal	Double-tokenization could be cached if performance-critical

Hidden assumptions

- Title tokens are pre-computed with the BM25 processor (lemmatized, stopwords removed). If a query term is a stopword (e.g., "right to education"), it will never match in the title because "to" is removed from `bm25_tokens` and wasn't in `title_tokens`. But since stopwords are removed from BM25 tokens, they're also removed from title tokens — consistency is maintained.
- `bm25_processor.process_text()` and `proximity_processor.process_text()` both use the same spaCy pipeline under the hood. If spaCy behaves differently between calls (e.g., model reload), the two tokenizations could diverge. In practice this doesn't happen because the model is loaded once at startup.

Clever ideas

- Dual `matched_terms` tracking: `matched_terms` (lemmatized, from BM25 tokens) for scoring analysis and `exact_matched_terms` (original query tokens) for frontend highlighting. The frontend can highlight exactly what the user typed, while the backend uses standardized lemmas.
- Early exit on zero BM25: If BM25 returns 0.0, the document is skipped entirely — no proximity scoring or title boost is computed. This is a significant optimization for long-tail candidates that have no real BM25 match.

Possible bugs

- `_generate_candidates()` collects `doc_ids` from `tf_index[token].keys()` for each token. If `tf_index` has a token with no postings (empty dict), `.keys()` returns an empty view — no error, but no candidates added. This shouldn't happen with a valid index.

- `_score_document` line 168: `set(bm25_tokens) & set(self.title_tokens[doc.doc_id])`. If `doc.doc_id` is not in `self.title_tokens` (which would happen only if documents were added after construction), this raises `KeyError`. The constructor populates `title_tokens` from `self.documents`, so this is safe as long as the document list and `SearchEngine` are consistent.

Performance implications

- Scoring loop: For `recall_k=50` candidates, the loop runs 50 iterations. Each iteration does BM25 scoring ($O(q)$ for q query terms), title boost (set intersection), and proximity scoring ($O(p)$ for p pairs). For a 5-token query with synonym expansion to ~ 15 tokens, this is about 100-200 operations per candidate. Total per query: $\sim 10,000$ operations. Well within acceptable range.
- Candidate generation: The union set operation runs in $O(k)$ where k is total postings across all tokens. For common terms like "right" and "constitution", this could be 200-300 `doc_ids`. The union set handles deduplication efficiently.

reranker.py — Review

Why it is written this way

The `Reranker` implements three independent stages (`RRF` $\rightarrow$ `MMR` $\rightarrow$ `boost`) as private methods called sequentially from the public `rerank()`. Each stage is composable — they could be rearranged, skipped, or replaced without changing the others.

Alternative considered: Using a single formula that combines all three stages into one score. Rejected: three-stage pipeline allows each stage to be tuned independently and makes the mathematical contribution of each stage explicit.

Tradeoffs

Decision	Pro	Con
<code>`rrf_k=60`</code>	Less sensitive to single-rank noise (common in 3-signal fusion)	Higher k dilutes the contribution of strong signals
<code>`mmr_lambda=0.5`</code>	Equal weight for relevance and diversity	May over-prioritize diversity over relevance for precision-focused queries
BM25 cosine similarity (no embeddings)	Zero ML dependencies, no model download, deterministic	Cannot capture semantic similarity beyond term overlap
Unbounded <code>`_vector_cache`</code>	Fast subsequent queries for the same <code>doc_ids</code>	Memory grows with distinct <code>doc_ids</code> seen (for 700 docs, $\sim 700 \times \text{avg } 20 \text{ terms} = \text{negligible}$)

Hidden assumptions

- `MMR` assumes that `rrf_score` (or `score`) is a reasonable measure of query-document relevance. If the original scores from `SearchEngine` are poorly calibrated (e.g., all scores are within a narrow range), `MMR` will primarily optimize for diversity and may return irrelevant results.
- The `_vector_cache` assumes the TF index never changes. If the index is rebuilt between queries (it isn't), cached vectors would be stale.

Clever ideas

- Static `_apply_boost` method — it doesn't use `self`, so it could be tested independently or reused in other contexts.
- Fallback `results.get('rrf_score', results.get('score', 0.0))` in `MMR` — if for some reason `RRF` wasn't called first, `MMR` still works using the raw score from `SearchEngine`.
- The `_ranked` helper inside `_rrf_fuse` is a closure that captures `signal_key` — avoids three copies of the same sorting logic.

Possible bugs

- `_mmr_diversify` line 103-108: `vec_candidates` and `vec_selected` are recomputed on every iteration of the while loop. This is correct because candidates shrinks each iteration, but `vec_selected` grows. The full recomputation is $O(r^2 \cdot t)$ which is the inherent cost of MMR.
- `_apply_boost` line 156: `result["score"] = score * multiplier`. This overwrites the previous score. If `rerank()` is called twice on the same results, scores are multiplied twice. In practice, `rerank()` is called once per query.
- `_cosine_similarity` line 41: `intersection = set(vec_a.keys()) & set(vec_b.keys())`. Creating two sets from dict keys and intersecting them is $O(a + b)$. For small TF vectors this is fine, but for documents with hundreds of unique terms it's wasteful — a for `k, v in vec_a.items()`: if `k in vec_b` would be $O(\min(a,b))$ with short-circuit.

Performance implications

- MMR is $O(r^2 \cdot t)$: For `r=50` results with `t=20` avg unique terms, that's $50^2 \times 20 = 50,000$ operations. Acceptable for a single query, but at high concurrency this could become a bottleneck.
- Vector cache miss: The first time a `doc_id` is seen, `_get_tf_vector` traverses the entire `tf_index` (all $\sim 3,000$ unique terms $\times$ their postings lists). This is expensive but happens only once per `doc_id` per process lifetime.

rag_repository.py — Review

Why it is written this way

The RAGRepository acts as the bridge between the pure IR engine (RetrievalWorkflow) and the external LLM (Ollama). It owns all LLM-specific concerns: client creation, connectivity caching, retry logic, and prompt-level data transformations (article promotion, context truncation).

Alternative considered: Splitting into LLMClient (Ollama management) and ArticleService (promotion). The current design keeps everything in one class but the responsibilities are clearly separated by method groups (article promotion, retrieval, LLM, context truncation).

Tradeoffs

Decision	Pro	Con
Constructor builds article lookup	Single initialization, fast promotion	Cannot handle runtime corpus changes
Lazy Ollama connectivity check	Keeps startup fast	First request pays ~ 100 ms latency
Cached <code>`_ollama_available`</code> (never re-checks)	Avoids <code>`/api/tags`</code> call on every request	Won't detect Ollama restart until process restart
3 retries with 0.5s delay	Resilient to transient Ollama failures	1.5s max additional latency before failure
<code>`promote_to_articles`</code> deduplicates by <code>article_no</code> (first wins)	Simple, deterministic	Highest-scoring clause determines article score — lower-scoring clauses of the same article are discarded

Hidden assumptions

- `_build_article_lookup()` assumes that documents with `level="article"` have the full article text. Some articles may be stored as a single `level="article"` doc with all sub-clauses included in the text, while others are stored as individual `level="clause"` docs. The code handles both cases, but the distinction depends on the flattening script's output format.
- The article lookup key is `doc.article_no` as a string. If article numbers are not unique across parts (e.g., Part

1 Article 1 and Part 2 Article 1), the lookup would collide. In the Constitution of Nepal, article numbers are unique across the document, so this is safe.

Clever ideas

- `_extract_model_names()` handles two response shapes — Ollama's Python SDK may return a dict or an object depending on the version. The method handles both with `isinstance` checks, making the code resilient to SDK version changes.
- `build_truncated_text()` — reduces LLM context by returning only matched clauses. For an article with 10 clauses where only 2 match, the context is 80% smaller. This is important because the LLM context window is 4096 tokens.
- The `_ENRICHED_RE` regex strips the "Part X Article Y" header from enriched text, allowing the formatter to present clean clause text to the LLM.

Possible bugs

- `promote_to_articles` line 196-208: iterates results in their existing order (sorted by descending score). The first occurrence of each `article_no` wins. But if the same article appears at rank 1 and rank 15 with different clause matches, the rank-1 version is kept but its `matched_clauses` may be a subset. This means the context truncation might miss some matching clauses that appeared in lower-ranked results.
- `_build_article_lookup` line 80-85: `defaultdict(lambda: {...})` creates default group dicts with `"article_doc": None`, `"clause_docs": []`. The list is shared across all groups created by the same lambda? No — `defaultdict` calls the lambda each time, so each group gets its own list. This is correct.
- `_clean_body` uses `re.sub(f'^{re.escape(title)}\n*', ...)`. If the title contains regex-special characters (e.g., parentheses), `re.escape` handles them. But the title is compared case-insensitively, while the regex doesn't specify `re.IGNORECASE`. Wait, `_ENRICHED_RE` uses `re.IGNORECASE` for the header, but the title-subtraction regex does not (`re.sub` doesn't use flags). If the title in the text differs in case from the Document's title field, the title won't be stripped. This is likely benign (titles are consistent) but is a subtle inconsistency.

Performance implications

- `_build_article_lookup()` traverses all ~700 documents, grouping by `article_no` and concatenating clause texts. This happens once at startup and is fast (~10ms).
 - `promote_to_articles()` iterates the promoted results (max 8 after `top_k`). This is very fast.
 - `build_truncated_text()` does a dict lookup + sorted iteration over matched clauses. Negligible cost.
 - `call_llm()` network I/O dominates — retry delay is 0.5s, but each Ollama call can take 2-15s depending on model and query complexity.
-

Common Patterns Across All Files

Strong engineering demonstrated:

1. Dependency injection throughout the IR engine — scorers, processors, and expanders are injected rather than created inside the search engine. This makes unit testing possible without loading JSON files.
2. Composability — each pipeline stage (`SearchEngine` → `Reranker` → `RetrievalWorkflow` → `RAGRepository` → `RAGWorkflow`) is a separate class with a small public API. They can be tested in isolation or wired in different configurations.
3. Fallback patterns — spaCy blank model as fallback (lemmatization degrades to identity), DB-unavailable fallback in JWT decorator, Ollama connectivity caching, fire-and-forget persistence.
4. No ML dependencies in the core engine — the entire retrieval and reranking system uses pure Python and standard libraries. No numpy, no sklearn, no embedding models. This makes the system easy to deploy, debug, and audit.

Areas for improvement:

1. Tuple indexing in `_score_document` returns a 7-element tuple that's destructured by position in

`_format_results`. A namedtuple or dataclass would be more readable and maintainable.

2. No type hints in some methods — `reranker.py` uses generic dict consistently, but the exact shapes could be documented.
3. `_get_tf_vector` traverses the entire `tf_index` on cache miss. For a corpus with 3,000+ unique terms, this is $O(t \cdot d)$ where t = unique terms and d = docs per term. Could pre-compute all vectors at construction time.
4. `_mmr_diversify` computes `vec_candidates` and `vec_selected` lists on every while-loop iteration instead of incrementally updating them.
5. `_article_lookup` and `_clause_structure` in `RAGRepository` are built from `retrieval.engine.documents` — this couples `RAGRepository` to the `SearchEngine`'s document list. If the retrieval pipeline changes (e.g., to an external search service), the article lookup would need a new data source.

Architecture Critique — Pre-Release Review

Strengths

1. Clean layered architecture with clear boundaries

The backend follows a strict layers pattern: Routes → Controllers → Services → Domain (IR Engine + RAG Layer) → Data. Each layer has a distinct responsibility and communicates through well-defined interfaces.

- The IR engine (src/core/) has zero Flask dependencies — it's pure Python that could be imported into any application.
- The RAG layer (src/llm/) depends on the IR engine but not on Flask or the service layer.
- Controllers handle HTTP concerns only (validation, response formatting, JWT extraction).
- This layering makes the engine independently testable, deployable, and replaceable.

2. Exceptional error handling and graceful degradation

The system handles failure at every level without crashing:

- MongoDB unreachable at startup → fail-fast (correct for a data-dependent system)
- MongoDB unreachable at runtime → JWT still validates (offline auth mode)
- Ollama unreachable → LLM features disabled, retrieval still works
- LLM call fails after 3 retries → error reported in response body
- spaCy model missing → tokenization works, lemmas are identity
- Persistence failure → logged, doesn't break the response

This is production-quality defensive programming.

3. No ML dependencies in the core retrieval engine

The entire IR system uses pure Python: `math.log` for BM25, `math.sqrt` for cosine similarity, and string operations for proximity scoring. This means:

- No numpy, no sklearn, no embedding models, no GPU requirements
- Launch latency is ~1 second (JSON loads + spaCy init)
- Behavior is fully deterministic and auditable
- No "model drift" — the same query always returns the same results

4. Well-considered default constants

Every tunable parameter has a documented rationale:

- `b=1.0` (full length normalization) — justified by the variance in legal document length
- `rrf_k=60` — appropriate for 3 signals (BM25 + proximity + title)
- `mmr_lambda=0.5` — equal balance of relevance and diversity
- `recall_k=50` / `top_k=8` — appropriate for a ~700-doc corpus
- `max_window=30` — based on legal clause structure

5. Good separation of offline and online concerns

- Ingestion pipeline runs independently of the server
- Indexes are built once, loaded at startup
- No runtime writes to search indexes
- This keeps the online path fast and simple

6. Clean frontend architecture

React SPA with standard patterns:

- AuthProvider context for JWT management

- useAskStream hook for SSE consumption (reusable across components)
 - UI primitives as composable presentational components
 - Pages as containers that wire primitives + data fetching
-

Weaknesses

1. Async def with synchronous I/O (Medium)

message_service.py and article_service.py declare methods as async def but use synchronous mongoengine calls internally. This is misleading:

- Callers may think they can await the methods
- No asyncio event loop is involved — the async def decorator creates a coroutine that runs synchronously
- If Flask ever switches to async mode, these methods would need real async implementations

Suggested fix: Either remove async def and make them synchronous, or implement real async MongoDB access.

2. CORS configuration is wide open (Medium)

CORS(app) in app.py:21 allows all origins, methods, and headers. For a production legal research tool, this should be restricted to known frontend origins.

Suggested fix: CORS(app, origins=["https://your-frontend-domain.com"], supports_credentials=True)

3. No dedicated search-only endpoint (Low)

Retrieval without LLM requires POST /api/v1/ask?use_llm=false. This is non-obvious — a separate GET /api/v1/search endpoint would be cleaner. The current approach also requires a JWT for simple searches, which may be too restrictive for public access.

4. MMR vector cache is unbounded (Low)

Reranker._vector_cache stores TF vectors for every document ever accessed during the process lifetime. For 700 documents this is negligible (~50KB), but if the corpus grows significantly, the cache has no eviction policy.

5. Article lookup couples RAGRepository to SearchEngine (Medium)

RAGRepository._build_article_lookup() accesses self.retrieval.engine.documents to build the article lookup. This means:

- RAGRepository depends on RetrievalWorkflow having an .engine attribute
- The retrieval engine must have a .documents list
- If the retrieval pipeline switches to an external service, article lookup would break

Suggested fix: Inject the document list into RAGRepository directly, or load it from the same source as the engine.

6. Token_version check has a silent skip when DB is down (Low)

The _get_user function in decorators.py returns None if the database is unreachable. The decorator then skips the token_version check — the JWT is validated (signature + expiry) but version check is bypassed. This means a logged-out user's JWT remains valid until expiry if MongoDB is temporarily down. This is a deliberate tradeoff (uptime over invalidation precision) but should be documented.

Technical Debt

Debt	Location	Severity	Effort to Fix
Async def with sync I/O	`message_service.py`, `article_service.py`	Medium	1 hour (remove `async def`)
`responses.txt` preprocessing	in `preprocessing_scripts/`	Low	5 min (delete or .gitignore)

Unused CSS	`frontend/src/App.css` (~150 lines)	Low	10 min (delete)
Unused npm dep (`pptxgenjs`)	`frontend/package.json`	Low	5 min (remove)
UTF-16 `requirements.txt`	`backend/requirements.txt`	Low	5 min (re-save as UTF-8)
No admin blueprint	Admin routes exist in `UserService` but no HTTP endpoints	Medium	2 hours (blueprint + controller)
Hardcoded paths in `qa_service.py`	`_DEFAULT_DOCS_PATH` etc.	Low	Use relative paths from module location

Scalability

Current state

- Single Flask process with `threaded=True`
- All data in memory (~700 Documents, 3 indexes in dicts)
- MongoDB handles persistence (users, messages, articles)
- Concurrent requests are limited by Python's GIL

Bottlenecks

1. MMR computation: $O(r^2 \cdot t)$ per query. At 50 concurrent queries, each computing MMR on 50 results, this becomes noticeable (~2,500 iterations × cosine sims).
2. Ollama single-process inference: Ollama serves one request at a time per model (unless using vLLM or similar). Concurrent LLM requests queue.
3. In-memory indexes: All indexes must fit in RAM. For 700 docs this is fine, but if the corpus grows to 70,000 documents, the indexes (~500MB) plus TF vectors could stress a single process.

Scaling strategies

1. Read replicas for MongoDB — messages and articles are read-heavy, especially chat history.
2. Separate Ollama hosts — multiple Ollama instances behind a load balancer for concurrent LLM inference.
3. Vector index offload — for large corpora, move TF index to a document database or an embedded key-value store (e.g., LMDB).
4. Stateless Flask — the only in-process state is the engine (loaded from JSON at startup). Horizontal scaling with a load balancer is straightforward: each process loads its own engine copy (~50MB RAM).

Maintainability

Good:

- The two-processor architecture (BM25 vs proximity) is well-documented and makes the code easy to reason about
- Each file has a clear, single responsibility
- The standalone `rag_workflow.py` demo is useful for testing without the Flask server
- Consistent error-return patterns in services (`{"success": bool, "error": str, "data": ...}`)
- No clever metaprogramming, decorators (except JWT), or dynamic dispatch

Needs improvement:

- Tuple destructuring by position in `_score_document` / `_format_results` — a namedtuple would be more readable
- Some methods have 10+ parameters with no grouping (e.g., `ArticleService.create_article` with 16

parameters)

- No `__init__.py` exports for `src/core` — consumers import directly from modules
- The `_clean_body` regex in `rag_repository.py` depends on the exact output format of `flatten_constitution.py` — a format change breaks silently

Testing Quality

Current state: Partial pytest suite at `backend/temp/tests/`.

What's tested:

- `rag_workflow.py` standalone demo (manual)
- Basic API endpoints (via Postman collection in `postman/`)

What's not tested:

- Unit tests for IR engine: `BM25Scorer`, `ProximityScorer`, `SearchEngine`, `Reranker` — none of these have automated unit tests. Given that they contain the most complex algorithms in the system (BM25 math, MMR iteration, TF vector computation), this is the highest-priority testing gap.
- Integration tests with MongoDB: No automated tests for `UserService`, `MessageService`, `ArticleService`.
- Frontend tests: Not tested.
- Regression tests: No test suite that runs on every commit.

Suggestions:

1. Add pytest tests for `BM25Scorer.score()` with known inputs/outputs.
2. Add pytest tests for `ProximityScorer` with a small positional index.
3. Add pytest tests for `Reranker` with synthetic results — test RRF, MMR, and boost independently.
4. Add a test for `promote_to_articles` with a small corpus.
5. Add integration test for the `/ask` endpoint using a test client with mocked retrieval.

Coupling

From	To	Strength	Notes
<code>`QAService`</code>	<code>`RAGWorkflow`</code>	Tight (module-level global)	Single initialization at startup, global variable
<code>`RAGRepository`</code>	<code>`RetrievalWorkflow.engine.documents`</code>	Tight (attribute access)	Article lookup traverses <code>engine.documents</code>
<code>`api_controller`</code>	<code>`QAService`</code>	Loose (static methods)	Controllers don't instantiate services
<code>`auth_controller`</code>	<code>`UserService`</code>	Loose (static methods)	Standard service call
<code>`Reranker`</code>	<code>`BM25Scorer.tf_index`</code>	Loose (constructor injection)	Could reuse any TF-like dict
<code>`SearchEngine`</code>	<code>`BM25Scorer`</code> , <code>`ProximityScorer`</code>	Loose (constructor injection)	Scorers are swappable
<code>`Frontend`</code>	Backend API	Loose (REST + SSE)	Standard HTTP contract

The overall coupling is well-managed. The IR engine is the most decoupled part — it could be extracted into a standalone library. The tightest coupling is between `RAGRepository` and `RetrievalWorkflow`'s internal `.engine.documents`.

Cohesion

- High cohesion in `src/core/`: Each class does exactly one thing (BM25 scoring, proximity scoring, search orchestration, reranking). Methods within each class operate on the same data.
 - High cohesion in `src/llm/`: `RAGRepository` owns all LLM data access, `RAGFormatter` owns all string formatting, `RAGWorkflow` owns orchestration.
 - Medium cohesion in `api_controller.py`: Handles Q&A endpoints AND message CRUD endpoints. These could be split into separate controllers.
 - Medium cohesion in `user_service.py`: CRUD + authentication + JWT generation. Authentication could be separated from CRUD.
-

Summary

What to fix before release

1. Async def → sync in message/article services (easy, misleading)
2. Restrict CORS origins (easy, security)
3. Add unit tests for the IR engine core algorithms (medium, risk mitigation)

What to fix after release

1. Add a dedicated `/api/v1/search` endpoint (medium)
2. Inject documents into `RAGRepository` instead of accessing `.engine.documents` (medium)
3. Replace tuple destructuring with `namedtuples` in `_score_document` (low)
4. Remove unused CSS, deps, and clean up `responses.txt` (low)
5. Admin blueprint for admin API routes (medium)
6. Normalize `requirements.txt` to UTF-8 (low)

What not to change

- The two-processor architecture — it's the system's most important design insight
- Eager initialization with module-global workflow — keeps response times predictable
- Pure-Python math for ranking — avoids ML dependency issues
- Graceful degradation at every level — production-quality resilience

Teach Me Like the Author — Design Rationale

I chose every decision in this system. Here is why.

Why $b = 1.0$ instead of the standard 0.75?

I chose this because the Constitution of Nepal has provisions at wildly different levels of granularity. A part title might be 3 words long ("Part 3: Fundamental Rights"). A multi-clause article with sub-clauses can be 500+ words. With $b = 0.75$, long articles get an inherent advantage — they have more term occurrences even if they're less relevant to the query. With $b = 1.0$, the length penalty is proportional and full. Short, dense provisions compete fairly with verbose ones.

Inference: The standard BM25 literature recommends $b \approx 0.75$ for typical document collections. But a legal constitution is not a typical collection — it's a small, highly structured corpus with extreme length variance. The choice of $b = 1.0$ suggests the author either tested b values empirically on this specific corpus or had prior experience with legal retrieval.

Why two separate text processors instead of one?

I chose this because lemmatization and stopwords removal serve BM25 scoring but hurt proximity matching.

`bm25_processor` (lemmatize ON, stopwords REMOVED): BM25 measures term importance. "Rights" and "right" should match. Stopwords like "the", "of", "to" add noise to IDF calculations and contribute nothing to term importance. If a query has "right to education", the BM25 processor produces ["right", "education"] — the correct semantic units.

`proximity_processor` (lemmatize OFF, stopwords KEPT): Proximity scoring measures how close query words appear. "Right to education" must match exactly as three tokens in order — "right" (3 tokens) "education" would give a different distance than "right education" (1 token) without the stopwords. And lemmatization would conflate "rights" and "right", but the user typed "right", so that's what we should find in the text.

This dual-processor design is the single most important architectural insight in the system. It acknowledges that different retrieval signals need different text representations.

Directly observed in the code: `TextProcessor.__init__()` accepts `use_lemmatization` and `remove_stopwords` flags. `EngineFactory.from_artifacts()` creates two instances with opposite settings. `SearchEngine.search()` calls both.

Why RRF fusion with MMR instead of learning-to-rank?

I chose this because a legal retrieval system must be auditable and predictable. If a lawyer asks "why did this article rank #3 and not #1?", the answer must be a mathematical formula, not "the model weights learned during training." RRF fusion is transparent: "You rank #1 in BM25, #4 in proximity, and #2 in title match — your RRF score is $1/61 + 1/64 + 1/62 = 0.048$." MMR diversity prevents three articles on the same topic from dominating the top results, which is critical for legal research where breadth of coverage matters.

Additionally, learning-to-rank requires labeled training data (relevance judgments for query-document pairs). No such dataset exists for the Constitution of Nepal. Even if it did, the system would need ongoing maintenance to keep the model trained. The algorithmic approach is zero-maintenance and zero-training.

Directly observed: `reranker.py` uses only `math.sqrt` and `math.log` — no ML imports. The RRF formula, MMR formula, and boost rules are all hardcoded with explanations.

Why `recall_k = 50` and `top_k = 8`?

I chose these because with ~700 documents in the corpus, `recall_k = 50` (about 7% of the corpus) is wide enough to capture relevant documents even with imperfect queries. The reranker then selects the top 8 for the user. Eight is enough to cover different constitutional areas in a single query but not so many that the user is overwhelmed.

The relationship between `recall_k` and retrieval quality is: if the relevant article doesn't make the top 50 candidates in

Phase 1 (pure BM25 + proximity), it can never be recovered by Phase 2 reranking. For a 700-doc corpus, 50 candidates gives high recall for single-topic queries. For broader queries like "what are my rights", it's more than enough.

Inference: The author likely tested with real queries and observed that 50 candidates consistently included the target articles for known test cases. The 8:1 ratio of `recall_k:top_k` is common in two-stage retrieval systems.

Why proximity pair heuristic (all pairs ≤ 5 , adjacent > 5)?

I chose this because the cost of full cross-product grows as $O(n^2)$ where n is the number of query tokens. For "What are the fundamental rights relating to education and freedom?" (9 tokens), all-pairs would be 36 pairs, most of which have no meaningful proximity signal. Adjacent pairs (8 pairs) capture the phrase structure that matters.

The threshold of 5 tokens is empirical. Below 5, the number of pairs is small enough that all-pairs computation is cheap, and distant pairs can still carry weak signal. Above 5, adjacent pairs dominate the useful signal. A pair between "education" and "freedom" (3 tokens apart in the query) tells us nothing useful about their expected distance in the document.

Inference: This is a standard technique in proximity-based IR. The threshold of 5 is at the low end of typical values (3-10), suggesting the author prioritizes precision over recall for proximity scoring.

Why article-level promotion instead of keeping clause-level results?

I chose this because users ask about constitutional concepts ("What does the constitution say about education?"), not sub-clauses ("What does Article 31(2)(a) say?"). Displaying clause-level results would overwhelm the user with fragmented snippets. By promoting to articles, each result is a self-contained unit with a citation like "Part 3, Article 31 — Right relating to education" — something a user can understand and reference.

The tracked matched clauses allow the LLM context to be truncated to only the relevant portions, saving precious context window space. But the user sees the full article context.

Inference: The promotion logic (`promote_to_articles`) was likely added after user testing showed that clause-level results were confusing. The `_build_article_lookup()` method suggests careful work to reconstruct full articles from fragmented documents.

Why Ollama (local LLM) instead of an API-based LLM?

I chose this because the system is designed for the Nepali legal context, where internet connectivity may be unreliable and API costs may be prohibitive. Running a local model with Ollama keeps the system:

1. Offline-capable: Once the model is downloaded, no internet connection is needed for LLM features.
2. Cost-free: No per-token API charges.
3. Private: Legal queries never leave the local machine.
4. Controllable: The model can be swapped (current default: `qwen3:8b`) without changing any API credentials.

The tradeoff is inference quality — a local 8B model is less capable than GPT-4 or Claude. But the strict grounding in the system prompt ("Answer ONLY using the Context") mitigates this: the LLM's job is summarization and rephrasing, not knowledge generation.

Directly observed: `ollama_llm.py` creates a Client from env vars with fallback defaults. `RAGRepository` manages the Ollama lifecycle. Graceful degradation handles the case where Ollama is not running.

Why fire-and-forget persistence?

I chose this because the primary value to the user is the Q&A answer. Persistence to MongoDB is important for chat history but should never delay or break the response. If MongoDB is slow or temporarily unavailable, the user should still

get their answer.

The `_persist_message()` function wraps all persistence in a try/except that logs failures but never re-raises. The controller calls it after successfully generating the response. For streaming, persistence happens in the `_stream_events` generator after the "done" event is yielded to the client.

Inference: This pattern suggests the author experienced MongoDB latency issues during development. The fire-and-forget approach is pragmatic — it prioritizes the primary user experience over perfect data consistency.

Directly observed: `api_controller.py:ask()` checks if `status_code == 200` before calling `persist_message`. The `QAService.persist_message` method wraps everything in try/except.

Why JWT token version invalidation instead of blacklisting?

I chose this because blacklisting requires storing invalidated tokens (in memory or database) and checking every request against the blacklist. This adds complexity, memory pressure, and a race condition window. With token versioning:

1. No storage: The version is a single integer in the User document.
2. Immediate invalidation: Increment `token_version` and all existing JWTs (with old version) are instantly invalid.
3. No race conditions: The check is atomic — read user document, compare version, accept/reject.
4. No cleanup: Old tokens expire naturally after 12 hours.

The only cost is a database read on every authenticated request (to get the current `token_version`). For low-to-medium traffic legal research, this is acceptable.

Inference: The author likely came from a security-conscious background or had prior experience building auth systems. The `token_version` pattern is less common in small projects but is standard in enterprise auth.

Directly observed: `User.token_version` field (default=0). `decorators.py` line 53-54: if `token_version < user.token_version`:
401. `auth_controller.py` line 85: `user.token_version += 1`.

Why eager initialization instead of lazy loading?

I chose this because the first request should be fast. Loading 4 JSON files, creating two `TextProcessors`, loading `spaCy`, building the `SearchEngine`, wiring the workflow, and building the article lookup takes about 1 second. If this happened on the first request, the user would experience a 1-second delay on their first query — a poor first impression.

Eager initialization also makes failures immediate: if any index file is missing or malformed, the server refuses to start. Users find out the system is broken when they run `python app.py`, not when they submit their first question.

Inference: The author values developer experience and reliability. The 1-second startup penalty is a deliberate investment in predictable runtime behavior.

Why no embedding-based similarity for MMR?

I chose this because BM25 term-frequency vectors provide a reasonable measure of topical overlap without requiring any embedding model. Two articles that share many terms ("right", "education", "fundamental") are likely about similar topics. Cosine similarity on TF vectors captures this.

The alternative — embedding models (SBERT, Instructor, etc.) — would add:

- A model download (500MB-2GB)
- GPU dependency for reasonable latency
- Non-deterministic behavior (different model versions give different similarities)
- Cold-start latency (first request loads the model)

For a 700-doc corpus where term overlap is a reasonable proxy for topical similarity, the BM25 vector approach is sufficient.

Why k=60 in RRF instead of a smaller value?

I chose this because with only 3 signals (BM25 rank, proximity rank, title-match rank), a higher k reduces the impact of any single ranking. If k were smaller (e.g., 10), the top-ranked signal would dominate. With k=60, the three signals contribute more equally to the fused score. For a 50-result candidate set, the inverse rank contribution ranges from $1/61 \approx 0.016$ for rank 1 to $1/110 \approx 0.009$ for rank 50 — a narrower range that prevents any single signal from dominating.

This is consistent with the system's philosophy: no single signal (BM25, proximity, or title match) is trusted enough to determine ranking alone. The fusion of all three provides robustness.

Why the system prompt enforces answer style by question type?

I chose this because legal questions come in different forms and need different answer styles:

- "What is X" → concise definition
- "Who has the power to..." → identify the person/body
- "How is the President elected" → step-by-step procedure
- "Can the President dissolve Parliament" → Yes/No first, then explanation

Without this instruction, the LLM tends to produce generic paragraphs that don't match the user's expectation. A "Yes/No" question answered with a paragraph of explanation is frustrating. The system prompt adapts the answer to the question type.

Inference: The author tested the system with real users and discovered that answer presentation style significantly affected user satisfaction — leading to the detailed system prompt.

Why the standalone main() function in rag_workflow.py?

I chose this because developing the RAG pipeline without the Flask server is faster. The standalone demo:

1. Hardcodes 3 preset questions
2. Runs the full pipeline (retrieval → LLM → response)
3. Prints results to console

This allows the author to iterate on prompt engineering, retrieval parameters, and LLM integration without starting a web server, sending HTTP requests, or dealing with authentication. It's also useful for debugging — if the standalone demo works, the issue is in the Flask layer. If it doesn't, the issue is in the RAG pipeline.

Tradeoffs that were intentionally NOT chosen

I did NOT choose:

- Elasticsearch/Solr: They're overkill for a 700-doc corpus and add operational complexity (separate server process, schema management, learning curve).
- Embedding-based semantic search: Adds GPU dependency, non-deterministic results, and model maintenance burden. For legal text where exact term matching matters, BM25 + proximity is more appropriate.
- Asynchronous Flask: Not needed for the request volume. Threaded mode handles concurrent requests adequately.
- WebSockets instead of SSE: SSE is simpler (standard HTTP), easier to debug, and works through all proxies. WebSockets would need connection management.
- API key authentication instead of JWT: JWT allows token version invalidation, stateless verification, and payload embedding (user_id, email, role). API keys lack these features.
- Containerization/Docker: The project has no Dockerfile. This is a deliberate choice to keep the setup simple — a Python environment + MongoDB + npm install. Containerization can be added when deployment requirements are known.

Confidence Estimates

Claim	Confidence	Evidence
`b=1.0` for length variance in legal docs	Strong inference	The choice is deliberate (different from standard 0.75) but the rationale is not documented in comments
Two-processor architecture is the key insight	Direct observation	Code clearly shows two processors with opposite settings
FIRE-AND-FORGET persistence from MongoDB issues	Strong inference	The pattern is defensive but no comment explains why
Token version from security experience	Speculation	The implementation is solid but could come from following best practices
Standalone demo for faster iteration	Strong inference	Common practice; the demo has specific test questions
No embeddings to avoid GPU dependency	Strong inference	MMR uses BM25 vectors despite embeddings being a common alternative
`recall_k=50` from empirical testing	Strong inference	The ratio is reasonable and the numbers are rounded
Proximity pair threshold chosen from experience	Strong inference	5-token threshold is non-standard (typically 3-10) suggesting empirical tuning